

Privacy by Design: Entwicklung und Bewertung eines Offline-Sprachassistenten als Alternative zu Cloud-basierten Systemen

Moritz Rühm

Bachelor-Thesis

zur Erlangung des akademischen Grades Bachelor of Science (B.Sc.)

Studiengang Unternehmens- und Wirtschaftsinformatik

Fakultät für Informatik

Technische Hochschule Mannheim

31.05.2026

Betreuer

Prof. Kai Eckert, Technische Hochschule Mannheim

Prof. Oliver Hummel, Technische Hochschule Mannheim

Rühm, Moritz:

Privacy by Design: Entwicklung und Bewertung eines Offline-Sprachassistenten als Alternative zu Cloud-basierten Systemen / Moritz Rühm. –

Bachelor-Thesis, Mannheim: Technische Hochschule Mannheim, 2026. 85 Seiten.

Rühm, Moritz:

Privacy by Design: Development and evaluation of an offline voice assistant as an alternative to cloud-based systems / Moritz Rühm. –

Bachelor Thesis, Mannheim: University of Applied Sciences Mannheim, 2026. 85 pages.

Um die Lesbarkeit zu vereinfachen, wird auf die zusätzliche Formulierung der weiblichen Form bei Personenbezeichnungen verzichtet. Ich weise deshalb darauf hin, dass die Verwendung der männlichen Form explizit als geschlechtsunabhängig verstanden werden soll.

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Der Einsatz künstlicher Intelligenz erfolgte ausschließlich unterstützend, insbesondere zur Literaturrecherche sowie zur Überprüfung von Rechtschreibung und sprachlicher Verständlichkeit.

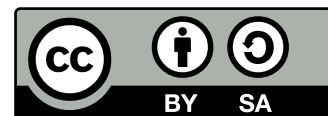
Mannheim, 31.05.2026

A handwritten signature in black ink, consisting of a large, stylized capital 'M' followed by a capital 'R' and a capital 'H'.

Moritz Rühm

Ich bin damit einverstanden, dass meine Arbeit veröffentlicht wird, d. h. dass die Arbeit elektronisch gespeichert, in andere Formate konvertiert, auf den Servern der Technischen Hochschule Mannheim öffentlich zugänglich gemacht und über das Internet verbreitet werden darf.

Dieses Werk ist lizenziert unter einer [Creative Commons „Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International“](#) Lizenz.



Abstrakt

Privacy by Design: Entwicklung und Bewertung eines Offline-Sprachassistenten als Alternative zu Cloud-basierten Systemen

Jemand musste Josef K. verleumdet haben, denn ohne dass er etwas Böses getan hätte, wurde er eines Morgens verhaftet. Wie ein Hund! sagte er, es war, als sollte die Scham ihn überleben. Als Gregor Samsa eines Morgens aus unruhigen Träumen erwachte, fand er sich in seinem Bett zu einem ungeheueren Ungeziefer verwandelt. Und es war ihnen wie eine Bestätigung ihrer neuen Träume und guten Absichten, als am Ziele ihrer Fahrt die Tochter als erste sich erhob und ihren jungen Körper dehnte. Es ist ein eigentümlicher Apparat, sagte der Offizier zu dem Forschungsreisenden und überblickte mit einem gewissermaßen bewundernden Blick den ihm doch wohl bekannten Apparat. Sie hätten noch ins Boot springen können, aber der Reisende hob ein schweres, geknotetes Tau vom Boden, drohte ihnen damit und hielt sie dadurch von dem Sprunge ab. In den letzten Jahrzehnten ist das Interesse an Künstlern sehr zurückgegangen. Aber sie überwandten sich, umdrängten den Käfig und wollten sich gar nicht fortrühren.

Abstract

Privacy by Design: Development and evaluation of an offline voice assistant as an alternative to cloud-based systems

The European languages are members of the same family. Their separate existence is a myth. For science, music, sport, etc, Europe uses the same vocabulary. The languages only differ in their grammar, their pronunciation and their most common words. Everyone realizes why a new common language would be desirable: one could refuse to pay expensive translators. To achieve this, it would be necessary to have uniform grammar, pronunciation and more common words. If several languages coalesce, the grammar of the resulting language is more simple and regular than that of the individual languages. The new common language will be more simple and regular than the existing European languages. It will be as simple as Occidental; in fact, it will be Occidental. To an English person, it will seem like simplified English, as a skeptical Cambridge friend of mine told me what Occidental is.

Inhaltsverzeichnis

Abkürzungsverzeichnis	viii
Abbildungsverzeichnis	ix
Tabellenverzeichnis	x
Quellcodeverzeichnis	xi
1 Einleitung	1
1.1 Problemstellung	1
1.2 Motivation	1
1.3 Ziel der Arbeit	1
2 Theoretische Grundlagen	2
2.1 Sprachassistenzsysteme	2
2.2 Wake-Word-Erkennung	3
2.3 Speech-To-Text	4
2.4 Large Language Models	6
2.5 Text-to-Speech	7
2.6 Datenschutzaspekte moderner Sprachassistenzsysteme	9
3 Methodik	11
3.1 Forschungsansatz: Design Science Research Methodology	11
3.2 Evaluationskonzept	12
4 Konzeption und Architektur	14
4.1 Anforderungsanalyse	14
4.1.1 Funktionale Anforderungen	14
4.1.2 Nicht-funktionale Anforderungen	18
4.2 Systemarchitektur	21
4.2.1 Modularer Aufbau	21
4.2.2 Zustandsbasierte Steuerung	22
4.2.3 Datenfluss innerhalb des Systems	24

4.3	Auswahl der Softwarekomponenten	27
4.3.1	Programmiersprache	27
4.3.2	Audioverarbeitung	28
4.3.3	Wake-Word-Erkennung	29
4.3.4	Speech-To-Text	30
4.3.5	Large Language Model	33
4.3.6	Text-To-Speech	37
5	Prototypische Umsetzung	39
5.1	Projektstruktur	39
5.1.1	Konfiguration	41
5.1.2	Interaktionsablauf	44
5.1.3	Demonstration	46
5.1.4	Demonstration	47
6	Evaluation	49
6.1	Versuche auf verschiedener Hardware	49
6.1.1	Testaufbau	50
6.1.2	Raspberry Pi 4 Model B	51
6.1.3	Asus F550CC	53
6.1.4	Macbook Air M4	55
6.1.5	Bewertung des Hardwaretests	57
6.2	Vergleich mit Cloud-basierten Lösungen und DIY-Architekturen	58
6.2.1	Alexa	58
6.2.2	Siri	60
6.2.3	Home Assistant Voice Assistant	61
6.3	Use-Cases	64
6.3.1	Smart-Home Steuerung	64
6.3.2	Pflege	64
6.3.3	Hausaufgaben Helfer	64
7	Fazit und Ausblick	65
7.1	Ergebnisinterpretation	65
7.2	Limitationen der Studie	65
7.3	Beitrag der Arbeit	65
7.4	Forschungsausblick	65
	Literatur	xii

A	Evaluationsergebnisse	xvi
B	Anhang B	xx

Abkürzungsverzeichnis

ASR	Automatic Speech Recognition
BfDI	Bundesbeauftragte für den Datenschutz und die Informationsfreiheit
DSGVO	Datenschutz-Grundverordnung
DSR	Design-Science-Research
DSRM	Design-Science-Research-Methodology
HMM	Hidden-Markov-Modell
KI	Künstliche-Intelligenz
LLM	Large Language Model
SBC	Single-Board-Computer
STT	Speech-to-Text
TTS	Text-to-Speech
VAD	Voice Activity Detection

Abbildungsverzeichnis

4.1	Zustandsdiagramm der Sprachinteraktion	23
4.2	Datenfluss innerhalb des lokalen Sprachassistenzsystems	25
5.1	Systemarchitektur des Offline-Sprachassistenten	40
5.2	Sequenzdiagramm einer Sprachinteraktion	44

Tabellenverzeichnis

4.1	Anforderung F1 – Lokale Wake-Word-Erkennung	15
4.2	Anforderung F2 – Lokale Speech-To-Text-Verarbeitung	15
4.3	Anforderung F3 – Lokale Verarbeitung durch ein LLM	16
4.4	Anforderung F4 – Lokale Sprachsynthese	16
4.5	Anforderung F5 – Begrenzung des Gesprächskontexts	17
4.6	Anforderung F6 – Modulare Systemarchitektur	17
4.7	QAS NF1 – Lokale Datenverarbeitung	18
4.8	QAS NF2 – Offlinefähigkeit	18
4.9	QAS NF3 – Ressourceneffizienz	19
4.10	QAS NF4 – Reaktionsgeschwindigkeit	19
4.11	QAS NF5 – Wartbarkeit und Nachvollziehbarkeit	20
4.12	QAS NF6 – Plattformunabhängigkeit	20
4.13	Vergleich der getesteten Speech-To-Text-Komponenten anhand eigener Te- steingaben	31
4.14	Vergleich von Whisper und faster-Whisper mit dem Large-v2-Modell auf GPU [31]	32
4.15	Vergleich von Whisper und faster-Whisper mit dem Small-Modell auf CPU [31]	32
4.16	Vergleich betrachteter lokaler Large-Language-Models	36
4.17	Vergleich betrachteter Text-To-Speech-Komponenten	38
6.1	Durchschnittliche Evaluationswerte auf dem Raspberry Pi 4 Model B . . .	52
6.2	Durchschnittliche Evaluationswerte auf dem Asus F550CC	54
6.3	Durchschnittliche Evaluationswerte auf dem MacBook Air M4	56
A.1	Einzelmesswerte der Evaluationstests auf dem Raspberry Pi 4 Model B . .	xvii
A.2	Einzelmesswerte der Evaluationstests auf dem Asus F550CC	xviii
A.3	Einzelmesswerte der Evaluationstests auf dem MacBook Air M4	xix

Quellcodeverzeichnis

Kapitel 1

Einleitung

1.1 Problemstellung

Test1

1.2 Motivation

Test2

1.3 Ziel der Arbeit

- Zeigen dass es möglich
- software bauen die später auf verschiedenster Hardware läuft
- zeigen dass es sinn hat

Kapitel 2

Theoretische Grundlagen

Moderne Sprachassistenzsysteme basieren auf dem Zusammenspiel zahlreicher komplexer Bausteine, die gemeinsam eine natürliche und effiziente Interaktion zwischen Mensch und Maschine ermöglichen [1]. Dabei kommen insbesondere Verfahren aus den Bereichen Künstliche-Intelligenz (KI), maschinellem Lernen und Sprachverarbeitung zum Einsatz [1, 2]. Für ein fundiertes Verständnis der in den folgenden Kapiteln behandelten Konzepte und Implementierungen müssen zunächst grundlegende theoretische Ansätze dieser einzelnen Elemente erläutert werden.

2.1 Sprachassistenzsysteme

Sprachassistenzsysteme, auch Voice Assistants genannt, sind softwarebasierte Systeme, die gesprochene Sprache verarbeiten, interpretieren und darauf reagieren können. Bekannte Vertreter sind beispielsweise Alexa, Siri oder der Google Assistant. Diese Systeme ermöglichen es Nutzern über natürliche Sprachbefehle mit digitalen Geräten zu interagieren und verschiedene Aufgaben, beispielsweise das Abfragen von Informationen, die Steuerung von Smart-Home-Geräten oder der Terminplanung, ohne manuelle Eingaben durchzuführen [1].

Die Entwicklung moderner Sprachassistenzsysteme wurde insbesondere durch Fortschritte im Bereich der KI sowie der digitalen Sprachverarbeitung ermöglicht [1–3]. Während frühe Systeme häufig nur auf fest definierte Befehle reagieren konnten, sind aktuelle Sprachassistenten deutlich flexibler und können unterschiedliche Formulierungen einer Anfrage verarbeiten [1]. Dadurch wirkt die Kommunikation natürlicher und benutzerfreundlicher [4].

Der grundlegende Ablauf eines Sprachassistenzsystems besteht aus mehreren Verarbeitungsschritten [3]. Zunächst wird die Sprache des Nutzers mithilfe eines Mikrofons aufgenommen und in ein digitales Audiosignal umgewandelt [1, 3]. Sobald das System ein bestimmtes Aktivierungswort, das sogenannte Wake-Word, erkennt, beginnt die eigentliche Spracherkennung [1, 5]. Hierbei wird die gesprochene Sprache in Text umgewandelt, um sie anschließend durch ein Sprachmodell interpretieren zu können [1, 2]. Das Sprachmo-

dell analysiert die Bedeutung der Anfrage und generiert eine passende Antwort, welche schließlich in eine Audioausgabe umgewandelt und dem Nutzer zurückgegeben wird [1]. Sprachassistentensysteme finden heute in zahlreichen Bereichen Anwendung [3]. Sie werden beispielsweise in Smartphones, Lautsprechersystemen, Fahrzeugen oder Smart-Home-Geräten eingesetzt [1]. Nutzer können damit Informationen abrufen, Erinnerungen erstellen oder verschiedene Funktionen sprachgesteuert bedienen [1].

2.2 Wake-Word-Erkennung

Die Wake-Word-Erkennung kennzeichnet bei sprachbasierten Assistenzsystemen die erste technische und gleichzeitig interaktive Stufe der Sprachverarbeitung. Ein Wake-Word, beispielsweise „Alexa“, „Hey Siri“ oder „Ok Google“, dient als Aktivierungssignal, mit dem ein System erkennt, dass eine nachfolgende Äußerung an den Sprachassistenten gerichtet ist [5]. Erst nach der Erkennung dieses Schlüsselwortes beginnt das System damit, die anschließende Spracheingabe als eigentlichen Befehl aufzunehmen und weiterzuverarbeiten [3]. Damit übernimmt die Wake-Word-Erkennung eine Filterfunktion und trennt alltägliche Hintergrundgespräche von Befehls- und Fragesätzen, welche an das System gerichtet sind [5].

Aus architektonischer Sicht wird die Wake-Word-Erkennung typischerweise vor der eigentlichen Speech-To-Text-Verarbeitung platziert [3]. Während automatische Transkribersysteme gesprochene Sprache in Text umwandeln und dabei akustische Modellierung, Sprachmodellierung und Dekodierung einsetzen, ist die Wake-Word-Erkennung stärker auf die robuste Erkennung eines sehr kleinen, fest definierten Vokabulars spezialisiert [3, 5]. Sie muss daher nicht jede gesprochene Äußerung vollständig verstehen, sondern lediglich entscheiden, ob ein bestimmtes Aktivierungswort im Audiosignal enthalten ist. Zur Erkennung solcher Sprachaktivität wird häufig auch Voice Activity Detection (VAD) eingesetzt, welche Sprachabschnitte innerhalb eines Audiosignals identifiziert und damit den Beginn beziehungsweise das Ende einer Spracheingabe bestimmt [5].

Technisch betrachtet muss ein Wake-Word-System dauerhaft oder nahezu dauerhaft aktiv sein, da es das Aktivierungssignal jederzeit erkennen muss [1, 5]. Daraus ergibt sich eine besondere Anforderung an Effizienz und Ressourcenschonung [1]. Im Gegensatz zur vollständigen automatischen Spracherkennung darf die Wake-Word-Erkennung nur geringe Rechenleistung benötigen, insbesondere wenn sie auf leistungsarmen Geräten wie Smart Speakern oder Single-Board-Computern (SBCs) betrieben wird. In vielen Systemen wird deshalb ein leichtgewichtiges lokales Modell eingesetzt, das kontinuierlich kurze Audiodbereiche analysiert [1]. Erst wenn dieses Modell eine ausreichende Wahrscheinlichkeit für das Wake-Word erkennt, werden rechenintensivere Verarbeitungsschritte aktiviert [5].

Auch wenn die Wake-Word-Erkennung in den meisten Fällen lokal erfolgt, entsteht bei Nutzern häufig der Eindruck eines permanent mithörenden Systems [5, 6]. Anbieter betonen zwar, dass Aufzeichnungen erst nach dem Wake-Word erfolgen [7–10], dennoch bleibt die technische Voraussetzung bestehen, dass das Gerät fortlaufend Audiodaten lokal analysiert [5, 6]. Dies führt zu einer gewissen Unsicherheit hinsichtlich Datenschutz und Privatsphäre, da Nutzer nicht immer genau wissen, welche Daten wann verarbeitet werden [5].

Für die Gestaltung moderner Sprachassistentensysteme ist die Wake-Word-Erkennung daher mehr als nur ein technisches Detail. Sie beeinflusst, wie natürlich, zuverlässig und vertrauenswürdig ein System wahrgenommen wird. Eine gute Wake-Word-Erkennung muss robust gegenüber Hintergrundgeräuschen, unterschiedlichen Stimmen, Akzenten und ähnlichen Lautfolgen sein [5]. Perspektivisch könnten deshalb Alternativen oder Ergänzungen zum klassischen Wake-Word, wie etwa kontextabhängige Aktivierung, dedizierte Gesprächsmodi oder lernende Systeme, die aus vorherigen Interaktionen ableiten, wann eine Äußerung wahrscheinlich an den Assistenten gerichtet ist, an Bedeutung gewinnen [1].

2.3 Speech-To-Text

Speech-to-Text (STT), häufig auch als Automatic Speech Recognition (ASR) bezeichnet, beschreibt die automatische Umwandlung gesprochener Sprache in geschriebenen Text. Damit bildet es eine weitere Grundlage moderner Sprachassistentensysteme [3].

Wie vorangehend bereits beschrieben, beginnt der Prozess eines Sprachassistentensystems meist bereits vor der eigentlichen Spracherkennung. Zunächst wird lokal auf das Wake-Word gewartet. Sobald dieses erkannt wird, aktiviert sich der Sprachassistent und beginnt mit der Aufnahme der Spracheingabe [5]. Ähnlich zur Wake-Word-Erkennung kommt hier auch häufig VAD zum Einsatz. Sobald eine kurze Sprechpause erkannt wird, gilt die Eingabe als beendet und die aufgenommene Audiodatei wird daraufhin für die Weiterverarbeitung meistens an einen Server gesendet [3].

Ein klassisches ASR-System besteht intern aus mehreren Komponenten: akustischer Modellierung, Sprachmodellierung und Decoding [3, 6]. Die akustische Modellierung analysiert das Sprachsignal und extrahiert Merkmale wie Tonhöhe, Intensität und spektrale Eigenschaften. Hierbei werden akustische Eigenschaften sprachlichen Einheiten wie Phonemen (gesprochenen Lauten) zugeordnet. Die Sprachmodellierung berücksichtigt dann den sprachlichen Kontext und hilft dabei, aus möglichen Laut- beziehungsweise Wortfolgen die wahrscheinlichste textuelle Interpretation zu bestimmen. Das Decoding verbindet schließlich die Ergebnisse aus akustischem Modell und Sprachmodell und erzeugt daraus die finale Textausgabe [3].

Frühe Systeme basierten häufig auf Hidden-Markov-Modellen (HMMs), die zeitliche Abläufe von Sprache modellieren konnten [3]. Modernere Ansätze verwenden dagegen Deep-Learning-Verfahren wie neuronale Netze und damit insbesondere Transformer-basierte Modelle. Diese ermöglichen eine robustere Verarbeitung natürlicher Sprache, erzielen deutlich höhere Erkennungsgenauigkeiten und repräsentieren heute die Grundlage vieler moderner Sprachassistenten [3].

Neben modularen ASR-Architekturen werden heutzutage zunehmend End-to-End-Systeme eingesetzt [6]. Während klassische Systeme akustische Modellierung, Sprachmodellierung und Decoding getrennt behandeln, lernen End-to-End-Modelle eine direkte Abbildung vom Audiosignal zum Text. Deshalb kann die Systemarchitektur deutlich vereinfacht werden. Solche Systeme erzielen heute bereits sehr gute Ergebnisse hinsichtlich Genauigkeit und Latenz, weshalb sie insbesondere für moderne Sprachassistentensysteme von hoher Relevanz sind [3].

Trotz der großen Fortschritte heutiger Systeme bestehen weiterhin verschiedenste Herausforderungen. Ein wesentliches Problem ist die Erkennungsgenauigkeit in lauten Umgebungen [1]. Hintergrundgeräusche, mehrere gleichzeitig sprechende Personen oder schlechte Mikrofonqualität können die Qualität der Transkription deutlich beeinträchtigen. Weitere Schwierigkeiten entstehen durch unterschiedliche Akzente, Dialekte, Sprechgeschwindigkeiten und Sprachen. Auch fachspezifische Begriffe, Eigennamen oder Wörter außerhalb des Trainingsvokabulars können Fehler verursachen [1, 3].

Dennoch konnte sich STT in zahlreichen Anwendungsbereichen etablieren und wird heute vielfältig genutzt [1, 3, 6]. Transkriptionsdienste ermöglichen STT die automatische Verschriftlichung gesprochener Inhalte, etwa in Medizin, Recht, Medienproduktion oder Bildung. Der Kundenservice setzt STT für sprachbasierte Chatbots, Callcenter-Automatisierung und virtuelle Serviceagenten ein. Für das Gesundheitswesen kann STT medizinische Dokumentation erleichtern, indem Ärztinnen und Ärzte Informationen per Sprache erfassen. Zudem trägt STT maßgeblich zur Barrierefreiheit bei, etwa durch automatische Untertitelung von Videos, Podcasts oder Vorlesungen für Menschen mit Hörbeeinträchtigungen [3]. In virtuellen Assistenten wie Alexa, Siri oder Google Assistant ist STT notwendig, damit gesprochene Befehle überhaupt maschinell verstanden werden können [1].

Für Sprachassistenten besitzt STT damit eine Schlüsselrolle. Die Qualität der gesamten Interaktion hängt stark davon ab, wie zuverlässig die gesprochene Eingabe erkannt wird. Fehler in der Transkription können sich auf alle nachfolgenden Verarbeitungsschritte auswirken [3]. Wird ein Befehl falsch erkannt, kann auch ein leistungsfähiges Sprachmodell keine korrekte Antwort erzeugen [2]. STT ist daher nicht nur eine technische Vorverarbeitungsstufe, sondern eine grundlegende Voraussetzung für natürliche, effiziente und nutzerfreundliche Mensch-Computer-Interaktion per Sprache [1].

2.4 Large Language Models

Large Language Models (LLMs) sind große neuronale Sprachmodelle, die natürliche Sprache analysieren und generieren können. Sie stellen eine Weiterentwicklung klassischer Sprachmodelle dar und basieren heute meist auf der Transformer-Architektur. Durch das Training auf sehr großen Textmengen lernen sie sprachliche Muster, semantische Zusammenhänge und kontextabhängige Beziehungen zwischen Wörtern und Sätzen [2].

Die Leistungsfähigkeit moderner LLMs beruht auf drei wichtigen Entwicklungen: der Transformer-Architektur, stark gesteigerter Rechenleistung und der Verfügbarkeit großer Trainingsdatensätze [2]. Während frühere Sprachmodelle häufig auf statistischen Verfahren oder kleineren neuronalen Netzen basierten, können heutige LLMs mit Milliarden von Parametern trainiert werden. Die Parameter speichern dabei keine festen Fakten wie in einer Datenbank, sondern modellieren Zusammenhänge innerhalb von Sprache. Vereinfacht ausgedrückt berechnet ein LLM, welches nächste Token in einem gegebenen Kontext wahrscheinlich ist [2]. Tokens sind dabei die kleinsten Verarbeitungseinheiten des Modells und können Wörter, Wortteile, Zeichen oder Symbole sein. Die Tokenisierung ist daher ein bedeutender Vorverarbeitungsschritt, bei dem Eingabetext in verarbeitbare Einheiten zerlegt wird [1, 2].

Der Entwicklungsprozess eines LLMs lässt sich grob in Pre-Training, Finetuning und Inferenz unterteilen. Im Pre-Training wird das Modell auf sehr großen Datenmengen selbstüberwacht trainiert [2]. Dabei lernt es, fehlende oder folgende Tokens vorherzusagen. Dieses Training vermittelt dem Modell allgemeine sprachliche und teilweise auch faktische Muster. Anschließend kann das Modell durch Finetuning an bestimmte Aufgaben oder Anwendungsbereiche angepasst werden [2]. Eine für Sprachassistenzsysteme besonders relevante Form ist das Instruction Tuning [2, 11]. Dabei wird ein Modell mit Datensätzen trainiert, die aus Anweisungen und passenden Antworten bestehen. Solche Instruction-Modelle sind nicht nur auf reine Textfortsetzung ausgelegt, sondern stärker darauf optimiert, Nutzeranfragen zu verstehen und angemessen zu beantworten. Zusätzlich kann eine Ausrichtung an menschlichen Präferenzen erfolgen, um hilfreichere, sicherere und besser steuerbare Antworten zu erzeugen [11].

Bei der Ausführung eines LLM wird zwischen Training und Inferenz unterschieden. Während das Training sehr große Datenmengen und erhebliche Rechenressourcen erfordert, bezeichnet die Inferenz die Nutzung eines bereits trainierten Modells zur Erzeugung konkreter Antworten. Für den praktischen Einsatz in einem Sprachassistenzsystem ist deshalb vor allem die Inferenz relevant. Sie bestimmt, wie schnell ein Modell auf eine Anfrage reagieren kann, wie viel Arbeitsspeicher benötigt wird und ob eine Ausführung auf lokaler Hardware realistisch möglich ist.

Für den lokalen Betrieb von LLMs ist neben der Modellarchitektur auch die effiziente Ausführung entscheidend. Große Modelle benötigen viel Arbeitsspeicher und Rechenleistung, wodurch sie auf ressourcenbegrenzter Hardware nur eingeschränkt einsetzbar sind [2]. Ein wichtiges Verfahren zur Reduzierung dieser Anforderungen ist die Quantisierung. Dabei werden die Gewichte eines Modells mit geringerer numerischer Genauigkeit gespeichert, beispielsweise statt 16-Bit- oder 32-Bit-Gleitkommazahlen mit 8-Bit- oder 4-Bit-Werten [2, 11]. Folglich sinken Speicherbedarf und Rechenaufwand, was die lokale Inferenz auf Geräten wie SBCs erleichtern kann. Andererseits führt eine zu starke Quantisierung zu schlechterer Antwortqualität, da die Modellgewichte nur noch näherungsweise repräsentiert werden [11].

Weiterhin besitzen LLMs auch klare Grenzen. Sie können falsche oder nicht belegbare Informationen erzeugen, was häufig als Halluzination bezeichnet wird. Zudem sind sie abhängig von ihren Trainingsdaten und können darin enthaltene Verzerrungen übernehmen [2]. Auch Datenschutz, Rechenaufwand, Energieverbrauch und Inferenzkosten sind relevante Herausforderungen. Eine hohe Leistungsfähigkeit großer Modelle ist mit langsamerem Training, hohen Hardwareanforderungen und höheren Betriebskosten verbunden [2].

2.5 Text-to-Speech

Text-to-Speech (TTS), auch als Sprachsynthese bezeichnet, beschreibt die automatische Umwandlung von geschriebenem Text in Sprache. Während STT gesprochenes in eine textuelle Repräsentation überführt, zeigt TTS den umgekehrten Prozess [3]. In Sprachassistentensystemen stellt diese Komponente normalerweise damit den letzten Schritt der Interaktion dar [1, 6].

Ein TTS-System soll also aus einem Eingabetext eine möglichst verständliche, natürlich klingende und ausdrucksstarke Stimme erzeugen. Dafür reicht es nicht aus, einzelne Wörter lediglich mechanisch aneinanderzureihen [3]. Natürliche Sprache besitzt neben der reinen Lautfolge weitere Eigenschaften wie Betonung, Sprechgeschwindigkeit, Pausen, Rhythmus und Intonation. Diese Merkmale werden unter dem Begriff Prosodie zusammengefasst und haben einen wesentlichen Einfluss darauf, ob eine synthetisch erzeugte Stimme natürlich wirkt. Ein Satz kann je nach Betonung oder Tonhöhe beispielsweise als Aussage, Frage oder Aufforderung wahrgenommen werden [3]. Moderne TTS-Systeme müssen daher nicht nur den Inhalt eines Textes erfassen, sondern auch geeignete prosodische Merkmale für die Sprachausgabe bestimmen [3, 6].

Der Verarbeitungsprozess eines TTS-Systems beginnt in der Regel mit der Analyse und Normalisierung des Eingabetextes. Dabei wird der Text zunächst in eine standardisierte Form überführt [3]. Dies ist notwendig, da geschriebener Text zahlreiche Elemente

enthält, die nicht direkt ausgesprochen werden können. Dazu gehören beispielsweise Zahlen, Abkürzungen, Sonderzeichen, Datumsangaben oder Interpunktion. Ein Ausdruck wie „12.05.2026“ soll nicht zeichenweise vorgelesen, sondern als Datum interpretiert werden [3, 6]. Ebenso kann eine Abkürzung abhängig vom Kontext unterschiedlich ausgesprochen werden [6].

Im Anschluss wird der normalisierte Text sprachlich weiterverarbeitet. Dazu gehört unter anderem die Zerlegung des Textes in kleinere Einheiten sowie die Bestimmung der passenden Aussprache. Ein wichtiger Schritt hierbei ist die Umwandlung von Graphemen, also geschriebenen Buchstaben oder Buchstabenkombinationen wie „a“, „sch“ oder „ei“, in Phoneme, also gesprochene Laute wie /a/, /ʃ/ oder /aɪ/. Phoneme stellen damit die kleinsten bedeutungsunterscheidenden Laute einer Sprache dar [12]. Diese Umwandlung ist notwendig, da die Schreibweise eines Wortes nicht immer eindeutig auf seine Aussprache schließen lässt. Besonders bei Fremdwörtern, Eigennamen oder mehrdeutigen Schreibweisen kann dies eine Herausforderung darstellen. Das TTS-System muss daher Regeln oder gelernte Modelle verwenden, um aus dem geschriebenen Text eine geeignete phonemische Repräsentation abzuleiten [3, 12].

Nach der linguistischen Analyse folgt die Modellierung der Prosodie. In diesem Schritt wird festgelegt, wie die synthetische Sprache klingen soll. Dazu zählen etwa Tonhöhe, Betonung, Sprechtempo, Lautstärke und Pausenstruktur. Diese Merkmale tragen entscheidend dazu bei, dass die Ausgabe nicht monoton oder künstlich wirkt [12]. Eine unnatürliche oder schlecht betonte Sprachausgabe kann die Benutzerfreundlichkeit deutlich verringern, selbst wenn der Inhalt der Antwort korrekt ist. Mit diesen Merkmalen wird anschließend die eigentliche Sprachsynthese erzeugt, die entweder direkt über Lautsprecher ausgegeben oder als Audiodatei gespeichert werden kann [1].

Historisch wurden verschiedene Ansätze zur Sprachsynthese entwickelt [3]. Bei der konkatativen Synthese werden zuvor aufgenommene Sprachsegmente, etwa einzelne Laute, Silben oder kurze Wortbestandteile, aus einer Datenbank ausgewählt und zu einer vollständigen Äußerung zusammengesetzt. Dieser Ansatz kann sehr natürlich klingende Sprache erzeugen, benötigt jedoch große Mengen an aufgenommenem Sprachmaterial und ist nur eingeschränkt flexibel. Änderungen in Tonhöhe, Betonung oder Sprechstil lassen sich nur begrenzt realisieren, da die Ausgabe stark von den vorhandenen Sprachaufnahmen abhängt [3].

Ein weiterer traditioneller Ansatz ist die Formantsynthese. Hierbei wird versucht, den menschlichen Sprechapparat beziehungsweise bestimmte akustische Eigenschaften der menschlichen Stimme mathematisch zu modellieren. Dadurch lässt sich Sprache ohne große Datenbanken erzeugen, allerdings klingt die Ausgabe häufig weniger natürlich als

bei aufnahmebasierten Verfahren. Dieser Ansatz ermöglicht detaillierte Modellierung des Sprechvorgangs, ist jedoch vergleichsweise komplex und rechenintensiv [3].

Moderne TTS-Systeme basieren daher zunehmend auf Deep-Learning-Verfahren [3, 12]. Dabei werden neuronale Netze genutzt, um die Zusammenhänge zwischen Text, Aussprache, Prosodie und akustischem Sprachsignal aus großen Datenmengen zu lernen. Diese haben wesentlich dazu beigetragen, dass synthetische Sprache heute deutlich natürlicher und flüssiger erzeugt werden kann als mit vielen klassischen Verfahren [12]. Deep-Learning-basierte TTS-Systeme gelten daher als dominierender Ansatz in vielen aktuellen Systemen [3].

Neben Sprachassistenzsystemen findet TTS auch in zahlreichen weiteren Bereichen Anwendung. Dazu zählen unter anderem Screenreader für Menschen mit Sehbeeinträchtigungen, Vorlesefunktionen für digitale Texte E-Learning-Anwendungen, Navigationssysteme und Audiobooks sowie Anwendungen im Bereich der Sprach- und Kommunikationstherapie. Ähnlich zur STT-Technologie besitzt TTS im Bereich der Barrierefreiheit eine hohe Relevanz, da geschriebene Inhalte auch für Personen zugänglich werden, die Texte nicht oder nur eingeschränkt lesen können [3].

2.6 Datenschutzaspekte moderner Sprachassistenzsysteme

Die vorangehenden Kapitel haben bereits die technischen Grundlagen moderner Sprachassistenzsysteme erläutert. Aus einzelnen Aspekten wie der dauerhaften akustischen Umgebungserfassung, der cloudbasierten Verarbeitung oder der möglichen menschlichen Auswertung von Sprachdaten ergeben sich jedoch besondere Datenschutzrisiken [1]. Ebenfalls relevant ist, dass Sprache nicht nur den Inhalt einer Anfrage enthält, sondern auch Rückschlüsse auf Identität, Alter, Geschlecht, emotionale Verfassung oder Alltagssituationen ermöglichen kann [6]. Der europäische Datenschutzausschuss stellt deshalb klar, dass bei virtuellen Sprachassistenten die Verarbeitung personenbezogener Daten zum Kern der Dienstleistung gehört und die Datenschutz-Grundverordnung (DSGVO) grundsätzlich anwendbar ist [6].

Gemäß Artikel 5 der DSGVO müssen Anbieter daher mehrere Grundprinzipien beachten. Dazu gehören Transparenz, Zweckbindung, Datenminimierung, Speicherbegrenzung, Integrität und Vertraulichkeit. Nutzerinnen und Nutzer müssen verständlich darüber informiert werden, welche Daten erhoben werden, zu welchem Zweck sie verarbeitet werden, wie lange sie gespeichert bleiben und ob Dritte Zugriff erhalten. Gerade bei Sprachassistenzsystemen ist dies schwierig, weil neben der registrierten Hauptnutzerin oder dem Hauptnutzer auch Familienmitglieder, Gäste oder zufällig anwesende Personen erfasst werden können [6]. Die Bundesbeauftragte für den Datenschutz und die Informationsfreiheit (BfDI) weist deshalb

darauf hin, dass auch andere Personen im Haushalt über die Nutzung solcher Systeme informiert werden sollten [13].

Weitere Probleme bilden sich durch die Datenweitergabe an Drittanbieter. Viele Sprachassistentensysteme lassen sich mit Smart-Home-Geräten, Apps, Skills oder externen Diensten verbinden. Dadurch entstehen komplexe Datenflüsse zwischen Plattformbetreibern, Geräteherstellern und Drittanbietern. Aus Datenschutzperspektive erhöht dies das Risiko, dass Daten für zusätzliche Zwecke genutzt, zusammengeführt oder zur Profilbildung verwendet werden [6]. Es werden insbesondere der mögliche Missbrauch gesammelter Daten durch Hersteller oder Drittentwickler als zentrales Risiko genannt [1, 6].

Auch die Speicherung in der Cloud ist datenschutzrechtlich bedeutsam. Sprachbefehle, Transkripte, Nutzungszeiten, Gerätestandorte und verknüpfte Konten können langfristig gespeichert und analysiert werden. Daraus lassen sich detaillierte Profile über Gewohnheiten, Interessen, Tagesabläufe oder Haushaltsstrukturen ableiten [6].

Für einen datenschutzfreundlichen Betrieb sind daher technische und organisatorische Schutzmaßnahmen notwendig. Dazu zählen lokale Verarbeitung möglichst vieler Sprachdaten, klare Löschfunktionen, deaktivierbare Mikrofone, kurze Speicherfristen, transparente Datenschutzeinstellungen, Verschlüsselung, Zugriffsbeschränkungen und eine verständliche Einwilligungsverwaltung [6]. Auch Nutzerinnen und Nutzer können Risiken reduzieren, indem sie Aufnahmeverläufe regelmäßig löschen, Mikrofone bei Nichtgebrauch deaktivieren, Berechtigungen von Drittanbieter-Skills prüfen und Sprachassistenten nicht in besonders sensiblen Räumen einsetzen [6]. Daher ist Datenschutz nicht nur eine rechtliche Anforderung, sondern muss bereits bei der Systemgestaltung berücksichtigt werden.

Kapitel 3

Methodik

Die vorliegende Arbeit orientiert sich am Ansatz des Design-Science-Research (DSR). Dieser ist insbesondere in der Informatik und Wirtschaftsinformatik etabliert und fokussiert die Entwicklung sowie Evaluation innovativer IT-Artefakte zur Lösung konkreter Problemstellungen [14, 15]. Solche Artefakte können unterschiedliche Ausprägungen annehmen und beispielsweise als Konstrukte, Modelle oder Methoden realisiert werden [14, 15].

Die Wahl dieses Forschungsansatzes ergibt sich aus der Zielsetzung der Arbeit, einen datenschutzfreundlichen Sprachassistenten zu konzipieren und prototypisch umzusetzen. Im Kontext aktueller, überwiegend cloudbasierter Systeme besteht die zentrale Problemstellung in der umfangreichen Verarbeitung und Speicherung sensibler Nutzerdaten auf externen Servern [6, 16]. Daraus ergibt sich die Notwendigkeit, alternative Lösungsansätze zu entwickeln, die eine stärkere Wahrung der Privatsphäre ermöglichen [16]. DSR bietet hierfür einen geeigneten methodischen Rahmen, da er explizit auf die Entwicklung und Evaluation technischer Artefakte zur Lösung realer Probleme ausgerichtet ist [14, 15].

3.1 Forschungsansatz: Design Science Research Methodology

Für die systematische Durchführung der Forschung wird die Design-Science-Research-Methodology (DSRM) nach Peffers et al. [14] angewandt. Diese Methodik stellt ein prozessorientiertes Rahmenmodell bereit, das aus sechs zentralen Aktivitäten besteht und auf den wissenschaftlichen Gütekriterien für DSR nach Hevner et al. aufbaut [14, 15].

Ausgangspunkt der DSRM ist die Identifikation eines relevanten Problems sowie die Darlegung der zugrunde liegenden Motivation [14]. Im Rahmen dieser Arbeit besteht die Problemstellung in der mangelnden Datenschutzkonformität gängiger, cloudbasierter Sprachassistenten, bei denen Sprachdaten regelmäßig an externe Server übertragen und dort verarbeitet werden [6, 16]. Die Motivation ergibt sich aus dem zunehmenden gesellschaftlichen und regulatorischen Interesse an Datenschutz sowie dem Bedarf nach alternativen, lokal arbeitenden Systemen, die eine stärkere Kontrolle über personenbezogene Daten ermöglichen.

Auf Basis der identifizierten Problemstellung werden im zweiten Schritt konkrete Zielsetzungen für das zu entwickelnde Artefakt formuliert [14]. Im Mittelpunkt stehen dabei insbesondere die vollständige Offline-Fähigkeit des Systems, die Minimierung der Datenverarbeitung außerhalb des lokalen Geräts sowie eine hinreichende funktionale Leistungsfähigkeit, um eine praktikable Alternative zu bestehenden Lösungen darzustellen. Ergänzend werden Anforderungen an Benutzerfreundlichkeit und Systemreaktionszeit berücksichtigt. Daraufhin erfolgt die Konzeption und prototypische Umsetzung des Artefakts [14]. Ziel ist die Entwicklung eines Offline-Sprachassistenten, der zentrale Funktionen wie Wakeword-Erkennung, Sprachverarbeitung sowie Antwortgenerierung lokal ausführt. Die konkrete architektonische Ausgestaltung sowie die Implementierung werden in den nachfolgenden Kapiteln detailliert beschrieben.

Als vierter Schritt wird durch die Demonstration die Anwendbarkeit des entwickelten Artefakts im Nutzungskontext aufgezeigt [14]. Hierzu wird der Prototyp in typischen Anwendungsszenarien eingesetzt, in denen Nutzer über Sprachbefehle mit dem System interagieren. Ziel ist es, die grundsätzliche Funktionsfähigkeit sowie die praktische Einsetzbarkeit des Ansatzes zu veranschaulichen [15].

Nah an die Demonstration angeknüpft wird das Artefakt dann gegen die Ursprungsproblematik evaluiert [14]. Im Zuge dessen werden die gemessenen Ergebnisse mit den anfänglichen Zielen, aber auch ähnlichen Lösungen verglichen [14]. Im Falle des Sprachassistenten können beispielsweise die Antwortzeiten verschiedener Hardware gegen existierenden Cloud-Lösungen verglichen werden. Weiterhin werden im Evaluationsteil auch verschiedenste Use Cases für das Produkt vorgestellt und die Funktion des Sprachassistenten in diesen erläutert.

Abschließend werden im sechsten und letzten Schritt die Ergebnisse der Arbeit sowie die Bedeutung des entwickelten Artefakts für die Forschung und Praxis kommuniziert [14]. Dies beinhaltet unter anderem eine kritische Diskussion über die Limitationen, wie die begrenzte Rechenleistung bestimmter Hardware, sowohl auch der Herausforderungen, wie Modellaktualisierungen [15].

3.2 Evaluationskonzept

Die Evaluation dient der systematischen Bewertung des entwickelten Offline-Sprachassistenten hinsichtlich der zuvor definierten Zielsetzungen [14]. Im Fokus steht insbesondere die Untersuchung, inwieweit das Artefakt eine datenschutzfreundliche Alternative zu cloud-basierten Sprachassistenten darstellen kann und gleichzeitig eine ausreichende funktionale Leistungsfähigkeit bietet.

Orientiert wird sich an den im Rahmen der DSRM formulierten Anforderungen, wobei die Evaluation sowohl technische als auch qualitative Bewertungskriterien umfasst. Hierzu werden verschiedene Nutzungsszenarien definiert, in denen der Sprachassistent unter realitätsnahen Bedingungen getestet wird.

Ein zentrales Evaluationskriterium stellt die technische Leistungsfähigkeit des Systems dar. Hierbei werden insbesondere die Antwortzeiten des Sprachassistenten, die Stabilität der Verarbeitungskette sowie die Ressourcenanforderungen auf unterschiedlicher Hardware untersucht. Die Messung der Antwortzeiten erfolgt anhand definierter Sprachbefehle, wobei die Zeitspanne zwischen Spracheingabe und Systemantwort erfasst wird.

Ergänzend wird die funktionale Qualität des Systems bewertet. Hierzu wird untersucht, ob zentrale Funktionen wie Wakeword-Erkennung, Sprachtranskription und Antwortgenerierung zuverlässig ausgeführt werden. Die Bewertung erfolgt anhand derselben exemplarischen Sprachbefehle aus der vorangehenden Evaluation.

Zur Einordnung der Ergebnisse wird letztlich ein Vergleich mit bestehenden cloudbasierten Sprachassistenten und DIY-Architekturen durchgeführt. Dabei werden insbesondere Unterschiede hinsichtlich Datenschutz, Reaktionszeit und funktionalem Umfang betrachtet. Ziel des Vergleichs ist nicht die vollständige Gleichwertigkeit mit kommerziellen Systemen, sondern die Bewertung, inwieweit ein lokal arbeitender Sprachassistent eine praktikable Alternative darstellen kann.

Kapitel 4

Konzeption und Architektur

Die Konzeption und Architektur des Offline-Sprachassistenten sind ein essenzieller Teil der Entwicklung und stellen dar, welche Anforderungen sich für einen lokalen und datenschutzfreundlichen Assistenten ergeben und wie diese in eine geeignete Systemarchitektur überführt werden können. Damit werden sie zur Verbindung zwischen den theoretischen Grundlagen und der späteren prototypischen Umsetzung und legen fest, wie einzelne Komponenten zusammenwirken, der Ablauf einer Sprachinteraktion gesteuert wird und nach welchen Kriterien die eingesetzten Technologien ausgewählt werden.

4.1 Anforderungsanalyse

Durch eine Anforderungsanalyse kann eine effektive Basis für die spätere Auswahl geeigneter Technologien sowie für architektonische und technische Entscheidungen innerhalb der Implementierung geschaffen werden. Anhand der Motivation einen möglichst datenschutzfreundlichen und lokal betriebenen Sprachassistenten zu entwickeln, ergeben sich besondere Anforderungen hinsichtlich Datenschutz, Offlinefähigkeit, Ressourcenverbrauch und praktischer Nutzbarkeit. Diese Anforderungen werden folgend in funktionale Anforderungen, und nicht-funktionale Anforderungen unterteilt.

4.1.1 Funktionale Anforderungen

Funktionale Anforderungen beschreiben die konkreten Fähigkeiten und Funktionen, die der Sprachassistent bereitstellen muss, um definierte Anwendungsfälle zu erfüllen. Hierbei ergeben sich für das Artefakt insgesamt sechs dedizierte Anforderungen, die mithilfe des Vore-Requirements-Specification-Templates [17] von Suzanne Robertson et al. strukturiert dargestellt werden.

Tabelle 4.1: Anforderung F1 – Lokale Wake-Word-Erkennung

Nr	F1	Art	F	Prio	Hoch
Titel	Lokale Wake-Word-Erkennung				
Herkunft	Nutzer				
Beschreibung					
Das System muss dauerhaft lokal auf ein definiertes Wake-Word hören können. Nach der Erkennung des Wake-Words soll automatisch die Aufnahme der Spracheingabe gestartet werden. Zusätzlich muss erkannt werden, wann der Nutzer seine Eingabe beendet hat, sodass die Aufnahme selbstständig gestoppt werden kann.					
Fit-Kriterium					
Das System erkennt das definierte Wake-Word zuverlässig und startet anschließend automatisch die Sprachaufnahme. Nach Beendigung der Spracheingabe wird die Aufnahme selbstständig beendet, ohne dass eine manuelle Interaktion erforderlich ist.					

Tabelle 4.2: Anforderung F2 – Lokale Speech-To-Text-Verarbeitung

Nr	F2	Art	F	Prio	Hoch
Titel	Lokale Speech-To-Text-Verarbeitung				
Herkunft	Architekturkonzept				
Beschreibung					
Die aufgenommene Spracheingabe muss mithilfe eines lokalen STT-Systems in Text umgewandelt werden. Dabei soll der Sprachassistent typische Spracheingaben zuverlässig verarbeiten können. Die erzeugte Transkription dient als Grundlage für die weitere Verarbeitung innerhalb des Systems.					
Fit-Kriterium					
Gesprochene Spracheingaben werden lokal verarbeitet und als verständliche Texttranskription in einem Chat-Fenster ausgegeben. Die erzeugte Transkription kann anschließend erfolgreich durch weitere Systemkomponenten weiterverarbeitet werden.					

Tabelle 4.3: Anforderung F3 – Lokale Verarbeitung durch ein LLM

Nr	F3	Art	F	Prio	Hoch
Titel	Lokale Verarbeitung durch ein LLM				
Herkunft	Architekturkonzept				
Beschreibung					
Zur Interpretation der Nutzereingabe muss die Generierung einer passenden Antwort durch ein lokal ausführbares LLM erfolgen. Dieses soll die Sprachverarbeitung auf demselben Gerät gewährleisten, ohne dabei Daten an externe Dienste zu versenden.					
Fit-Kriterium					
Das System erzeugt auf Basis der Nutzereingabe verständliche und kontextbezogene Antworten. Frühere Interaktionen werden teilweise berücksichtigt. Die gesamte Verarbeitung erfolgt lokal ohne Nutzung externer Cloud-Dienste.					

Tabelle 4.4: Anforderung F4 – Lokale Sprachsynthese

Nr	F4	Art	F	Prio	Hoch
Titel		Lokale Sprachsynthese			
Herkunft		Architekturkonzept			
Beschreibung					
Das System muss generierte Antworten mithilfe eines lokalen TTS-Systems in gesprochene Sprache umwandeln können. Die Ausgabe soll vollständig lokal erzeugt werden, um eine durchgängige Offlinefähigkeit sicherzustellen. Dabei muss die Sprachausgabe verständlich und mit möglichst geringer Verzögerung erfolgen.					
Fit-Kriterium					
Textantworten werden in verständliche Sprache umgewandelt und mit geringer Verzögerung verständlich ausgegeben. Die Sprachsynthese erfolgt vollständig lokal.					

Tabelle 4.5: Anforderung F5 – Begrenzung des Gesprächskontexts

Nr	F5	Art	F	Prio	Mittel
Titel		Begrenzung des Gesprächskontexts			
Herkunft		Nutzer			
Beschreibung		Vergangene Nutzeranfragen und die dazugehörigen Antworten des Sprachassistenten dürfen bei der Generierung neuer Antworten nur im selben konsekutiven Gespräch berücksichtigt werden. Diese Anfragen müssen daraufhin dauerhaft gelöscht werden und dürfen anschließend nicht wieder aufrufbar sein. Eine langfristige Speicherung benutzerrelevanter Informationen ist ausschließlich über den System-prompt zulässig.			
Fit-Kriterium		Nach Beendigung eines konsekutiven Gesprächs müssen alle während dieses Gesprächs gespeicherten Nutzeranfragen und Antworten automatisch gelöscht werden. Bei einer neuen Sitzung darf das System keine Inhalte aus vorherigen Gesprächen zur Antwortgenerierung verwenden oder wieder aufrufen können. In einem Test muss nachgewiesen werden, dass frühere Gesprächsinhalte weder im Kontext der Antwortgenerierung enthalten sind noch über Systemfunktionen abrufbar sind. Langfristig verfügbare benutzerrelevante Informationen dürfen ausschließlich aus dem System-prompt stammen.			

Tabelle 4.6: Anforderung F6 – Modulare Systemarchitektur

Nr	F6	Art	F	Prio	Mittel
Titel		Modulare Systemarchitektur			
Herkunft		Architekturkonzept			
Beschreibung		Der Prototyp soll modular aufgebaut sein, sodass einzelne Komponenten wie Wake-Word-Erkennung, STT, LLM oder TTS unabhängig voneinander ausgetauscht oder erweitert werden können. Dadurch soll die Architektur flexibel bleiben und unterschiedliche Modelle oder Optimierungen unterstützen.			
Fit-Kriterium		Einzelne Komponenten des Systems können unabhängig voneinander ersetzt oder erweitert werden, ohne dass die gesamte Systemarchitektur angepasst werden muss.			

Da der Sprachassistent im Rahmen der Arbeit nur prototypisch umgesetzt wird, liegt der Fokus primär auf der Umsetzung der grundlegenden Verarbeitungskette eines lokal betriebenen Sprachassistentensystems. Erweiterte Funktionen kommerzieller Systeme, wie komplexe Smart-Home-Integrationen, Benutzerprofile oder Internetrecherchen, sind nicht Bestandteil der funktionalen Anforderungen des Prototyps.

4.1.2 Nicht-funktionale Anforderungen

Neben den funktionalen Anforderungen spielen auch nicht-funktionale Anforderungen eine zentrale Rolle bei der Entwicklung des Prototyps. Diese beschreiben nicht die konkreten Funktionen des Systems, sondern die qualitativen Eigenschaften und Rahmenbedingungen, unter denen der Sprachassistent betrieben werden soll und werden hier mit dem Quality-Attribute-Scenario Template [18] nach Len Bass et al. beschrieben.

Tabelle 4.7: QAS NF1 – Lokale Datenverarbeitung

Nr	NF1	Art	QAS	Prio	Hoch
Titel		Lokale Datenverarbeitung			
Quelle		Entwickler			
Stimulus		Der Benutzer startet eine Sprachinteraktion			
Artefakt		Sprachassistentensystem			
Umgebung		Das System befindet sich im normalen Betriebszustand			
Antwort		Alle Sprachdaten werden ausschließlich lokal verarbeitet und nicht an externe Dienste übertragen			
Maß für Antwort		Keine externe Datenübertragung			

Tabelle 4.8: QAS NF2 – Offlinefähigkeit

Nr	NF2	Art	QAS	Prio	Hoch
Titel		Offlinefähigkeit			
Quelle		Endbenutzer			
Stimulus		Das System wird ohne Internetverbindung genutzt			
Artefakt		Sprachassistentensystem			
Umgebung		Es besteht keine aktive Netzwerkverbindung			
Antwort		Der Sprachassistent bleibt weiterhin nutzbar			
Maß für Antwort		Alle Kernfunktionen sind offline verfügbar			

Tabelle 4.9: QAS NF3 – Ressourceneffizienz

Nr	NF3	Art	QAS	Prio	Hoch
Titel		Ressourceneffizienz			
Quelle		Entwickler			
Stimulus		Der Sprachassistent wird auf ressourcenbeschränkter Hardware ausgeführt			
Artefakt		Sprachassistentensystem			
Umgebung		Das System läuft auch auf kompakter Hardware			
Antwort		Die Sprachverarbeitung bleibt stabil und nutzbar			
Maß für Antwort		Ausführung auf Geräten wie dem Raspberry Pi möglich			

Tabelle 4.10: QAS NF4 – Reaktionsgeschwindigkeit

Nr	NF4	Art	QAS	Prio	Hoch
Titel		Reaktionsgeschwindigkeit			
Quelle		Benutzer			
Stimulus		Der Benutzer stellt eine Sprachanfrage			
Artefakt		Sprachassistentensystem			
Umgebung		Das System befindet sich im normalen Betriebszustand			
Antwort		Eine Antwort wird ohne lange Verzögerung ausgegeben			
Maß für Antwort		Antwort innerhalb weniger Sekunden			

Tabelle 4.11: QAS NF5 – Wartbarkeit und Nachvollziehbarkeit

Nr	NF5	Art	QAS	Prio	Mittel
Titel		Wartbarkeit und Nachvollziehbarkeit			
Quelle		Entwickler			
Stimulus		Die Implementierung wird erweitert oder evaluiert			
Artefakt		Softwarearchitektur			
Umgebung					
Der Quellcode und die Dokumentation liegen vollständig vor					
Antwort					
Die einzelnen Verarbeitungsschritte sind nachvollziehbar dokumentiert					
Maß für Antwort					
Klare modulare Struktur der Implementierung					

Tabelle 4.12: QAS NF6 – Plattformunabhängigkeit

Nr	NF6	Art	QAS	Prio	Mittel
Titel	Plattformunabhängigkeit				
Quelle	Entwickler				
Stimulus	Der Sprachassistent wird auf unterschiedlicher Hardware ausgeführt				
Artefakt	Sprachassistentensystem				
Umgebung					
Das System wird auf verschiedenen Plattformen bereitgestellt					
Antwort					
Der Prototyp bleibt auf unterschiedlichen Geräten lauffähig					
Maß für Antwort					
Ausführung auf Raspberry Pi und Desktop-Systemen möglich					

Auch bei den nicht-funktionalen Anforderungen liegt der Fokus auf den für lokal betriebenen Sprachassistenten wesentlichen Funktionen. Erweiterte Aspekte wie die Anpassbarkeit oder die Sicherheit des Systems sind zwar relevant, werden jedoch im Rahmen der vorliegenden Arbeit nicht explizit als Anforderungen formuliert, da sie den Rahmen des Prototyps überschreiten und für die Bewertung der grundlegenden Umsetzbarkeit nicht zentral sind.

4.2 Systemarchitektur

Die gewählte Architektur ergibt sich unmittelbar aus den zuvor formulierten funktionalen und nicht-funktionalen Anforderungen. Da der Sprachassistent vollständig lokal betrieben werden soll, müssen alle Verarbeitungsschritte auf dem Endgerät ausführbar sein und ohne externe Dienste zusammenwirken. Gleichzeitig erfordert der Umgang mit potenziell sensiblen Sprachdaten eine klare Begrenzung der Datenweitergabe zwischen den Komponenten. Aus diesem Grund wurde eine modulare Architektur mit zustandsbasierter Ablaufsteuerung verwendet. Die Modularisierung ermöglicht die getrennte Betrachtung und den Austausch einzelner Verarbeitungskomponenten, während eine State-Machine den Ablauf der Sprachinteraktion in klar definierte Phasen unterteilt.

Folgend wird die Architektur des Prototyps näher beschrieben und erläutert, wie das System grundsätzlich aufgebaut ist, wie die Verarbeitung einer Nutzereingabe gesteuert wird und wie die Daten innerhalb des Systems weitergegeben werden.

4.2.1 Modularer Aufbau

Um die Austauschbarkeit und damit auch die Testbarkeit einzelner Komponenten zu gewährleisten, muss das System modular aufgebaut sein. Alle einzelnen Verarbeitungsschritte sind dabei klar voneinander getrennt und haben feste Inputs und Outputs. Daraus entsteht eine Architektur, in der jede Komponente eine eindeutig abgegrenzte Aufgabe übernimmt und nur über definierte Schnittstellen mit den nachfolgenden Verarbeitungsschritten verbunden ist. Der Prototyp besteht dabei aus sechs für Sprachassistentensysteme typischen Modulen: Audioeingabe (`audio_input.py`), Wake-Word-Erkennung (`wakeword_detector.py`), Speech-To-Text (`stt.py`), LLM-Inferenz (`llm.py`), Text-To-Speech (`tts.py`) und der zentralen Ablaufsteuerung (`runner.py`).

Ein wesentlicher Vorteil dieser Modularisierung besteht darin, dass einzelne Komponenten unabhängig voneinander verändert oder ersetzt werden können. Beispielsweise kann ein anderes STT-Modell eingesetzt werden, solange dieses weiterhin eine textuelle Transkription als Ausgabe liefert. Ebenso kann die Sprachsynthese durch ein anderes TTS-System ersetzt werden, ohne dass die vorherigen Schritte der Verarbeitung angepasst werden müssen. Diese Eigenschaft ist besonders für die spätere Auswahl der konkreten Softwarekomponenten relevant, da dann unterschiedliche Modelle, Konfigurationen und Hardwareplattformen miteinander verglichen werden können.

Neben der Austauschbarkeit unterstützt der modulare Aufbau auch die Nachvollziehbarkeit und Wartbarkeit des Systems. Da jeder Verarbeitungsschritt eine klar abgegrenzte Funktion besitzt, lassen sich Fehler leichter einer bestimmten Komponente zuordnen. Tritt beispielsweise eine fehlerhafte Antwort des Sprachassistenten auf, kann getrennt betrachtet werden,

ob die Ursache bereits in der Audioaufnahme, in der Transkription, in der Antwortgenerierung oder erst in der Sprachausgabe liegt. Die Architektur erleichtert damit nicht nur die technische Umsetzung, sondern auch die spätere Evaluation des Prototyps.

4.2.2 Zustandsbasierte Steuerung

Die Verarbeitung einer Sprachinteraktion ist kein einzelner linearer Programmablauf, sondern wird über eine zustandsbasierte Steuerung (State-Machine) organisiert. Damit befindet sich der Sprachassistent zu jedem Zeitpunkt in genau einem definierten Zustand. Jeder dieser Zustände zeigt, welche Aufgabe das System aktuell ausführt und welche Ereignisse einen Wechsel in den nächsten Zustand auslösen. Diese Struktur macht den Ablauf der Interaktion nachvollziehbar und verhindert, dass Verarbeitungsschritte unkontrolliert oder in falscher Reihenfolge ausgeführt werden.

Das Zustandsdiagramm in [Abbildung 4.1](#) repräsentiert den grundsätzlichen Ablauf einer vollständigen Interaktion. Nach dem Start befindet sich das System zunächst im Zustand LISTENING. In diesem Zustand wird kontinuierlich lokal auf das definierte Wake-Word gewartet. Erst wenn dieses erkannt wurde, wechselt das System in den Zustand RECORDING. Es wird demzufolge sichergestellt, dass die eigentliche Sprachaufnahme nicht dauerhaft aktiv ist, sondern erst nach einer bewussten Aktivierung beginnt.

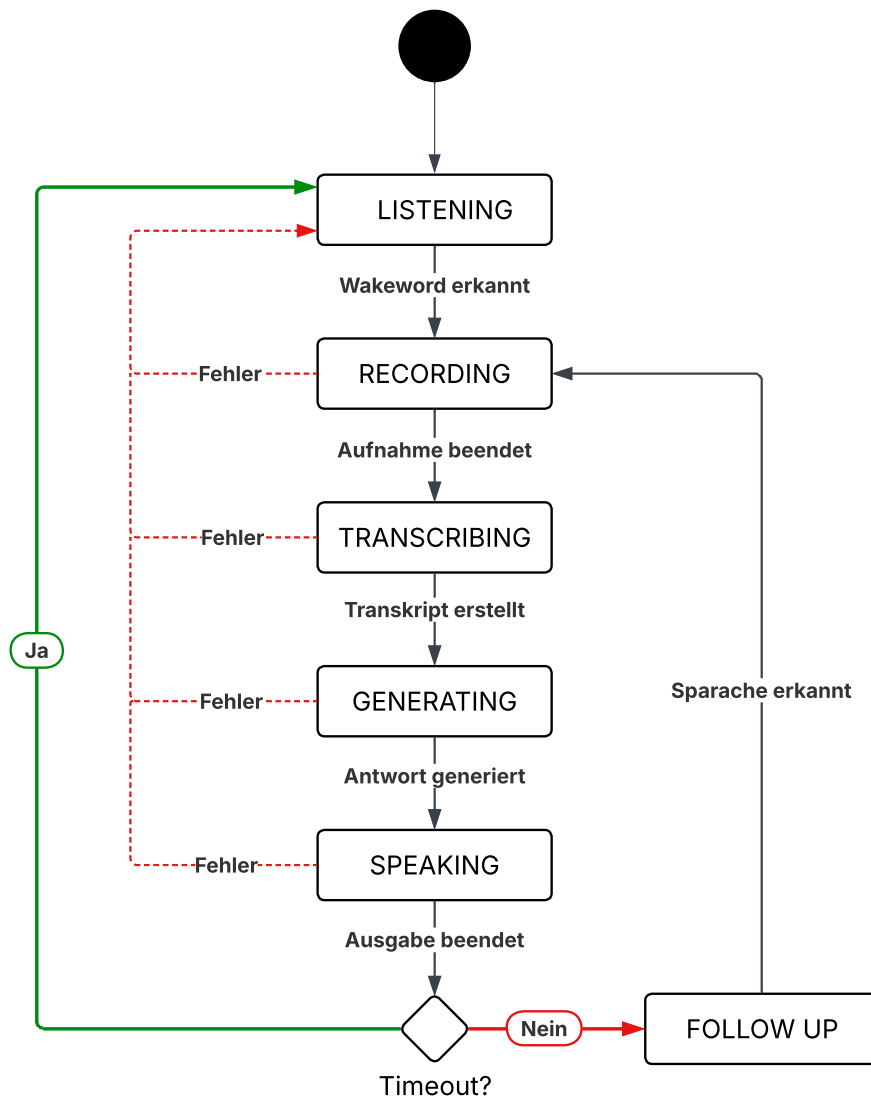


Abbildung 4.1: Zustandsdiagramm der Sprachinteraktion

Im Zustand **RECORDING** wird die Spracheingabe des Nutzers aufgezeichnet. Sobald das Ende der Eingabe erkannt wurde, wird die Aufnahme beendet und an den nächsten Verarbeitungsschritt übergeben. Anschließend folgt der Zustand **TRANSCRIBING**, in dem die aufgenommene Audiodatei in eine textuelle Repräsentation umgewandelt wird.

Nach erfolgreicher Transkription wechselt das System in den Zustand **GENERATING**. In diesem Zustand wird die Nutzereingabe an das LLM übergeben, welches eine passende Antwort generiert. Sobald die Antwort vorliegt, kann in den Zustand **SPEAKING** gewechselt werden. Dort wird die generierte Textantwort mithilfe der lokalen Sprachsynthese in gesprochene Sprache umgewandelt und ausgegeben. Die Zustandsfolge bildet damit die zen-

trale Verarbeitungskette des Sprachassistenten ab: Aktivierung, Aufnahme, Transkription, Antwortgenerierung und Sprachausgabe.

Durch diese sequentielle Zustandsfolge verarbeitet der Prototyp zu einem Zeitpunkt jeweils nur einen zentralen Interaktionsschritt. Während sich das System im Zustand SPEAKING befindet, wird daher keine neue Nutzereingabe aufgenommen oder ausgewertet. Eine Unterbrechung der laufenden Sprachausgabe durch eine neue Frage ist mit einer zustandsbasierten Architektur daher nur schwer umzusetzen. Diese Einschränkung reduziert die Komplexität der Ablaufsteuerung und verhindert konkurrierende Zugriffe auf Audioeingabe und Audioausgabe, begrenzt jedoch die Natürlichkeit der Interaktion bei längeren Antworten. Nach Abschluss der Sprachausgabe entscheidet das System, ob ein weiterer Gesprächsbeitrag erwartet wird und ein fortlaufendes Gespräch gewünscht ist. Wird innerhalb einer festgelegten Zeitspanne keine weitere Spracheingabe erkannt, kehrt der Sprachassistent in den Zustand LISTENING zurück und wartet erneut auf das Wake-Word. Wird dagegen eine Folgeeingabe erkannt, wechselt das System in den Zustand FOLLOWUP. Dieser Zustand ermöglicht die zusammenhängende Interaktion, ohne dass das Wake-Word für jede einzelne Nachfrage erneut ausgesprochen werden muss. Die Aufnahme beginnt dann erneut und die Verarbeitungskette wird von RECORDING aus fortgesetzt.

Für die Robustheit des Systems ist außerdem bedeutend, dass Fehler in den einzelnen Verarbeitungsschritten nicht gleich zum Abbruch des gesamten Programms führen. Tritt während der Aufnahme, Transkription, Antwortgenerierung oder Sprachausgabe ein Fehler auf, wird der aktuelle Verarbeitungsvorgang beendet und das System in einen stabilen Ausgangszustand zurückgeführt. Der Sprachassistent bleibt weiterhin nutzbar, auch wenn einzelne Komponenten temporär fehlschlagen oder keine verwertbaren Ergebnisse liefern. Die zustandsbasierte Steuerung unterstützt insofern mehrere zentrale Anforderungen des Prototyps. Sie verbessert die Nachvollziehbarkeit des Interaktionsablaufs, begrenzt die aktive Verarbeitung auf klar definierte Phasen und erleichtert die Erweiterung des Systems um zusätzliche Zustände oder alternative Verarbeitungswege. Gleichzeitig trägt sie zur datenschutzfreundlichen Gestaltung bei, da klar bestimmt ist, wann Audiodaten erfasst, verarbeitet und wieder verworfen werden. Die sequentielle Verarbeitung der Zustände stellt jedoch auch eine bewusste Vereinfachung dar, die insbesondere bei parallelen Interaktionsformen, wie dem Unterbrechen einer laufenden Sprachausgabe, an Grenzen stößt.

4.2.3 Datenfluss innerhalb des Systems

Neben dem modularen Aufbau und der zustandsbasierten Steuerung ist auch der Datenfluss innerhalb des Systems für die Architektur des Prototyps sehr wichtig. Er beschreibt, welche Daten während einer Sprachinteraktion entstehen, zwischen welchen Komponenten sie weitergegeben werden und an welchen Stellen eine Speicherung erfolgt. Für einen

datenschutzfreundlichen Sprachassistenten ist diese Betrachtung zentral, da Sprachdaten, Transkriptionen und generierte Antworten potenziell sensible Informationen enthalten können. [Abbildung 4.2](#) legt diesen Datenfluss innerhalb des lokalen Sprachassistentensystems dar.

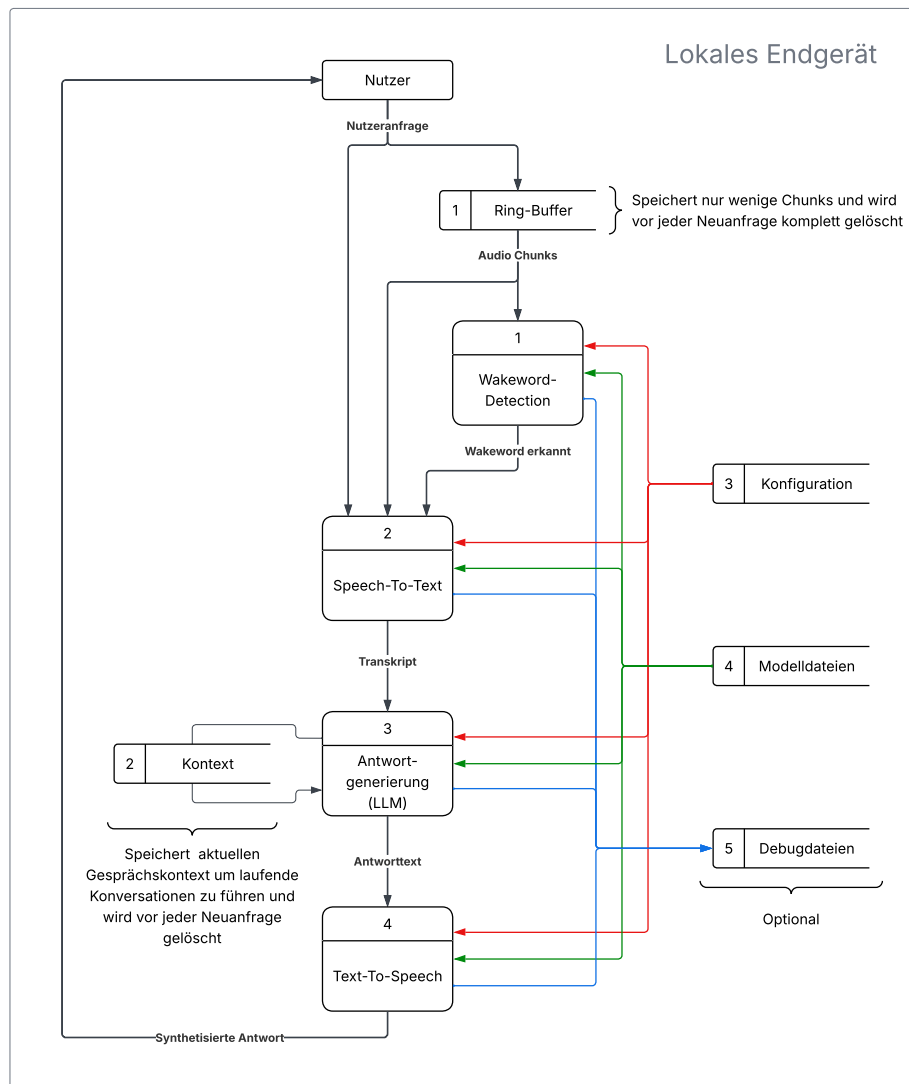


Abbildung 4.2: Datenfluss innerhalb des lokalen Sprachassistentensystems

Die kontinuierliche Aufnahme des Mikrofons gelangt zunächst in einen Ring-Buffer. Dieser speichert immer nur eine kleine, festgelegte Anzahl eingehender sogenannter Audio-Chunks. Der Ring-Buffer dient dabei nicht der dauerhaften Speicherung von Sprachdaten, sondern ermöglicht lediglich, den unmittelbar vor und während der Wake-Word-Erkennung erfassten Audiobereich kurzfristig vorzuhalten und an die restliche Aufnahme anzuhängen. Bei

jedem neuen Durchlauf des Buffers werden ältere Audiodaten überschrieben, sodass keine fortlaufende Audiodatei entsteht.

Solange im Ring-Buffer kein Wake-Word erkannt wird, verbleibt die Verarbeitung in dieser ersten Phase. Erst wenn das Wake-Word erkannt wurde, wird die nachfolgende Verarbeitungskette aktiviert und die Aufnahme der Spracheingabe gestartet. Es wird so sichergestellt, dass nicht jede akustische Umgebungssituation vollständig transkribiert oder weiterverarbeitet wird.

Nach der Aktivierung wird die aufgenommene Spracheingabe, sowie die verbliebenen Audio-Chunks des Ring-Buffers an die Speech-To-Text-Komponente übergeben. Diese verarbeitet den aufgenommenen Audiostream und erzeugt daraus ein textuelles Transkript. Damit verändert sich die Datenform von einer Audiodatei zu einer schriftlichen Version der Nutzeranfrage und ermöglicht die weitere Verarbeitung durch das LLM.

Für die folgende Antwortgenerierung wird neben der aktuellen Nutzeranfrage auch ein begrenzter Gesprächskontext berücksichtigt. Dieser Kontext enthält ausschließlich Informationen aus der laufenden Interaktion und dient dazu, zusammenhängende Folgefragen innerhalb derselben Konversation beantworten zu können. Der Kontext wird dabei nicht dauerhaft gespeichert, sondern vor jeder neuen Anfrage beziehungsweise nach Abschluss des Gesprächs gelöscht, lässt den Sprachassistenten damit aber kurzfristig auf den Verlauf einer aktuellen Unterhaltung Bezug nehmen, ohne eine langfristige Gesprächshistorie aufzubauen.

Das lokale LLM verarbeitet das Transkript gemeinsam mit dem aktuellen Gesprächskontext und erzeugt daraus einen Antworttext. Diese Antwort wird anschließend an die Text-To-Speech-Komponente weitergegeben. Die Sprachsynthese wandelt den Text in dann eine synthetisierte Sprachausgabe um, die schließlich an den Nutzer zurückgegeben wird.

Neben den unmittelbar während der Interaktion entstehenden Daten werden in [Abbildung 4.2](#) auch statische und optionale Datenquellen aufgeführt. Die Konfigurationsdatei stellt den Komponenten zentrale Parameter bereit, wie etwa Modellpfade, Geräteeinstellungen oder Schwellenwerte. Modelldateien werden lokal geladen und von den jeweiligen Komponenten verwendet. Diese Datenquellen sind für die Ausführung des Systems notwendig, enthalten jedoch keine während der Nutzung erzeugten Sprachinhalte.

Optional können Debug-Dateien erzeugt werden. Diese werden nicht im normalen Betriebsmodus erzeugt, sondern können ausschließlich über einen gesonderten Debugmodus aktiviert werden. Das dient der späteren Entwicklung, Fehlersuche und Evaluation des Systems, etwa um einzelne Verarbeitungsschritte nachvollziehbar zu prüfen oder Messergebnisse zu dokumentieren. Da Debug-Dateien je nach Konfiguration auch Audiodaten, Transkriptionen oder Verarbeitungsergebnisse enthalten können, sind sie nicht für den dauerhaften Regelbetrieb vorgesehen.

Die Darstellung verdeutlicht, dass die Verarbeitung innerhalb des lokalen Endgeräts stattfindet. Die einzelnen Komponenten erhalten jeweils nur die Daten, die für ihren Verarbeitungsschritt erforderlich sind. Audiodaten werden für die Aktivierung und Transkription genutzt, das LLM verarbeitet textuelle Eingaben und Kontextinformationen, während die Text-To-Speech-Komponente ausschließlich den generierten Antworttext benötigt. Auch Konfigurations- und Modelldateien liegen lokal vor. Es werden keine Daten an Dienste außerhalb des Geräts übertragen, was die zentralen Anforderungen der lokalen Datenverarbeitung und Offlinefähigkeit unterstützt.

4.3 Auswahl der Softwarekomponenten

Für die Umsetzung des Offline-Sprachassistenten wurden bestehende Open-Source-Komponenten eingesetzt, anstatt alle Teilfunktionen vollständig neu zu entwickeln. Diese Entscheidung ergibt sich aus dem prototypischen Charakter der Arbeit sowie aus der technischen Komplexität moderner Sprachverarbeitung. Komponenten wie Wake-Word-Erkennung, Speech-To-Text, LLM-Inferenz und Text-To-Speech basieren jeweils auf spezialisierten Verfahren, deren vollständige Eigenentwicklung den Rahmen der Arbeit deutlich überschreiten würde.

Der Einsatz etablierter Open-Source-Software ermöglicht es auch, den Schwerpunkt auf die Integration der Verarbeitungskette, die lokale Ausführung und die Bewertung der Architektur zu legen. Entscheidend ist dabei die Auswahl geeigneter Werkzeuge, die den Anforderungen an Datenschutz, Offlinefähigkeit, Ressourcenverbrauch und praktische Nutzbarkeit entsprechen. Die gewählten Komponenten müssen daher lokal ausführbar sein, klar in die modulare Systemarchitektur integrierbar sein und auf den vorgesehenen Hardwareplattformen grundsätzlich betrieben werden können.

4.3.1 Programmiersprache

Für die generelle Umsetzung wurde die Programmiersprache Python [19] verwendet. Da der Prototyp mehrere spezialisierte Komponenten miteinander verbindet, ist eine Programmiersprache erforderlich, die eine einfache Integration unterschiedlicher Softwarebibliotheken ermöglicht und gleichzeitig eine schnelle prototypische Entwicklung unterstützt.

Python eignet sich außerdem besonders, da viele Open-Source-Komponenten für Wake-Word-Erkennung, Speech-To-Text, LLM-Inferenz und Text-To-Speech direkt über Python-Schnittstellen nutzbar sind. Die einzelnen Verarbeitungsschritte können daher in einer einheitlichen Umgebung zusammengeführt werden, ohne dass für jede Komponente eigene Schnittstellen oder zusätzliche Hilfsprogramme entwickelt werden müssen. Dies unterstützt den modularen Aufbau des Systems, da die einzelnen Funktionen in getrennten Python-

Dateien umgesetzt und über klar definierte Eingaben und Ausgaben miteinander verbunden werden können.

Ein weiterer Vorteil besteht in der plattformübergreifenden Nutzbarkeit. Python Code kann nach der Installation sowohl auf Desktop-Systemen als auch auf Einplatinencomputern wie dem Raspberry Pi ausgeführt werden [19]. Dies ist für die Evaluation des Prototyps relevant, da das System auf unterschiedlicher Hardware getestet wird. Gleichzeitig erlaubt Python eine gut nachvollziehbare Implementierung der zentralen Ablaufsteuerung, insbesondere der State-Machine, ohne den Quellcode unnötig komplex zu machen.

Für die vorliegende Arbeit steht nicht die maximale Laufzeiteffizienz der Programmiersprache selbst im Vordergrund, sondern die praktische Umsetzbarkeit eines lokal betriebenen Sprachassistenten. Rechenintensive Verarbeitungsschritte werden überwiegend von spezialisierten Bibliotheken oder externen Laufzeitkomponenten übernommen, während Python vor allem zur Orchestrierung der Module, zur Konfiguration und zur Steuerung des Interaktionsablaufs eingesetzt wird. Damit stellt Python einen geeigneten Kompromiss zwischen Entwicklungsaufwand, Verständlichkeit, Erweiterbarkeit und technischer Anschlussfähigkeit dar.

4.3.2 Audioverarbeitung

Die Audioverarbeitung ist Grundlage für die Erfassung gesprochener Nutzereingaben und die Wiedergabe generierter Sprachantworten. Für den Prototyp ist dabei entscheidend, dass Audiodaten kontinuierlich vom Mikrofon eingelesen, in geeigneten Blöcken verarbeitet und an die nachfolgenden Komponenten weitergegeben werden können. Da die Wake-Word-Erkennung dauerhaft auf eingehende Audiodaten angewiesen ist, muss die Audioeingabe als fortlaufender Datenstrom realisiert werden. Gleichzeitig muss die generierte Antwort des Systems über eine Audioausgabe wiedergegeben werden können. Die gewählte Lösung sollte daher sowohl die Eingabe als auch die Ausgabe von Audiodaten unterstützen, plattformübergreifend nutzbar sein und sich gut in die Python-basierte Architektur integrieren lassen.

Für die Audioverarbeitung in Python stehen verschiedene Möglichkeiten zur Verfügung. Eine verbreitete Bibliothek ist PyAudio [20], die wie Sounddevice [21] auf PortAudio basiert und direkten Zugriff auf Audioein- und Ausgabegeräte ermöglicht. Sie eignet sich grundsätzlich für Echtzeit-Audioanwendungen, kann jedoch insbesondere bei der Installation auf unterschiedlichen Betriebssystemen zusätzlichen Aufwand verursachen. Die Bibliothek `speech_recognition` [22] bietet dagegen eine stärker abstrahierte Schnittstelle zur Aufnahme und Erkennung gesprochener Sprache. Für den vorliegenden Prototyp ist diese Abstraktion jedoch weniger geeignet, da nicht eine vollständig gekapselte Spracherkennung, sondern ein kontrollierter Zugriff auf den Audiostrom benötigt wird. Bibliotheken

wie `wave` [19] oder `librosa` [23] sind wiederum vor allem für das Lesen, Schreiben oder Analysieren bereits vorhandener Audiodateien ausgelegt und daher für die kontinuierliche Verarbeitung eines Live-Audiosignals nur eingeschränkt passend.

Für den Prototyp wurde deshalb `Sounddevice` eingesetzt. Die Bibliothek stellt eine Python-Schnittstelle zu `PortAudio` bereit und ermöglicht den direkten Zugriff auf Mikrofone und Lautsprecher. Im entwickelten System wird sie sowohl für die blockweise Aufnahme eingehender Audiodaten als auch für die Ausgabe der durch die Text-To-Speech-Komponente erzeugten Audiodaten verwendet. Eingehende Audiodaten können unmittelbar als numerische Datenstrukturen verarbeitet und an die Wake-Word-Erkennung übergeben oder temporär in einem Ring-Buffer zwischengespeichert werden.

Auch für die Sprachausgabe ist `Sounddevice` sinnvoll, da die von der Text-To-Speech-Komponente erzeugten Audiodaten direkt an ein Ausgabegerät weitergegeben werden können. Dies vereinfacht die Integration in die State-Machine, da die Aufnahme und Wiedergabe kontrolliert innerhalb des Programmablaufs gestartet und beendet werden können.

4.3.3 Wake-Word-Erkennung

Die Wake-Word-Erkennung ist der erste richtige Verarbeitungsschritt der Sprachinteraktion und dient dazu, den Sprachassistenten gezielt zu aktivieren. Zur Umsetzung einer Wake-Word-Erkennung können dafür verschiedene Ansätze und Libraries in Betracht gezogen werden. Eine Möglichkeit besteht darin, allgemeine Speech-To-Text-Systeme zur Erkennung eines Aktivierungswortes zu verwenden. Dieser Ansatz ist jedoch für eine dauerhaft aktive Erkennung nur eingeschränkt geeignet, da vollständige Spracherkennung in der Regel mehr Rechenleistung benötigt als spezialisierte Keyword-Spotting-Verfahren. Zudem würde dabei kontinuierlich Sprache transkribiert werden, obwohl zunächst nur ein einzelnes Aktivierungswort von Bedeutung wäre. Für den Prototyp ist daher ein spezialisiertes Wake-Word-System besser geeignet.

Eine verbreitete Lösung ist `Porcupine` [24] von Picovoice. Die Software ist für ressourcenschonende On-Device Wake-Word-Erkennung ausgelegt und unterstützt den Offline-Betrieb. Für wissenschaftliche und prototypische Arbeiten ist jedoch zu berücksichtigen, dass `Porcupine` kein vollständig offenes Open-Source-Projekt im gleichen Sinne wie andere Komponenten der Arbeit ist und stärker an das Ökosystem des Anbieters gebunden ist. Eine weitere bekannte Lösung ist `Snowboy` [25]. Diese wurde lange für Hotword-Erkennung in lokalen Sprachassistenten verwendet, wird jedoch nicht mehr aktiv weiterentwickelt und ist deshalb für eine neue prototypische Umsetzung ebenfalls nur eingeschränkt geeignet. Wesentlich interessanter ist `Mycroft Precise` [26], ein Open-Source-Ansatz, der eigentlich alle Anforderungen des Prototyps erfüllt, jedoch eher auf die Nutzung eigener

Wake-Words ausgelegt ist. Ein individuelles Wake-Word erfordert typischerweise großen Trainingsaufwand und eine geeignete Datengrundlage und würde damit den Rahmen der Arbeit überschreiten.

Letztendlich ist `openWakeWord` [27] als beste Lösung gewählt worden. Die Bibliothek ist als Open-Source-Produkt für Wake-Word-Erkennung konzipiert und stellt vortrainierte Modelle bereit, die lokal ausgeführt werden können. Gleichzeitig lässt sich `openWakeWord` direkt in eine Python-basierte Verarbeitungskette integrieren und punktet nicht zuletzt mit einer einfachen Handhabung im Code.

4.3.4 Speech-To-Text

Die Speech-To-Text-Komponente wandelt die nach der Wake-Word-Erkennung aufgezeichnete Spracheingabe in Text um. Hierfür muss die Komponente eine ausreichende Erkennungsqualität bei vertretbarer Latenz bieten, da lange Wartezeiten den natürlichen Ablauf einer Sprachinteraktion deutlich beeinträchtigen.

Eine leichtgewichtige Lösung zur Spracherkennung ist Vosk [28], das auf Kaldi basiert und gut auf ressourcenbeschränkter Hardware eingesetzt werden kann. Vosks Vorteil liegt vor allem in der vergleichsweise geringen Systemlast bei Ausführung [28]. Daraus ergibt sich eine Lösung, die grundsätzlich für lokale Anwendungen, bei denen niedriger Ressourcenverbrauch wichtiger ist als eine möglichst hohe Erkennungsqualität, geschaffen ist.

OpenAIs Whisper [29] stellt eine weitere Option dar. Whisper ist ein allgemeines Spracherkennungsmodell, das neben multilingualer Spracherkennung auch Sprachübersetzung und robuste Spracherkennung über verschiedene Audiobedingungen hinweg unterstützt [30]. Die offizielle Implementierung ist gut dokumentiert und als Open-Source-Software verfügbar [29]. Für einen interaktiven Offline-Sprachassistenten ist jedoch nicht nur die Erkennungsqualität relevant, sondern auch die Inferenzgeschwindigkeit. Die ursprüngliche Implementierung kann insbesondere auf leistungsschwächeren Geräten zu höheren Antwortzeiten führen, da sie im Vergleich zu optimierten Varianten mehr Rechenzeit und Speicher benötigt [31].

Als dritte Variante wurde `faster-whisper` [31] betrachtet. Dabei handelt es sich um eine Reimplementierung von Whisper auf Basis von `CTranslate2`, einer auf effiziente Transformer-Inferenz ausgelegten Laufzeitumgebung. Nach Angaben des Projekts kann `faster-whisper` bei gleicher Erkennungsqualität deutlich schneller als OpenAIs Whisper arbeiten und benötigt dabei weniger Speicher. Zusätzlich unterstützt die Implementierung 8-Bit-Quantisierung (siehe [Abschnitt 2.4](#)) auf CPU und GPU, wodurch sich der Ressourcenbedarf weiter reduzieren lässt [31]. Damit stellt `faster-whisper` eine für lokale Sprachassis-

tenzsysteme besonders relevante Variante dar, da sie die Erkennungsqualität von Whisper mit einer effizienteren Ausführung verbindet.

Zur finalen Bewertung der Erkennungsqualität wurden zusätzlich eigene Tests mit Vosk, Whisper und faster-Whisper durchgeführt. Hierfür wurden drei identische Spracheingaben verwendet, die typische Interaktionen mit dem Prototyp abbilden:

- „Wie wird das Wetter heute?“
- „Stelle einen Timer für 15 Minuten.“
- „Erzähle mir einen Witz.“

Der Test diente nicht einer umfassenden Evaluation der allgemeinen Erkennungsqualität, sondern einer anwendungsbezogenen Einschätzung, ob die jeweilige Komponente typische Spracheingaben des Prototyps unter identischen Bedingungen zuverlässig transkribieren kann.

Alle Tests fanden auf der Hardware eines Apple MacBook M4 Air statt. Für jede Komponente wurden dieselben drei Spracheingaben jeweils zehnmal ausgesprochen. Eine Transkription wurde als korrekt gewertet, wenn die für die Weiterverarbeitung relevante Bedeutung vollständig erhalten blieb. Kleinere Abweichungen in Zeichensetzung, Groß- und Kleinschreibung oder nicht bedeutungsverändernde Formulierungen wurden daher nicht als Fehler gewertet. Abweichungen bei Zahlenwerten, Schlüsselwörtern oder in der semantischen Aussage der Eingabe wurden dagegen als fehlerhafte Erkennung dokumentiert.

Tabelle 4.13: Vergleich der getesteten Speech-To-Text-Komponenten anhand eigener Testeingaben

Komponente	Korrekt Erkannt	Auffälligkeiten
Vosk	24 von 30	Häufigere Abweichungen bei einzelnen Wörtern und Zahlenwerten
Whisper	29 von 30	Sehr zuverlässige Erkennung
faster-Whisper	28 von 30	Leichte Abweichung zu Whisper trotz ähnlicher Technologie

Die Ergebnisse in [Tabelle 4.13](#) zeigen, dass alle drei Systeme die Eingaben grundsätzlich gut verarbeiten konnten. Vosk erbrachte in diesem Test jedoch häufiger Erkennungsfehler, vor allem bei Zahlenwörtern. Im Allgemeinen war die Erkennungsleistung ausreichend, bei variableren Nutzereingaben kann die geringere Genauigkeit allerdings dazu führen, dass das LLM eine inhaltlich verfälschte Eingabe erhält. Da der Prototyp hauptsächlich frei

formulierte Anfragen verarbeiten soll, wurde Vosk trotz der geringen Systemanforderungen nicht ausgewählt.

Zwischen Whisper und faster-Whisper ergaben sich in den eigenen Tests keine relevanten Unterschiede hinsichtlich der Erkennungsqualität. Beide Systeme konnten die Testfragen zuverlässig transkribieren. Für die Auswahl zwischen diesen beiden Varianten ist daher vor allem die Effizienz der Ausführung entscheidend. Hierfür wurden die in [Tabelle 4.14](#) und [Tabelle 4.15](#) dargestellten Benchmark-Ergebnisse aus der Dokumentation von faster-Whisper herangezogen. Die Werte beziehen sich auf die Transkription einer 13-minütigen Audiodatei. Der GPU-Benchmark wurde mit einer NVIDIA RTX 3070 Ti mit 8 GB VRAM durchgeführt, der CPU-Benchmark mit acht Threads auf einem Intel Core i7-12700K [31].

Tabelle 4.14: Vergleich von Whisper und faster-Whisper mit dem Large-v2-Modell auf GPU [31]

Implementierung	Präzision	Zeit	Speicherbedarf
Whisper	float16	2 min 23 s	4708 MB VRAM
faster-Whisper	float16	1 min 03 s	4525 MB VRAM
faster-Whisper	int8	59 s	2926 MB VRAM

Tabelle 4.15: Vergleich von Whisper und faster-Whisper mit dem Small-Modell auf CPU [31]

Implementierung	Präzision	Zeit	Speicherbedarf
Whisper	float32	6 min 58 s	2335 MB RAM
faster-Whisper	float32	2 min 37 s	2257 MB RAM
faster-Whisper	int8	1 min 42 s	1477 MB RAM

Die Benchmark-Ergebnisse zeigen, dass faster-Whisper sowohl auf GPU als auch auf CPU kürzere Transkriptionszeiten erreicht als die ursprüngliche Whisper-Implementierung. Besonders relevant ist zudem die Möglichkeit der int8-Ausführung. Dafür werden die Berechnungen mit einer geringeren Präzision durchgeführt, also Gewichtungen mit weniger Bits versehen, wodurch später der Speicherbedarf und damit die Systemanforderungen deutlich reduziert werden könnten.

Die Auswahl von faster-Whisper ergibt sich damit aus zwei Bewertungsschritten. Die eigenen Tests zur Erkennungsqualität sprechen gegen Vosk, da die geringere Genauigkeit bei frei formulierten Eingaben für den vorgesehenen Einsatzzweck problematisch ist. Zwischen Whisper und faster-Whisper zeigte sich dagegen eine vergleichbare Transkriptionsqualität, sodass die effizientere Ausführung ausschlaggebend wurde. faster-Whisper

stellt für den Offline-Sprachassistenten somit den geeignetsten Kompromiss aus lokaler Ausführbarkeit, Erkennungsqualität, Latenz und Ressourcenbedarf dar.

4.3.5 Large Language Model

Für die Generierung natürlicher Antworten wird im Prototyp ein LLM eingesetzt. Diese Komponente übernimmt innerhalb der Verarbeitungskette die Aufgabe, aus der transkribierten Nutzereingabe eine passende textuelle Antwort zu erzeugen. Diese Komponente ist für den Prototypen besonders ressourcenrelevant, da Sprachmodelle im Vergleich zu den übrigen Verarbeitungsschritten einen hohen Speicherbedarf und eine hohe Rechenlast verursachen können.

Um die Architektur der Large Language Komponente zu wählen, müssen jedoch zwei Aspekte betrachtet werden: Plattform und Modellauswahl. Die Plattform betrifft die technische Umgebung, in der die Inferenz des LLM stattfindet, also die Laufzeitumgebung. Die Modellauswahl betrifft die konkrete Wahl eines Sprachmodells, das für die Antwortgenerierung eingesetzt wird.

Lokale Inferenz

Für die lokale Ausführung von Sprachmodellen kommen unterschiedliche technische Ansätze infrage. Modelle können beispielsweise direkt über Frameworks wie PyTorch [32] oder Transformers [33] ausgeführt werden. Dieser Ansatz bietet eine hohe Flexibilität, da viele aktuelle Modelle direkt über bestehende Modellbibliotheken eingebunden werden können. Für Software, die auf unterschiedlichen Hardwareplattformen laufen soll, ist dieser Weg jedoch nur eingeschränkt geeignet. Die Ausführung ist vergleichsweise ressourcenintensiv und setzt je nach Modell eine leistungsfähige GPU oder umfangreiche Optimierungen voraus. Gerade auf leistungsschwächeren Zielsystemen wie einem Raspberry Pi steht dieser Ansatz damit im Konflikt zu den Anforderungen an geringen Ressourcenverbrauch, einfache Bereitstellung und kurze Antwortzeiten.

Eine weitere Möglichkeit stellen lokale Laufzeitumgebungen wie GPT4A11 [34] oder Ollama [35] dar. Beide sind darauf ausgelegt, Sprachmodelle lokal auf Alltagsgeräten auszuführen und stellen dafür neben einer Anwendung auch eine Python-Schnittstelle bereit. Die Lösungen eignen sich damit grundsätzlich für Szenarien, in denen keine Cloud-API genutzt werden soll. Für das Projekt ist jedoch nachteilig, dass GPT4A11 und Ollama stärker als eigenständige Laufzeitumgebung mit eigener Modellverwaltung konzipiert sind und sich damit schwierig in ein einzelnes Python-Package integrieren lassen. Beide Lösungen setzen auf `llama.cpp` als Inferenzbibliothek, bieten jedoch zusätzlich eine eigene Serverkomponente, die die Modellverwaltung und die Anbindung an die Anwendung übernimmt.

Für den Prototyp ist diese zusätzliche Schicht aber nicht zwingend erforderlich. Da der Prototyp eine eigene Verarbeitungskette besitzt und die LLM-Komponente direkt in diese eingebunden werden kann, ist ein zusätzlicher lokaler Dienst mit eigener Modellverwaltung unnötig.

Aus diesen Gründen wurde die auch von Lösungen wie Ollama und GPT4All genutzte Inferenztechnologie `llama.cpp` [36] gewählt. `llama.cpp` ist eine leichtgewichtige Inferenzbibliothek in C/C++, die auf die lokale Ausführung großer Sprachmodelle mit geringem Einrichtungsaufwand und hoher Leistung ausgelegt ist. Die Bibliothek unterstützt quantisierte Modelle und kann dadurch den Speicherbedarf sowie den Rechenaufwand gegenüber unquantisierten Modellvarianten deutlich reduzieren. Es ist damit leistungstechnisch eher möglich Modelle lediglich auf der CPU auszuführen. Gleichzeitig unterstützt `llama.cpp` neben reiner CPU-Inferenz auch verschiedene Beschleunigungs-Backends, darunter Metal für Apple-Systeme, CUDA für NVIDIA-Grafikkarten und weitere Schnittstellen wie Vulkan oder OpenCL, womit auch größere Modelle effizient betrieben werden können.

Für die Integration in den Python-basierten Prototyp kann `llama.cpp` über `llama-cpp-python` [37] angebunden werden. Diese Bibliothek stellt Python-Bindings für `llama.cpp` bereit und bietet sowohl eine direkte Programmierschnittstelle für Textgenerierung als auch weitere Integrationsmöglichkeiten. Das LLM kann so in die bestehende modulare Architektur eingebettet werden, ohne dass ein separater Serverprozess oder eine externe Schnittstelle erforderlich ist.

Modellauswahl

In der heutigen Zeit stehen eine große Anzahl unterschiedlicher LLMs zur Verfügung, wobei sich sowohl die Modelllandschaft als auch die zugrunde liegenden Technologien innerhalb kurzer Zeit stark weiterentwickeln. Zur Auswahl eines geeigneten Sprachmodells werden daher nur drei, zum Zeitpunkt der Entwicklung am meisten relevante, Modelle verglichen. Bei allen betrachteten Varianten handelt es sich um Instruction-Modelle (siehe [Abschnitt 2.4](#)), die auf das Verstehen natürlichsprachlicher Anweisungen und die Generierung passender Antworten ausgerichtet sind.

Eine mögliche Option ist Llama-3.2-3B-Instruct [38] der Firma Meta. Das Modell ist für mehrsprachige Dialoganwendungen vorgesehen und unterstützt laut Meta unter anderem Deutsch als offizielle Sprache. Zudem besitzt es eine Kontextlänge von 128 000 Tokens und ist mit 3,21 Milliarden Parametern noch in einer Größenordnung, die für lokale Tests grundsätzlich geeignet ist [39]. Für den Prototyp ist jedoch zu berücksichtigen, dass die Nutzung an die Llama-3.2-Community-License gebunden ist, während andere Modelle unter offeneren Lizenzen bereitgestellt werden.

Eine weitere betrachtete Alternative ist Phi-3.5-mini-instruct [40] von Microsoft. Das Modell besitzt 3,8 Milliarden Parameter, unterstützt ebenfalls eine Kontextlänge von 128 000 Tokens und ist unter der MIT-Lizenz veröffentlicht. Es ist auf textbasierte Chat-Prompts ausgelegt und erreicht in multilingualen Benchmarks gute Ergebnisse. Durch die etwas höhere Parameteranzahl ist jedoch auch ein höherer Ressourcenbedarf zu erwarten [41]. Für einen möglichst breit auf verschiedenen Geräten lauffähigen Prototyp ist deshalb ein kleineres Modell vorteilhaft, sofern die Antwortqualität für die vorgesehenen Anwendungsfälle ausreicht.

Als dritte Option wurde Qwen2.5-3B-Instruct [42] von Alibaba Cloud betrachtet. Das Modell umfasst 3,09 Milliarden Parameter, davon 2,77 Milliarden Nicht-Embedding-Parameter, und unterstützt eine Kontextlänge von 32 768 Tokens sowie eine maximale Generierungslänge von 8 192 Tokens. Qwen beschreibt die Modellreihe als verbessert hinsichtlich Instruktionsbefolgung, strukturierter Ausgaben sowie mathematischer und programmierbezogener Fähigkeiten [11].

Tabelle 4.16: Vergleich betrachteter lokaler Large-Language-Models

Modell	Parameter	Kontextlänge	Lizenz	Auffälligkeiten
Llama-3.2-3B-Instruct	3,21B	128K	Llama-3.2 Community License	Mehrsprachiges Dialogmodell, jedoch mit spezifischer Community-Lizenz
Phi-3.5-mini-instruct	3,8B	128K	MIT	Gute mehrsprachige Fähigkeiten, jedoch größer als die übrigen betrachteten 3B-Modelle
Qwen2.5-3B-Instruct	3,09B	32K	Apache-2.0	Kompaktes Instruct-Modell mit guter GGUF-Verfügbarkeit und direkter Eignung für llama.cpp

Der Vergleich in [Tabelle 4.16](#) zeigt, dass alle drei Modelle grundsätzlich für eine lokale Antwortgenerierung gut geeignet sind und eine Entscheidung zwischen ihnen eine geringere Rolle spielt. Llama-3.2-3B-Instruct und Phi-3.5-mini-instruct bieten sehr große Kontextfenster, was für lange Dokumente oder umfangreiche Dialogverläufe vorteilhaft sein kann. Für den entwickelten Sprachassistenten ist dieser Vorteil jedoch nur begrenzt relevant, da die Interaktionen aus kurzen Spracheingaben und kurzen Antwortsequenzen bestehen. Wichtiger sind eine möglichst geringe Modellgröße, eine einfache lokale Ausführung und eine gute Integration in die gewählte Inferenzumgebung.

Die Entscheidung für qwen2.5-3b-instruct-q4_k_m ergibt sich damit aus der Kombination aus kompakter Modellgröße, instruction-getunter Dialogfähigkeit, lokaler Ausführbarkeit mit llama.cpp, offener Lizenzierung und reduziertem Speicherbedarf durch Quantisierung. Für den entwickelten Offline-Sprachassistenten ist diese Kombination beson-

ders geeignet, da sie eine vollständig lokale Antwortgenerierung ermöglicht und zugleich die praktischen Hardwaregrenzen des Prototyps berücksichtigt.

4.3.6 Text-To-Speech

Zur Wahl der Text-To-Speech-Komponenten kommen verschiedene Möglichkeiten infrage. Eine besonders einfache und ressourcenschonende Option ist eSpeak NG [43]. Hierbei handelt es sich um einen kompakten Formantsynthesizer, bei dem Sprache künstlich durch ein Modell von menschlichen Sprachorganen erzeugt wird. Ein großer Nachteil dieser Technik besteht darin, dass die erzeugte Sprache vergleichsweise synthetisch und künstlich klingt. Um eine möglichst natürliche Interaktion ermöglichen ist deshalb eine neuronale Text-To-Speech-Lösung geeigneter.

Eine weitere moderne Alternative ist Kokoro [44]. Das Open-Weight Text-To-Speech-Modell umfasst 82 Millionen Parameter und soll trotz seiner kompakten Architektur eine hohe Synthesequalität bei effizienter Ausführung ermöglichen [44]. Für den Prototyp ist Kokoro jedoch nur eingeschränkt geeignet, da zum Zeitpunkt der Komponentenauswahl keine stabil nutzbaren deutschen Stimmen im offiziellen Modellangebot bereitstehen. Auch wenn englische Modelle grundsätzlich kein Problem darstellen, kann die Nutzung englischer oder nur experimentell verfügbarer Stimmen die Vergleichbarkeit im späteren Evaluationsteil stark beeinflussen und macht Kokoro damit für die Umsetzung weniger passend.

Mit Coqui TTS [45] wird ein umfangreiches Toolkit für neuronale Sprachsynthese mit vortrainierten Modellen in vielen Sprachen sowie Funktionen zum Trainieren und Fine-Tuning eigener Stimmen bereitgestellt. Dadurch bietet die Lösung viele Möglichkeiten für hochwertige und flexible Sprachsynthese. Für den Prototyp ist dieser Funktionsumfang wiederum nur begrenzt erforderlich. Die Arbeit konzentriert sich nicht auf das Training eigener Stimmen, sondern auf die Integration einer lokal ausführbaren und zuverlässig nutzbaren Sprachausgabe. Außerdem wird Coqui TTS zum heutigen Stand nicht mehr weiterentwickelt und bietet damit keine langfristige Perspektive für die Nutzung.

Schlussendlich sollte auch Piper [46] betrachtet werden. Piper ist ein lokales neuronales Text-To-Speech-System, das auf eine schnelle Sprachsynthese ausgelegt ist und auch für den Einsatz auf leistungsschwächerer Hardware geeignet ist. Interessanterweise basiert Piper die Phonemisierung, also die Umwandlung von Graphemen in Phoneme (siehe [Abschnitt 2.5](#)) auf der zuvor behandelten Technologie von eSpeak NG [46]. Modelltechnisch existieren für Piper nicht nur englische Stimmen, sondern auch eine vergleichsweise gut nutzbare deutsche Stimme von Thorsten Müller aus dem ThorstenVoice-Projekt [47]. Diese wurde ursprünglich für Coqui TTS entwickelt, kann und darf aber auch gut mit Piper genutzt werden.

Die zuvor beschriebenen Optionen unterscheiden sich vor allem hinsichtlich Natürlichkeit, Integrationsaufwand, Ressourcenbedarf und Verfügbarkeit geeigneter deutscher Stimmen. [Tabelle 4.17](#) fasst diese Kriterien verdichtet zusammen und ordnet die betrachteten Komponenten im Hinblick auf ihre Eignung für den entwickelten Offline-Sprachassistenten ein.

Tabelle 4.17: Vergleich betrachteter Text-To-Speech-Komponenten

Komponente	Charakteristik	Zentrale Einschränkung	Eignung
eSpeak NG	Sehr ressourcenschonende Offline-Sprachsynthese	Deutlich synthetische Sprachausgabe	mittel
Coqui TTS	Umfangreiches Toolkit für neuronale Sprachsynthese und Modelltraining	Für den Prototyp vergleichsweise komplex und nicht mehr aktiv weiterentwickelt	mittel
Kokoro	Kompaktes Open-Weight-TTS-Modell mit effizienter Architektur	Keine stabil nutzbaren deutschen Stimmen im offiziellen Modellangebot	gering
Piper	Lokale neuronale Sprachsynthese mit vortrainierten Stimmen und schlanker Einbindung	Weniger flexibel als umfangreiche Trainingsframeworks	hoch

Die Bewertung in [Tabelle 4.17](#) zeigt, dass Piper die optimalste Lösung für den entwickelten Offline-Sprachassistenten darstellt. Die Kombination aus einer vergleichsweise natürlichen Sprachausgabe, der Verfügbarkeit einer gut nutzbaren deutschen Stimme, der effizienten lokalen Ausführung und der einfachen Integration in die Python-basierte Architektur macht Piper zur optimalen Wahl für die Text-To-Speech-Komponente des Prototyps.

Kapitel 5

Prototypische Umsetzung

Nach Festlegung der konzeptionellen Architektur des Sprachassistenten wird folgend Umsetzung behandelt. Dafür wird gezeigt, wie die zuvor beschriebenen Anforderungen und Architekturentscheidungen in eine lauffähige Implementierung überführt wurden. Der Fokus liegt nicht auf der Entwicklung eines vollständig produktionsreifen Sprachassistentensystems, sondern auf einem funktionsfähigen Artefakt, mit dem die lokale Verarbeitungskette praktisch erprobt und anschließend evaluiert werden kann.

5.1 Projektstruktur

Die Umsetzung des Offline-Sprachassistenten ist modular aufgebaut und orientiert sich an der zuvor beschriebenen Systemarchitektur. Die Funktionen des Systems sind daher nicht in einer einzelnen Anwendungsdatei zusammengeführt, sondern auf mehrere Python-Module verteilt. Diese werden im Systemarchitekturdiagramm ([Abbildung 5.1](#)) einer von drei Kategorien zugeordnet.

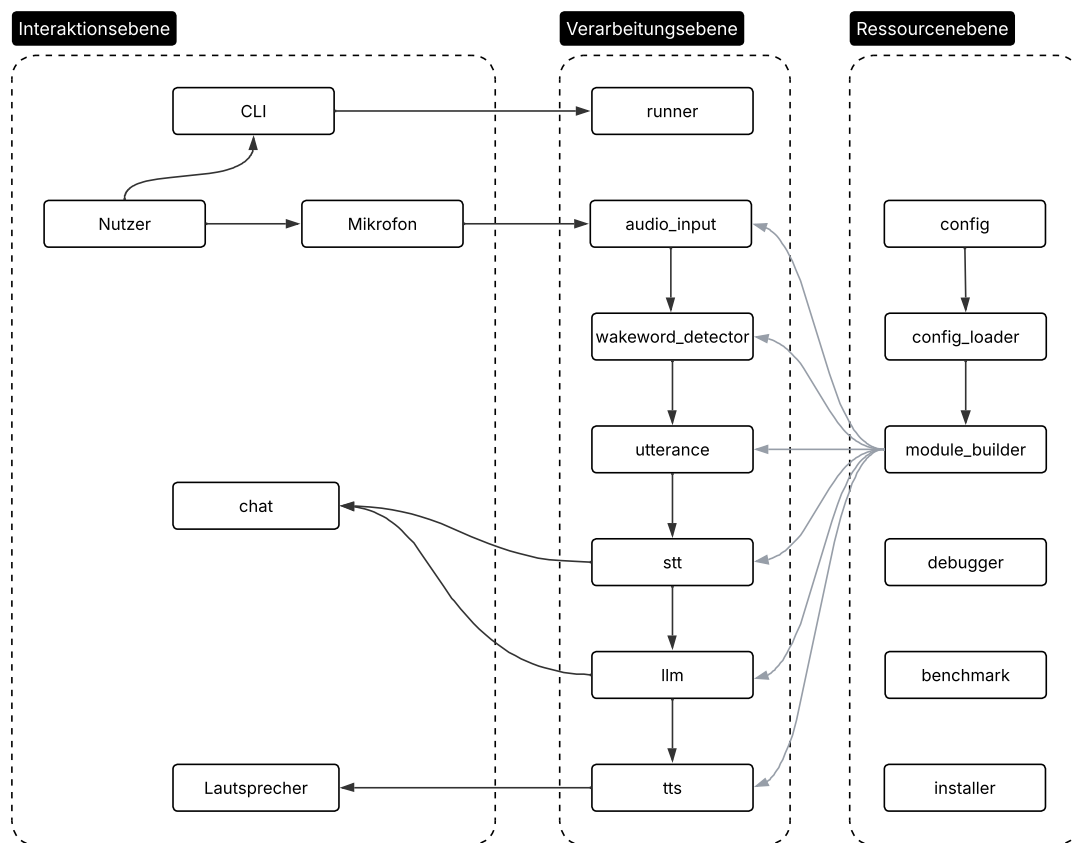


Abbildung 5.1: Systemarchitektur des Offline-Sprachassistenten

Die Interaktionsebene ist die Schnittstelle zwischen Nutzer und System. Hierzu zählen die Spracheingabe über das Mikrofon und die Sprach- und Textausgabe über den Lautsprecher oder die Kommandozeile. Die Kommandozeilenschnittstelle (CLI) dient vor allem der Start der Software und der Darstellung einfacher Status-Ausgaben während der Interaktion. Die Verarbeitungsebene enthält die eigentlichen Softwaremodule des Sprachassistenten. Den zentralen Einstiegspunkt bildet `runner.py`. Diese Datei steuert den Ablauf der Anwendung und verbindet die einzelnen Verarbeitungsschritte miteinander. Der Prototyp durchläuft dabei definierte Zustände einer State-Machine (siehe [Unterabschnitt 4.2.2](#)). Die Datei `runner.py` übernimmt somit die koordinierende Rolle innerhalb des Systems, während die fachlichen Verarbeitungsschritte in separaten Modulen gekapselt sind. Initialisierung werden diese Module über `module_builder.py`. Dieses Modul lädt die Konfiguration, erstellt die benötigten Modellverzeichnisse und instanziiert die einzelnen Teilkomponenten der Verarbeitungsebene. Dazu gehören das Audio-Modul `audio_input.py`, die Wake-Word-Erkennung `wakeword_detector.py`, der Recorder für Nutzereingaben `utterance.py`, die Speech-To-Text-Komponente `stt.py`, das lokale Sprachmodell `llm.py`

sowie die Text-To-Speech-Komponente `tts.py`. Durch diese Trennung wird verhindert, dass Initialisierungslogik und Ablaufsteuerung vermischt werden. Änderungen an Modellpfaden, Schwellenwerten oder Geräteparametern können dadurch über die Konfiguration in `config.yaml` vorgenommen werden, ohne die zentrale Ablaufsteuerung anpassen zu müssen.

In der von `config_loader.py` bereitgestellten Konfigurationsdatei werden zentrale Parameter des Prototyps definiert, etwa die Abtastrate der Audioaufnahme, Modellpfade, Schwellenwerte der Wake-Word-Erkennung, Einstellungen der Speech-To-Text-Komponente, Parameter des lokalen Sprachmodells und Einstellungen für die Sprachsynthese. Die Modelldateien selbst werden beim initialen Ausführen von `installer.py` einmalig heruntergeladen und lokal in separaten Verzeichnissen abgelegt.

Ergänzend unterstützen die Module `debugger.py` und `benchmark.py` die Fehlersuche und Leistungsanalyse des Prototyps. `debugger.py` ermöglicht die Ausgabe von Statusinformationen, Fehlermeldungen und speichert jeden Schritt der Interaktion als Audio- oder Textdatei. `benchmark.py` übernimmt die Messung von Antwortzeiten und Ressourcenverbrauch für die einzelnen Prozessschritte. Diese Module sind nicht direkt in den Hauptablauf integriert, können aber bei Bedarf zur Analyse und Optimierung des Systems herangezogen werden.

Die Projektstruktur unterstützt damit den datenschutzorientierten Charakter des Prototyps. Lediglich `installer.py` greift auf Wunsch auf externe Ressourcen zu, wird aber nicht im Hauptprozess `runner.py` aufgerufen. Gleichzeitig bleibt die technische Umsetzung erweiterbar, da einzelne Module durch alternative Implementierungen ersetzt werden können, ohne die gesamte Anwendung neu strukturieren zu müssen.

5.1.1 Konfiguration

Die Konfiguration des Prototyps erfolgt zentral über die Datei `config.yaml`. Darin werden die wesentlichen Parameter der einzelnen Systemkomponenten festgelegt, ohne dass hierfür Änderungen am Programmcode erforderlich sind. Diese Trennung zwischen Konfiguration und Implementierung erleichtert die Anpassung des Systems an unterschiedliche Hardwareumgebungen und unterstützt zugleich die modulare Struktur des Prototyps. Modellpfade, Audioeinstellungen, Schwellenwerte und Inferenzparameter können dadurch an einer zentralen Stelle verändert werden.

Das Laden und Verteilen der Konfigurationswerte wird in `module_builder.py` gebündelt. Beim Start der Anwendung wird dort zunächst geprüft, ob bereits eine Konfigurationsdatei im äußersten Verzeichnis existiert. Falls nicht, wird die interne Standardkonfiguration dorthin kopiert. Hiermit wird sichergestellt, dass ein versehentliches Löschen der Originaldatei die Programmausführung verhindert. Sobald eine gültige Konfiguration vorliegt,

werden die Werte über den `ConfigLoader` eingelesen und in einem zentralen Objekt gespeichert. Analog dazu wird jeder Eintrag auch grundlegend nach Typ und Format geprüft und bei unzulässigen Eingaben mit dem Standardwert ersetzt. Anschließend instanziiert der `ModuleBuilder` die einzelnen Komponenten und übergibt ihnen jeweils nur die für sie relevanten Parameter des Objekts. Beispielsweise erhält das Audio-Modul aus `audio_input.py` Werte wie Abtastrate, Blockgröße, Kanalanzahl und Größe des Ring-Buffers. Die Wake-Word-Erkennung aus `wakeword_detector.py` wird dagegen mit dem Pfad zum Wake-Word-Modell sowie den Schwellenwerten für Erkennung und Sprachaktivität initialisiert.

Die Audioverarbeitung ist in der Konfigurationsdatei über die Parameter `au_sample_rate`, `au_block_size`, `au_channels` und `au_ring_buffer_chunks` beschrieben. Im Prototyp wird eine Abtastrate von 16000 Hz und ein einzelner Audiokanal verwendet, da die genutzten STT- beziehungsweise Wake-Word-Verfahren ohnehin nur mit 16 kHz Audio arbeiten und Stereoinformationen prinzipiell ebenfalls keinen Mehrwert bringen. Die Blockgröße legt fest, in welchen zeitlichen Abschnitten das Mikrofonsignal verarbeitet wird. Diese wird standardmäßig auf 80 ms gesetzt um bei einer 16 kHz Samplingrate auf genau 1280 verarbeiteten Samples pro Audioblock zu gelangen. Dieser Wert ist noch klein genug, damit das System regelmäßig neue Audiodaten erhält und Wake-Word-Erkennung sowie Aufnahmestatus zeitnah reagieren können. Gleichzeitig ist der Block groß genug, um nicht zu viele Callback-Aufrufe pro Sekunde zu erzeugen und das System unnötig zu überlasten. Der Parameter `au_ring_buffer_chunks` zeigt die Menge an Audioblocken an, die letztendlich im Ring-Buffer gehalten werden. Standardmäßig sind dies 20 Blöcke, was bei einer Blockgröße von 80 ms einem Zeitfenster von 1,6 s entspricht und damit die ungefähre Länge einer typischen Wake-Word-Äußerung abdeckt. Der Buffer stellt damit sicher, dass unmittelbar vor der Erkennung eines Wake-Words gesprochene Audiodaten noch in die weitere Verarbeitung einbezogen werden können.

Für die Wake-Word-Erkennung werden in `config.yaml` der Modellpfad, der Erkennungsschwellenwert und der VAD-Schwellenwert festgelegt. `wwd_threshold` steuert ab welchem Wahrscheinlichkeitswert die Erkennung eines Wake-Words als positiv gilt und `vad_threshold` legt fest, ab welchem Pegel eine Sprachaktivität angenommen oder stiller erkannt wird. Beide werden standardmäßig auf 0,5 gesetzt, sollten aber je nach Mikrofon und Umgebungsgeräuschpegel angepasst werden.

Die Speech-To-Text-Komponente wird über Parameter wie `stt_model_path`, `stt_language`, `stt_device`, `stt_compute_type` und `stt_beam_size` konfiguriert. Neben dem Modellpfad wird damit festgelegt, ob die Inferenz auf der CPU oder GPU ausgeführt wird und welcher Berechnungstyp verwendet werden soll. `stt_device: auto` prüft automatisch ob eine unterstützte NVIDIA-GPU vorhanden ist und benötigte CUDA-Treiber instal-

liert sind. Für alle weiteren Systeme oder bei fehlender CUDA-Unterstützung wird die Transkription CPU-basiert ausgeführt. Ergänzend steuern `stt_silence_threshold` und `stt_silence_blocks` das Ende einer aufgenommenen Äußerung.

ändern falls Silero benutzt wird!

Gerade LLM-Inferenz besitzt eine Vielzahl an konfigurierbaren Variablen. Ähnlich der anderen Komponenten verweist `llm_model_path` auf das lokal gespeicherte Modell. Mit `llm_n_ctx` wird die Kontextlänge auf 4096 Tokens festgelegt, sodass Systemanweisung, Nutzereingabe, begrenzter Gesprächsverlauf und Antwort innerhalb eines ausreichend großen, aber weiterhin ressourcenschonenden Kontextfensters verarbeitet werden können. Der Parameter `llm_n_gpu_layers` ist auf -1 gesetzt, das bei unterstützter Hardware möglichst viele Modellschichten auf eine verfügbare GPU ausgelagert werden. Die maximale Antwortlänge wird mit `llm_max_tokens` auf 256 Tokens begrenzt, um kurze und interaktive Antworten zu fördern und die anschließende Sprachsynthese nicht unnötig zu verzögern. Mit einer Temperatur von 0,5 erzeugt das Modell vergleichsweise stabile Antworten, ohne vollständig deterministisch zu reagieren. Zusätzlich begrenzen `llm_history_limit` und `llm_conversation_timeout` den Gesprächskontext auf wenige aktuelle Interaktionen und setzen ihn nach einer kurzen Inaktivitätsphase zurück. Dadurch werden Ressourcen geschont und alte Gesprächsinhalte nicht unnötig in spätere Anfragen übernommen.

Die Text-To-Speech-Komponente verwendet mit `tts_model_path` und `tts_config_path` ein lokal gespeichertes Piper-Modell einschließlich der zugehörigen Konfigurationsdatei. Die übrigen Parameter orientieren sich an einer möglichst neutralen und gut verständlichen Sprachausgabe. Mit `tts_length_scale` 1,0 wird die Standardsprechgeschwindigkeit des Modells beibehalten, sodass die Ausgabe weder künstlich beschleunigt noch verlangsamt wird. Die Parameter `tts_noise_scale` 0,667 und `tts_noise_w_scale` 0,8 steuern die Variabilität der synthetisierten Stimme und sind so gewählt, dass die Ausgabe nicht vollständig monoton wirkt, aber weiterhin stabil und gut verständlich bleibt. Mit `tts_sentence_silence` 0,2 wird zwischen einzelnen Sätzen eine kurze Pause eingefügt, wodurch längere Antworten besser verständlich werden, ohne die Interaktion unnötig zu verzögern.

Neben den eigentlichen Verarbeitungsparametern enthält die Konfiguration auch Angaben für Debugging und Benchmarking. Der Eintrag `debug_dir` legt fest, in welchem Verzeichnis Debug-Ausgaben gespeichert werden. Der Parameter `benchmark_samples_dir` verweist auf ein Verzeichnis mit vorgefertigten Audiodateien um für spätere Messungen nicht jedesmal den selben Satz sprechen zu müssen und insgesamt die Evaluation konsistent zu halten.

5.1.2 Interaktionsablauf

Nachdem die konzeptionelle Ablaufstruktur bereits in [Unterabschnitt 4.2.2](#) beschrieben wurde, steht nun die konkrete Umsetzung einer vollständigen Interaktion im Prototyp im Vordergrund. Im Mittelpunkt steht dabei die erneute Beschreibung der Zustände, sondern das Zusammenspiel der beteiligten Module innerhalb der Anwendung. Das Sequenzdiagramm in [Abbildung 5.2](#) veranschaulicht, wie die Verarbeitungsschritte zwischen der zentralen Steuerung und den einzelnen Komponenten ausgeführt werden.

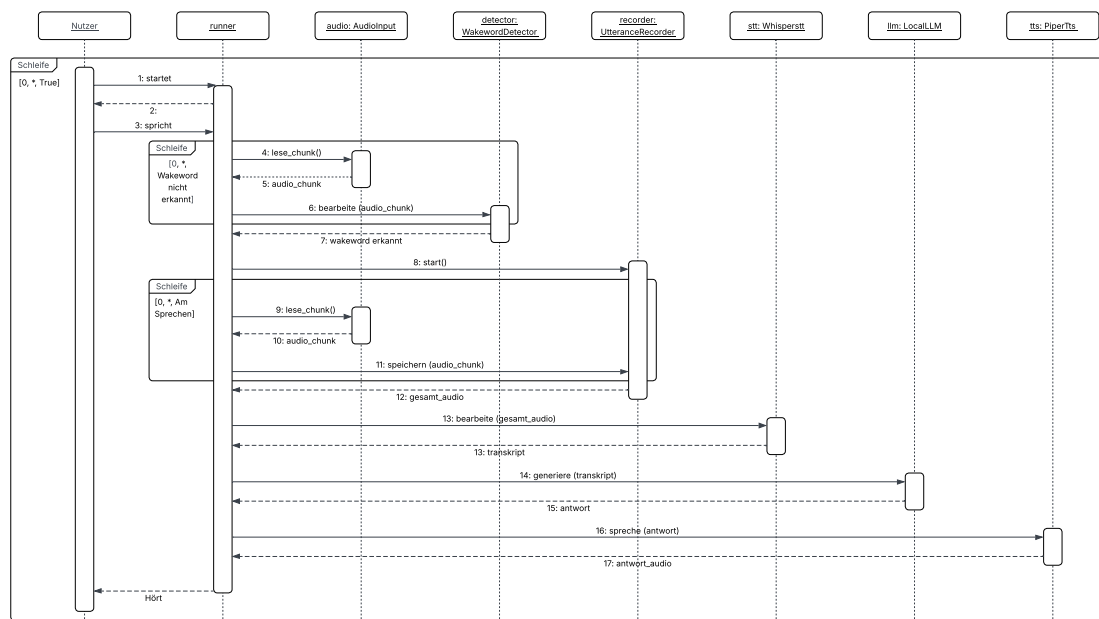


Abbildung 5.2: Sequenzdiagramm einer Sprachinteraktion

Der Einstiegspunkt des Prototyps befindet sich in der Funktion `run()` aus `runner.py`. Dort wird zunächst eine Instanz des `ModuleBuilder` erzeugt, der die benötigten Komponenten anhand der Konfigurationsdatei initialisiert. Über `build_all()` werden nacheinander alle benötigten Module erstellt. Die Hauptschleife muss die Komponenten somit nicht selbst konfigurieren, sondern kann sie direkt für die Verarbeitung verwenden.

Nach der Initialisierung startet das Audio-Modul über `audio.start()` den Mikrofoneingang. Anschließend liest die Hauptschleife fortlaufend einzelne Audio-Chunks mit `audio.read_chunk()`. Diese blockweise Verarbeitung ist für den Prototyp zentral, da dieselben Audiodaten zunächst für die Wake-Word-Erkennung und anschließend für die Aufnahme einer vollständigen Äußerung genutzt werden können. Das `AudioInput`-Modul speichert die gelesenen Chunks zusätzlich in einem Ring-Buffer.

Die Wake-Word-Erkennung erfolgt durch das Modul `WakewordDetector`. In der Hauptschleife wird jeder gelesene Audio-Chunk an `detector.process(chunk)` übergeben. Die

Methode liefert zurück, ob ein Wake-Word erkannt wurde, welches Wake-Word betroffen ist und mit welchem Score die Erkennung erfolgt ist. Im Code wird hierfür zusätzlich eine einfache Edge-Logik verwendet: Ein Wake-Word löst nicht bei jedem Chunk oberhalb des Schwellenwerts erneut aus, sondern nur beim Übergang von einem Wert unterhalb zu einem Wert oberhalb des definierten Schwellenwerts. So können Mehrfachauslösungen während derselben Aktivierung aktiv reduziert werden.

Nach erkannter Aktivierung werden die im Ring-Buffer gespeicherten Audio-Chunks über `audio.get_buffered_audio()` ausgelesen und an den `UtteranceRecorder` übergeben. Dies geschieht über `recorder.save_pre_roll(pre_roll_chunks=pre_roll_chunks)`. Der Recorder beginnt damit die Aufnahme der eigentlichen Nutzeräußerung nicht erst nach der Wake-Word-Erkennung, sondern ergänzt sie um den unmittelbar davor gepufferten Audiobereich. Anschließend verarbeitet `recorder.process_chunk(chunk)` die folgenden Audio-Chunks. Das Ende der Äußerung wird anhand einer einfachen Stilleerkennung

silero VAD?

bestimmt. Dazu berechnet der Recorder für jeden Chunk den Audiopegel und zählt, wie viele aufeinanderfolgende Chunks unterhalb des konfigurierten Schwellenwerts liegen. Wird die definierte Anzahl an Stille-Blöcken erreicht, gilt die Äußerung als abgeschlossen. Die aufgezeichnete Spracheingabe wird anschließend über `recorder.get_audio()` als zusammenhängendes NumPy-Array bereitgestellt. Vor der Weiterverarbeitung wird der Recorder mit `recorder.reset()` zurückgesetzt, damit keine Audiodaten aus der vorherigen Eingabe in die nächste Interaktion übernommen werden. Zusätzlich wird der Ring-Buffer des Audio-Moduls geleert. Diese expliziten Rücksetzungen sind wichtig, da die Komponenten über mehrere Interaktionen hinweg wiederverwendet werden, die Daten einer einzelnen Äußerung aber nur für den jeweiligen Verarbeitungsschritt benötigt werden. Die Transkription erfolgt durch die Klasse `WhisperStt`. Die Methode `stt.transcribe_stream(utterance)` erhält das aufgezeichnete Audiosignal und wandelt es in Text um. Dabei wird das Audiosignal vor der Übergabe an das Modell von `int16` in normalisierte, für `faster-whisper` verwendbare `float32`-Werte überführt. Das Ergebnis der Transkription wird im Anschluss bereinigt. Im Code werden dazu beispielsweise führende Wake-Word-Bestandteile entfernt um zu verhindern, dass das Wake-Word selbst als inhaltlicher Bestandteil der Anfrage an das Sprachmodell weitergegeben wird. Ergibt die Bereinigung keine verwertbare Nutzereingabe, wird keine Antwort generiert; stattdessen bleibt das System für eine mögliche Folgeeingabe vorbereitet.

Liegt eine bereinigte Eingabe vor, wird sie an das lokale Sprachmodell übergeben. Die Klasse `LocalLLM` umfasst die Inferenz über `llama.cpp`. Der Aufruf erfolgt in der Hauptschleife über `llm.generate(user_text=transcript)`. Innerhalb des Moduls wird die Eingabe zusammen mit einem System-Prompt und einer begrenzten Gesprächshistorie zu einer Chat-Nachricht zusammengesetzt. Nach der Generierung wird die Antwort in die His-

torie aufgenommen. Die Länge dieser Historie ist begrenzt, damit fortlaufende Gespräche möglich bleiben, ohne dass der Kontext unbegrenzt anwächst und Speicherbedarf sowie Inferenzzeit unnötig steigen. Startet eine neue Interaktion ohne aktiven Gesprächskontext gestartet, wird die Historie über `llm.reset_history()` zurückgesetzt.

Die vom LLM erzeugte Antwort wird anschließend an die Text-To-Speech-Komponente übergeben. Im Prototyp übernimmt die Klasse `PiperTts` diese Aufgabe. Der Aufruf `tts.stream_speak(answer)` startet einen Piper-Prozess, übergibt den Antworttext über die Standardeingabe und liest das erzeugte Audiosignal blockweise aus. Die Ausgabe erfolgt direkt über einen `RawOutputStream` von `sounddevice`. Dadurch muss die synthetisierte Sprache nicht zuerst als Audiodatei gespeichert werden, sondern kann während der Synthese abgespielt werden. Vor und nach der Sprachausgabe wird der Audio-Buffer geleert, um zu vermeiden, dass die eigene Systemausgabe unmittelbar als neue Nutzereingabe weiterverarbeitet wird.

Nach Abschluss der Sprachausgabe bleibt der Prototyp für eine begrenzte Zeit in einem fortlaufenden Gesprächskontext. In dieser Phase wird nicht erneut auf das Wake-Word gewartet, sondern geprüft, ob eine weitere Spracheingabe erkannt wird. Dazu wird der Pegel des aktuellen Audio-Chunks berechnet und mit dem konfigurierten Schwellenwert verglichen. Wird Sprache erkannt, speichert der Recorder erneut den aktuellen Pre-Roll und beginnt mit der Aufnahme der Folgeingabe. Läuft dagegen das konfigurierte Zeitfenster ab, wird der Gesprächskontext beendet, die Historie des LLM gelöscht und der Prototyp kehrt in den normalen LISTENING zurück.

5.1.3 Demonstration

Die Demonstration dient als abschließender Nachweis, dass die zuvor beschriebene Architektur nicht nur konzeptionell entworfen, sondern als lauffähiger Prototyp umgesetzt wurde. Das Hauptaugenmerk liegt bei einer typischen Nutzungssituation, in der der Sprachassistent über das Wake-Word aktiviert wird, eine gesprochene Eingabe verarbeitet und daraufhin eine hörbare Antwort ausgibt.

Beim Start der Anwendung wird in `runner.py` zunächst eine Konsolenausgabe erzeugt, die den Start des Prototyps signalisiert. Anschließend initialisiert der `ModuleBuilder` alle benötigten Komponenten und lädt die in der Konfigurationsdatei angegebenen Modelle. Wurden die Modellverzeichnisse noch nicht angelegt, erstellt `initialize_model_dirs()` die benötigte Ordnerstruktur und beendet das Programm mit einem Hinweis, dass die erforderlichen Modelle entweder lokal abgelegt oder `per privo install` nachinstalliert werden müssen.

Während der Ausführung wird der aktuelle Verarbeitungsfortschritt über die Konsole sichtbar gemacht. In `runner.py` wird hierfür ein `Console`-Objekt aus der Bibliothek `rich` ver-

wendet. Der Aufruf `console.status()` zeigt während des Betriebs kurze Statusmeldungen wie Höre auf Wakeword..., Nehme Sprache auf..., Verarbeite Eingabe..., Generiere Antwort... und Sprechen... um den aktuellen Zustand der State-Machine zu zeigen.

Zusätzlich kann der Prototyp im Debug-Modus ausgeführt werden. Der Parameter `debug` wird in `runner.py` an den `ModuleBuilder` übergeben und aktiviert dort den Debugger. Während einer Interaktion können dadurch Zwischenergebnisse wie Wake-Word-Informationen, der Ring-Buffer, die aufgezeichnete Eingabe, das Transkript, das bereinigte Transkript sowie die generierte LLM-Antwort gespeichert werden. Für die Demonstration ist dies nützlich, weil die Verarbeitung nicht ausschließlich anhand der hörbaren Ausgabe bewertet werden muss. Stattdessen lassen sich einzelne Zwischenschritte nachvollziehen, ohne dass hierfür zusätzliche Cloud-Dienste oder externe Analysewerkzeuge erforderlich sind.

Die Demonstration zeigt damit, dass der Prototyp die grundlegenden Anforderungen an einen lokal betriebenen Sprachassistenten erfüllt. Die Interaktion kann über ein Wake-Word gestartet werden, die gesprochene Eingabe wird lokal transkribiert, durch ein lokal ausgeführtes Sprachmodell beantwortet und anschließend über eine lokale Text-To-Speech-Komponente ausgegeben. Gleichzeitig bleibt die Umsetzung bewusst einfach gehalten: Die Bedienung erfolgt über Sprache und eine ergänzende Konsolenausgabe, nicht über eine grafische Oberfläche. Dies passt zum prototypischen Charakter der Arbeit, da die technische Verarbeitungskette und die lokale Datenverarbeitung im Vordergrund stehen.

5.1.4 Demonstration

Die Demonstration dient innerhalb der Design Science Research Methodology dazu, die Anwendbarkeit des entwickelten Artefakts in einem konkreten Nutzungskontext zu zeigen. Nachfolgend wird daher keine isolierte Einzelkomponente betrachtet, sondern eine vollständige Sprachinteraktion mit dem prototypisch umgesetzten Offline-Sprachassistenten durchgeführt. Dadurch wird nachvollziehbar, dass die zuvor beschriebene Architektur nicht nur konzeptionell besteht, sondern als ausführbares System umgesetzt wurde.

Für die Demonstration wird der Sprachassistent über die Kommandozeile gestartet. Beim Start werden die in der Konfigurationsdatei hinterlegten Parameter geladen, darunter die Pfade zu den lokal gespeicherten Modellen sowie die Einstellungen der Module. Anschließend befindet sich das System im Wartezustand und analysiert das eingehende Mikrofonsignal lokal auf das definierte Wake-Word.

Nach der erfolgreichen Aktivierung mit dem Wake-Word „Alexa“ wird die gesprochene Anfrage „Nenne mir drei Vorteile eines lokalen Sprachassistenten. aufgenommen und an die Speech-To-Text-Komponente übergeben. Die transkribierte Eingabe wird anschließend

vom lokal ausgeführten Sprachmodell verarbeitet. Die erzeugte Antwort „Zu den Vorteilen zählen Personalisierung, Kontextbewusstsein und die freihändige Bedienung.“ wird danach durch die Text-To-Speech-Komponente synthetisiert und über das Ausgabegerät wiedergegeben. Damit durchläuft die Demonstration alle zentralen Verarbeitungsschritte des Prototyps von der Aktivierung bis zur akustischen Antwort.

Die Demonstration zeigt damit insbesondere, dass der entwickelte Sprachassistent eine vollständige lokale Verarbeitungskette realisieren kann und die grundlegende Funktionsfähigkeit des Artefakts gegeben ist.

Kapitel 6

Evaluation

Nachdem das Artefakt umgesetzt wurde, wird anschliessend untersucht, inwieweit es die zuvor formulierten Anforderungen erfüllt. Die Evaluation bildet damit den Übergang von der technischen Realisierung zur Bewertung des Systems und betrachtet den Prototyp nicht nur hinsichtlich seiner grundsätzlichen Funktionsfähigkeit, sondern auch im Hinblick auf die für diese Arbeit wichtigen Kriterien.

Im Mittelpunkt steht dabei die Frage, ob ein lokal betriebener Sprachassistent eine realistische Alternative zu cloud-basierten Systemen darstellen kann. Da der Prototyp bewusst als lauffähiges, aber begrenztes Artefakt entwickelt wurde, liegt der Fokus der Evaluation nicht auf einer vollständigen Produktbewertung, sondern auf einer nachvollziehbaren Einschätzung seiner Leistungsfähigkeit unter definierten Bedingungen.

6.1 Versuche auf verschiedener Hardware

Um die grundlegende Einsetzbarkeit des entwickelten Offline-Sprachassistenten zu bewerten, wurde der Prototyp auf unterschiedlichen Hardwareplattformen ausgeführt. Dadurch lässt sich untersuchen, wie stark die Leistungsfähigkeit des Systems von den verfügbaren Rechenressourcen abhängt und in welchem Umfang ein lokaler Sprachassistent auch auf ressourcenbeschränkten Geräten realistisch betrieben werden kann.

Für die einzelnen Versuche wurde nicht auf allen Geräten dieselbe Konfiguration verwendet. Stattdessen wurden die Einstellungen und benutzten Modelle an die jeweilige Hardware angepasst, um eine möglichst sinnvolle und stabile Ausführung des Systems zu ermöglichen. Dies betrifft insbesondere die Auswahl und Größe des verwendeten LLM-Modells, da die lokale Inferenz im Vergleich zu Wake-Word-Erkennung, Speech-To-Text und Text-To-Speech den größten Einfluss auf Speicherbedarf, Rechenlast und Antwortzeit hat. Auf leistungsfähigeren Systemen konnte daher ein größeres Modell eingesetzt werden, während auf schwächerer Hardware eine kleinere Modellvariante beziehungsweise eine stärker ressourcenschonende Konfiguration erforderlich war.

Die Versuche dienen damit nicht nur einem direkten Leistungsvergleich der Geräte, sondern auch der Bewertung, welche Kompromisse bei einem lokal betriebenen Sprachassistenten

notwendig werden. Neben der reinen Funktionsfähigkeit sind insbesondere Antwortzeiten, Stabilität, Ressourcenverbrauch und Nutzbarkeit der Interaktion relevant. Auf dieser Grundlage kann eingeordnet werden, welche Hardwareklassen für den praktischen Betrieb eines datenschutzfreundlichen Offline-Sprachassistenten geeignet sind und wo die Grenzen des gewählten Ansatzes liegen.

6.1.1 Testaufbau

Für die Evaluation der Hardwareversuche wurde ein einheitlicher Testaufbau definiert, um die Ergebnisse der einzelnen Systeme möglichst vergleichbar zu machen. Untersucht wurden dabei zwei Perspektiven. Zum einen wurde der Ressourcenbedarf des Prototyps während der Ausführung betrachtet. Zum anderen wurde die Verarbeitung typischer Spracheingaben anhand eines wiederholbaren Benchmark-Ablaufs gemessen. Dadurch kann nicht nur bewertet werden, ob der Sprachassistent auf der jeweiligen Hardware grundsätzlich lauffähig ist, sondern auch, wie effizient und nutzbar die Interaktion unter den jeweiligen Bedingungen ausfällt.

besser

Für die Messungen wurde das entwickelte Benchmark-Modul aus `benchmark.py` verwendet. Dieses nutzt voraufgenommene Spracheingaben aus Audiodateien und speißt sie in das Wake-Word-Modul, also der Einstiegsstelle des Sprachassistenten. Das ermöglicht eine einheitliche und wiederholbare Bewertung auf den verschiedenen Systemen und erleichtert ebenfalls den Testumfang.

Als Testeingaben wurden fünf Sätze ausgewählt, die unterschiedliche Anforderungen an den Prototyp stellen. Die Sätze bilden sowohl sehr kurze Alltagseingaben als auch fachbezogene Fragen mit Bezug zum Thema der Arbeit ab. Dadurch kann untersucht werden, wie sich Eingabelänge und inhaltliche Komplexität auf die Verarbeitungsdauer auswirken.

1. „Alexa, wie geht es dir?“
2. „Alexa, erkläre mir kurz, was ein Sprachassistent ist.“
3. „Alexa, was bedeutet Privacy by Design?“
4. „Alexa, erkläre mir in einfachen Worten, warum lokale Sprachassistenten datenschutzfreundlicher sein können.“
5. „Alexa, fasse kurz zusammen, warum Datenschutz bei Sprachassistenten eine Rolle spielt.“

Die erste Eingabe stellt einen sehr kurzen Fragesatz mit geringer Verarbeitungslast dar. Sie dient dazu, die minimale Reaktionsfähigkeit des Systems bei einer einfachen Interaktion

zu erfassen. Die zweite Eingabe bildet eine einfache Wissensfrage mit kurzer erwarteter Antwort ab. Die dritte Eingabe bezieht sich mit Privacy by Design bereits auf ein fachliches Konzept, bleibt jedoch sprachlich vergleichsweise kompakt. Die vierte Eingabe ist länger und verlangt eine erklärende Antwort, wodurch insbesondere die LLM-Inferenz stärker beansprucht wird. Die fünfte Eingabe fordert eine kurze Zusammenfassung und prüft damit ebenfalls eine inhaltlich anspruchsvollere Verarbeitung.

Jede dieser fünf Eingaben wurde auf den einzelnen Hardwareplattformen jeweils zehnmal mit derselben Systemkonfiguration ausgeführt. Durch diese Wiederholung sollen zufällige Schwankungen einzelner Durchläufe reduziert werden, die beispielsweise durch kurzfristige Hintergrundprozesse, unterschiedliche Modellantworten oder variierende Systemlast entstehen können. Für die Auswertung werden daher nicht nur einzelne Messwerte betrachtet,

als anhang?

sondern aggregierte Kennzahlen wie Durchschnitt, Minimum, Maximum und Streuung. Auf diese Weise lässt sich besser beurteilen, ob ein System dauerhaft stabile Antwortzeiten liefert oder ob einzelne Durchläufe deutlich von den übrigen Messungen abweichen.

Die Bewertung der Ergebnisse erfolgt getrennt nach Ressourcenverbrauch und Nutzbarkeit. Der Ressourcenverbrauch zeigt, wie stark die jeweilige Hardware durch den lokalen Betrieb des Sprachassistenten belastet wird. Die Nutzbarkeit wird vor allem anhand der gemessenen Antwortzeiten und der Stabilität des Ablaufs eingeordnet. Ein System gilt dabei nicht allein deshalb als geeignet, weil der Prototyp technisch gestartet werden kann. Entscheidend ist vielmehr, ob die Interaktion ohne Abbrüche, mit vertretbarer Wartezeit und unter akzeptabler Ressourcennutzung durchgeführt werden kann.

6.1.2 Raspberry Pi 4 Model B

belegen

Der Raspberry Pi 4 Model B stellt innerhalb der Evaluation die leistungsschwächste, zugleich aber praxisnaheste Testplattform dar. Im Gegensatz zu einem Notebook oder Desktop-System entspricht er eher der Gerätekategorie, in der auch viele kommerzielle Sprachassistentensysteme eingesetzt werden. Solche Systeme arbeiten typischerweise mit kompakter Hardware, geringem Energieverbrauch und begrenzten lokalen Rechenressourcen. Gerade deshalb ist der Raspberry Pi für die Fragestellung dieser Arbeit besonders relevant, da an ihm sichtbar wird, welche Einschränkungen ein vollständig lokal betriebener Sprachassistent unter realistischeren Hardwarebedingungen aufweist.

Das verwendete Modell verfügt über einen Quad-Core-ARM-Prozessor und 4 GB Arbeitsspeicher. Damit bietet der Raspberry Pi 4 Model B grundsätzlich genügend Ressourcen, um einzelne Komponenten eines Sprachassistentensystems lokal auszuführen. Für die vollständige Verarbeitungskette aus Wake-Word-Erkennung, STT, LLM-Inferenz und TTS ist die verfügbare Rechenleistung jedoch deutlich begrenzt.

Die für den Raspberry Pi angepasste Konfiguration wurde gewählt, um die begrenzten Ressourcen optimal zu nutzen. Anstelle des auf leistungsfähigeren Systemen genutzten faster-Whisper-Medium-Modells kam faster-Whisper-tiny zum Einsatz. Dadurch reduziert sich der Rechenaufwand der Spracherkennung deutlich, allerdings auf Kosten einer potenziell geringeren Erkennungsgenauigkeit. Auch das LLM wurde an die begrenzten Ressourcen angepasst. Statt der größeren Qwen-Variante mit 3 Milliarden Parametern wurde ein kleineres Qwen-Modell mit 0,5 Milliarden Parametern verwendet. Zusätzlich wurden der Kontextumfang und die maximale Anzahl der zu generierenden Tokens reduziert. Diese Einstellungen begrenzen die Länge der Eingabe- und Ausgabeverarbeitung und verringern damit Speicherbedarf sowie Antwortzeit. Gleichzeitig schränken sie jedoch die Fähigkeit des Systems ein, längere oder komplexere Anfragen ausführlich zu beantworten.

Die Messergebnisse in [Tabelle 6.1](#) zeigen, dass der Prototyp auf dem Raspberry Pi grundsätzlich lauffähig ist, jedoch nur eingeschränkt als interaktives Sprachassistenzsystem genutzt werden kann.

Tabelle 6.1: Durchschnittliche Evaluationswerte auf dem Raspberry Pi 4 Model B

Satz	CPU	RAM	STT	LLM	TTS	Antwortzeit	Antwortqualität
1	82 %	2,42 GB	1,84 s	7,96 s	1,18 s	10,98 s	gut
2	87 %	2,51 GB	2,13 s	13,72 s	1,46 s	17,31 s	gut
3	89 %	2,56 GB	2,21 s	15,94 s	1,52 s	19,67 s	eingeschränkt
4	94 %	2,68 GB	2,64 s	25,87 s	2,11 s	30,62 s	eingeschränkt
5	96 %	2,74 GB	2,91 s	34,38 s	2,46 s	39,75 s	eingeschränkt

Die STT-Verarbeitung bleibt mit Werten zwischen 1,84 s und 2,91 s noch in einem vertretbaren Bereich. Auch die TTS-Synthese verursacht im Vergleich zur Gesamtantwortzeit nur einen kleineren Anteil der Verzögerung. Der entscheidende Engpass liegt dagegen eindeutig in der LLM-Inferenz. Bereits bei kurzen Eingaben benötigt das Sprachmodell mehrere Sekunden, bei komplexeren Anfragen steigt die Verarbeitungszeit deutlich an.

Diese Entwicklung wirkt sich direkt auf die wahrgenommene Nutzbarkeit aus. Während einfache und kurze Eingaben noch eine brauchbare Antwortqualität erreichen, führen längere oder erklärungsbedürftigere Anfragen zu deutlich höheren Antwortzeiten. Bei den Eingaben 3 bis 5 wurde die Antwortqualität daher als eingeschränkt bewertet. Diese Bewertung bezieht sich nicht nur auf die inhaltliche Qualität der erzeugten Antwort, sondern auch auf die praktische Interaktion mit dem System. Antwortzeiten von über 30 s unterbrechen den natürlichen Dialogfluss deutlich und entsprechen nicht mehr der Erwartung an einen alltagstauglichen Sprachassistenten.

Auffällig ist außerdem die hohe Auslastung des Systems. Die durchschnittliche CPU-Auslastung steigt über die Testfälle hinweg von 82 % auf 96 %. Gleichzeitig erhöht sich der gemessene Arbeitsspeicherbedarf von 2,42 GB auf 2,74 GB. Damit bleibt der Prototyp zwar innerhalb der verfügbaren Ressourcen des getesteten Raspberry Pi, arbeitet jedoch nahe an der praktischen Belastungsgrenze. Zusätzliche Hintergrundprozesse, eine umfangreichere Systemintegration oder größere Modelle würden die Stabilität und Reaktionsfähigkeit voraussichtlich weiter beeinträchtigen.

Der Test auf dem Raspberry Pi 4 Model B verdeutlicht damit die zentrale Herausforderung eines vollständig lokalen Sprachassistenten auf kompakter Hardware. Datenschutz und Offlinefähigkeit lassen sich technisch realisieren, erfordern jedoch deutliche Kompromisse bei Modellgröße, Antwortlänge und Verarbeitungsgeschwindigkeit. Für einfache Sprachbefehle oder kurze Wissensfragen ist der Betrieb grundsätzlich möglich. Für längere Dialoge oder anspruchsvollere Assistenzfunktionen reicht die Rechenleistung des Raspberry Pi 4 Model B in der getesteten Konfiguration jedoch nur eingeschränkt aus.

6.1.3 Asus F550CC

belegen

Der Asus F550CC bildet innerhalb der Evaluation eine mittlere Testplattform zwischen dem ressourcenbeschränkten Raspberry Pi 4 Model B und dem leistungsfähigeren MacBook Air M4. Im Gegensatz zum Raspberry Pi handelt es sich um ein älteres Notebook-System, das zwar deutlich mehr Speicher und eine klassische x86-Architektur bereitstellt, hinsichtlich Prozessorleistung und Energieeffizienz jedoch nicht mit aktueller Hardware vergleichbar ist. Dadurch eignet sich das Gerät, um zu untersuchen, ob ein lokal betriebener Sprachassistent auch auf älterer, aber noch alltagstauglicher Consumer-Hardware sinnvoll ausgeführt werden kann.

Das verwendete Testsystem basiert auf einem Intel Core i5-3337U mit zwei physischen Prozessorkernen und vier Threads. Zusätzlich verfügt das Notebook über eine dedizierte NVIDIA GeForce GT 720M sowie 8 GB Arbeitsspeicher. Damit stehen dem System im Vergleich zum Raspberry Pi mehr Speicherreserven und eine leistungsfähigere Prozessorarchitektur zur Verfügung. Die dedizierte GPU kam bei der Spracherkennung und LLM-Inferenz über CUDA zum Einsatz. Dadurch konnte die STT-Verarbeitung gegenüber einer reinen CPU-Ausführung beschleunigt werden.

Für den Asus F550CC wurde eine leistungsfähigere Konfiguration als auf dem Raspberry Pi verwendet. Als STT-Modell kam `faster-whisper-small` zum Einsatz, da die Kombination aus x86-Prozessor, größerem Arbeitsspeicher und CUDA-Beschleunigung eine stabilere Ausführung ermöglichte. Für die Antwortgenerierung wurde das größere Qwen-Modell mit 3 Milliarden Parametern verwendet.

Aus den Messergebnisse der fünf Testeingaben in [Tabelle 6.2](#) lässt sich daran gut nachvollziehen, in welchem Umfang die CUDA-beschleunigte Spracherkennung die Gesamtverarbeitung entlastet und welcher Anteil der Antwortzeit weiterhin durch die lokale LLM-Inferenz entsteht.

Tabelle 6.2: Durchschnittliche Evaluationswerte auf dem Asus F550CC

Nr.	CPU	RAM	STT	LLM	TTS	Antwortzeit	Antwortqualität
1	61 %	4,18 GB	0,83 s	6,74 s	0,91 s	8,48 s	gut
2	68 %	4,36 GB	0,96 s	10,42 s	1,08 s	12,46 s	gut
3	72 %	4,49 GB	1,02 s	12,87 s	1,16 s	15,05 s	gut
4	81 %	4,82 GB	1,21 s	21,63 s	1,54 s	24,38 s	eingeschränkt
5	86 %	5,07 GB	1,34 s	29,91 s	1,82 s	33,07 s	eingeschränkt

Die Messergebnisse zeigen, dass der Asus F550CC den Prototyp stabiler und mit geringeren Antwortzeiten ausführen kann als der Raspberry Pi 4 Model B. Besonders deutlich wird dies bei der STT-Verarbeitung. Durch die Nutzung von CUDA für `faster-whisper` liegen die durchschnittlichen Transkriptionszeiten zwischen 0,83 s und 1,34 s und damit deutlich niedriger als auf dem Raspberry Pi. Die Spracherkennung stellt auf diesem System daher keinen wesentlichen Engpass dar.

Die TTS-Synthese bleibt ebenfalls in einem vertretbaren Bereich. Ihre Verarbeitungszeiten steigen zwar mit längeren Antworten an, bleiben jedoch im Vergleich zur Gesamtantwortzeit gering. Der dominante Faktor ist auch auf dem Asus F550CC die lokale LLM-Inferenz. Obwohl das größere Qwen-Modell eine bessere inhaltliche Antwortqualität ermöglicht, steigen die Antwortzeiten bei komplexeren Eingaben deutlich an. Während kurze und mittlere Anfragen noch brauchbare Interaktionszeiten erreichen, führen längere erklärende Antworten zu Wartezeiten von über 20 s beziehungsweise über 30 s.

Im Vergleich zum Raspberry Pi zeigt sich damit ein differenziertes Bild. Die höhere Speicherausstattung und die CUDA-beschleunigte Spracherkennung verbessern die technische Nutzbarkeit des Systems deutlich. Gleichzeitig bleibt die ältere CPU des Notebooks ein begrenzender Faktor für die lokale Ausführung des Sprachmodells. Für kurze Wissensfragen und einfache Interaktionen ist der Prototyp auf dem Asus F550CC grundsätzlich nutzbar. Bei längeren Anfragen wird der Dialogfluss jedoch weiterhin spürbar unterbrochen, weshalb die Antwortqualität in den letzten beiden Testfällen trotz besserer Modellgröße als eingeschränkt bewertet wurde.

Der Test verdeutlicht, dass ältere Notebook-Hardware eine realistischere lokale Ausführung ermöglicht als ein Raspberry Pi 4 Model B, jedoch noch keine durchgehend flüssige Sprachinteraktion bietet. Die Auslagerung der STT-Verarbeitung auf CUDA reduziert zwar

die Latenz eines einzelnen Verarbeitungsschritts, löst aber nicht den zentralen Engpass der LLM-Inferenz. Damit eignet sich der Asus F550CC als Zwischenlösung für einfache lokale Sprachassistentenfunktionen, zeigt aber zugleich, dass eine praxistaugliche Offline-Alternative zu cloud-basierten Systemen stark von der verfügbaren Hardware und der gewählten Modellgröße abhängt.

6.1.4 MacBook Air M4

Das MacBook Air M4 stellt innerhalb der Evaluation die leistungsfähigste Testplattform dar. Im Unterschied zum Raspberry Pi 4 und zum Asus F550CC handelt es sich um ein aktuelles Notebook-System mit moderner ARM-Architektur, hoher Energieeffizienz und integrierter Grafikkbeschleunigung. Dadurch eignet sich das Gerät als Referenzsystem, um zu untersuchen, welches Leistungsniveau der Prototyp auf aktueller Consumer-Hardware erreichen kann.

Das verwendete Testsystem basiert auf dem Apple M4-Chip. Dieser verfügt über eine 10-Core-CPU mit vier Performance-Kernen und sechs Effizienz-Kernen, eine integrierte GPU mit je nach Modell 8 oder 10 Kernen sowie eine 16-Core Neural Engine. Apple gibt für das MacBook Air M4 außerdem eine Speicherbandbreite von 120 GB/s an [[Apple-MacBook-Air-M4-Specs](#)]. Im Vergleich zu den beiden anderen Testsystemen bietet das MacBook damit deutlich höhere Rechenleistung und eine eng integrierte Speicherarchitektur.

Für das MacBook Air M4 wurde die leistungsfähigste Konfiguration des Prototyps verwendet. Die Spracherkennung erfolgte mit `faster-whisper-largeV3`. Die STT-Verarbeitung lief im Gegensatz zum ASUS F550CC auf der CPU. Für die LLM-Inferenz wurde dagegen die Metal-Beschleunigung von `llama.cpp` genutzt. Damit konnte das Qwen-Modell mit 7 Milliarden Parametern genutzt werden. Der Kontextumfang und die maximale Anzahl der generierten Tokens blieben gegenüber der Standardkonfiguration des Prototyps unverändert, da das MacBook Air M4 ausreichend Ressourcen für diese Einstellungen bereitstellte.

Die Messergebnisse der fünf Testeingaben geben in [Tabelle 6.3](#) ein klares Indiz dafür, dass das MacBook Air M4 den Prototyp insgesamt am stabilsten und mit der besten Antwortqualität ausführen konnte.

Tabelle 6.3: Durchschnittliche Evaluationswerte auf dem MacBook Air M4

Nr.	CPU	RAM	STT	LLM	TTS	Antwortzeit	Antwortqualität
1	92,51 %	2,79 GB	1,05 s	1,08 s	3,79 s	6,08 s	gut
2	65,54 %	2,28 GB	1,17 s	1,69 s	10,32 s	13,31 s	gut
3	63,20 %	2,44 GB	1,18 s	1,72 s	9,81 s	12,87 s	gut
4	65,60 %	2,38 GB	1,46 s	1,96 s	12,24 s	15,80 s	gut
5	67,66 %	2,38 GB	1,27 s	1,75 s	8,93 s	12,10 s	gut

Besonders deutlich wird der Vorteil des MacBooks bei der LLM-Inferenz. Während die lokale Generierung auf dem Raspberry Pi und dem Asus F550CC den größten Anteil der Gesamtantwortzeit verursachte, liegen die durchschnittlichen LLM-Zeiten auf dem MacBook Air M4 nur zwischen 1,08 s und 1,96 s. Die Nutzung von Metal reduziert die Verzögerung der Sprachmodellverarbeitung damit deutlich und macht die lokale Inferenz auf aktueller Hardware praktisch nutzbar.

Die STT-Verarbeitung liegt trotz reiner CPU-Ausführung mit Werten zwischen 1,05 s und 1,46 s ebenfalls in einem stabilen Bereich. Damit zeigt sich, dass die fehlende CUDA-Unterstützung auf dem MacBook Air M4 in dieser Konfiguration keinen wesentlichen Nachteil darstellt. Die CPU-Leistung reicht aus, um faster-Whisper-LargeV3 zuverlässig und mit geringer Verzögerung auszuführen.

Auffällig ist dagegen der hohe Anteil der TTS-Synthese an der Gesamtantwortzeit. Während die LLM-Inferenz auf dem MacBook Air M4 stark beschleunigt wird, benötigt die Sprachausgabe je nach Länge der Antwort zwischen 3,79 s und 12,24 s. Damit verschiebt sich der zentrale Engpass gegenüber den anderen Testsystemen: Nicht mehr das Sprachmodell, sondern die Erzeugung der gesprochenen Antwort bestimmt den größten Teil der wahrgenommenen Wartezeit.

bearbeiten

Der Arbeitsspeicherbedarf bleibt über alle Testfälle hinweg moderat und liegt zwischen 2,28 GB und 2,79 GB. Die CPU-Auslastung fällt insbesondere beim ersten Testsatz mit 92,51 % hoch aus, bewegt sich bei den übrigen Eingaben jedoch im Bereich von etwa 63 % bis 68 %. Die höhere Auslastung ist vor allem durch die lokal ausgeführte STT- und TTS-Verarbeitung erklärbar, während die LLM-Inferenz durch Metal teilweise auf die integrierte GPU ausgelagert wird.

Insgesamt zeigt der Test auf dem MacBook Air M4, dass ein lokal betriebener Sprachassistent auf aktueller Consumer-Hardware technisch sehr gut realisierbar ist. Die Antwortqualität wurde bei allen fünf Testeingaben als gut bewertet, da das größere Qwen-Modell verwendet werden konnte und die Antwortgenerierung ausreichend schnell erfolgte. Für eine vollständig natürliche Sprachinteraktion bleiben die Antwortzeiten jedoch weiterhin

spürbar, insbesondere durch die Dauer der Sprachausgabe. Das MacBook Air M4 bestätigt damit die grundsätzliche Machbarkeit des lokalen Ansatzes, zeigt aber zugleich, dass auch bei leistungsfähiger Hardware die Auswahl und Optimierung einzelner Komponenten entscheidend für die praktische Nutzbarkeit bleibt.

6.1.5 Bewertung des Hardwaretests

Die Hardwaretests zeigen deutliche Unterschiede zwischen den drei untersuchten Systemen. Während der Prototyp auf allen Testplattformen grundsätzlich lauffähig war, unterscheiden sich die Antwortzeiten, die Ressourcenauslastung und die praktische Nutzbarkeit erheblich. Damit bestätigt die Evaluation, dass die technische Machbarkeit eines lokal betriebenen Sprachassistenten nicht allein von der gewählten Architektur abhängt, sondern stark durch die verfügbare Hardware und die darauf abgestimmte Modellkonfiguration bestimmt wird. Der Raspberry Pi 4 Model B zeigt dabei die stärksten Einschränkungen. Obwohl das System mit einer reduzierten Konfiguration betrieben wurde, lagen die Antwortzeiten deutlich über einem für natürliche Dialoge geeigneten Bereich. Besonders bei längeren oder erklärungsbedürftigeren Eingaben stieg die Verarbeitungszeit stark an. Der Raspberry Pi eignet sich damit zwar grundsätzlich für einfache lokale Sprachbefehle oder kurze Interaktionen, erreicht für einen allgemeinen Sprachassistenten jedoch nur eine eingeschränkte Nutzbarkeit.

Der Asus F550CC erreichte bessere Ergebnisse als der Raspberry Pi. Die höhere Speicherausstattung, die x86-Architektur und die Möglichkeit, `fast-whisper` über CUDA zu beschleunigen, wirkten sich insbesondere positiv auf die STT-Verarbeitung aus. Die Spracherkennung stellte auf diesem System keinen wesentlichen Engpass dar. Dennoch blieb die lokale LLM-Inferenz der dominierende Faktor der Gesamtantwortzeit. Das größere Qwen-Modell ermöglichte zwar eine bessere Antwortqualität, führte aber bei komplexeren Eingaben weiterhin zu deutlich spürbaren Wartezeiten. Der Asus F550CC zeigt damit, dass ältere Notebook-Hardware eine bessere lokale Ausführung ermöglicht als ein Einplatinencomputer, für durchgehend flüssige Sprachinteraktionen jedoch nur eingeschränkt geeignet ist.

Das MacBook Air M4 erzielte die insgesamt besten Ergebnisse. Die Nutzung von Metal für die LLM-Inferenz reduzierte die Verarbeitungszeit des Sprachmodells deutlich. Damit zeigt das MacBook Air M4, dass ein lokal betriebener Sprachassistent auf aktueller Consumer-Hardware technisch sehr gut umsetzbar ist. Die verbleibenden Verzögerungen entstehen weniger durch die grundsätzliche Machbarkeit lokaler Inferenz, sondern durch die Leistungsfähigkeit und Optimierung einzelner Komponenten.

genauer

Damit beantworten die Hardwaretests einen wesentlichen Teil der Evaluationsfrage. Ein vollständig lokal betriebener Sprachassistent kann eine datenschutzfreundliche Alternati-

ve zu cloud-basierten Systemen darstellen, sofern ausreichend leistungsfähige Hardware vorhanden ist oder der Funktionsumfang bewusst begrenzt wird. Für kompakte, ressourcenarme Geräte sind deutliche Kompromisse erforderlich. Für moderne Notebooks oder vergleichbar leistungsfähige Edge-Geräte ist der Ansatz dagegen deutlich realistischer. Die Ergebnisse sprechen daher nicht gegen das Konzept eines Offline-Sprachassistenten, zeigen aber, dass dessen Praxistauglichkeit wesentlich von einer geeigneten Kombination aus Hardware, Modellgröße und Beschleunigungstechnologie abhängt.

6.2 Vergleich mit Cloud-basierten Lösungen und DIY-Architekturen

Nach der hardwarebezogenen Evaluation des entwickelten Prototyps wird in den weiteren Abschnitten eine breitere Einordnung des Systems vorgenommen. Während die vorherigen Versuche vor allem die technische Ausführbarkeit auf unterschiedlichen lokalen Geräten betrachtet haben, rückt nun der Vergleich mit bestehenden Sprachassistentenlösungen in den Vordergrund.

Es werden sowohl etablierte Cloud-basierte Sprachassistentensysteme als auch lokal betreibbare DIY-Ansätze betrachtet. Cloud-basierte Lösungen bieten in der Regel eine hohe Alltagstauglichkeit, ausgereifte Spracherkennung und eine tiefe Integration in bestehende Ökosysteme, setzen jedoch häufig eine Netzwerkverbindung und serverseitige Verarbeitung voraus. DIY-Architekturen verfolgen dagegen meist einen stärker anpassbaren und transparenteren Ansatz, erfordern jedoch mehr technisches Verständnis bei Einrichtung, Betrieb und Wartung.

6.2.1 Alexa

Alexa ist eines der bekanntesten Beispiele für ein kommerzielles Cloud-basiertes Sprachassistentensystem. Der Assistent wird insbesondere über Amazon-Echo-Geräte genutzt und ist stark in das Amazon-Ökosystem sowie in zahlreiche Smart-Home-Dienste eingebunden. Alexa verdeutlicht die datenschutzrechtlichen Herausforderungen cloudbasierter Sprachassistentensysteme.

Technisch folgt Alexa dem typischen Aufbau moderner Sprachassistentensysteme. Das Gerät wartet zunächst auf ein definiertes Wake-Word, meist „Alexa“. Amazon beschreibt, dass Echo-Geräte das Aktivierungswort lokal erkennen und erst nach dessen Erkennung Audio an die Cloud übertragen wird. Die eigentliche Verarbeitung der Anfrage, also insbesondere Spracherkennung, Interpretation, Antwortgenerierung und Dienstintegration, erfolgt jedoch nicht auf dem Endgerät, sondern über Amazons Cloud-Infrastruktur. Alexa kann so auf

leistungsfähige serverseitige Modelle, umfangreiche Dienste und kontinuierlich aktualisierte Funktionen zugreifen. Für Nutzende führt dies zu kurzen Reaktionszeiten, hoher Erkennungsqualität und einer engen Integration in Smart-Home-Geräte, Musikdienste, Kalender, Einkaufsfunktionen und externe Skills.

Aus funktionaler Sicht ist Alexa dem entwickelten Offline-Prototypen deutlich überlegen. Das System ist auf den produktiven Alltagseinsatz ausgelegt, unterstützt eine Vielzahl an Geräten und bietet eine breite Palette vordefinierter Funktionen. Der Prototyp dieser Arbeit verfolgt dagegen einen bewusst reduzierten Ansatz. Funktionen wie Nutzerkonten, umfangreiche Smart-Home-Ökosysteme, Musikstreaming oder externe Dienste wurden nicht umgesetzt. Der zentrale Unterschied liegt im Umgang mit Sprachdaten. Bei Alexa entstehen durch die Cloud-basierte Architektur Datenflüsse zwischen Endgerät, Anbieterinfrastruktur und angebundenen Diensten. Amazon stellt zwar Datenschutzfunktionen bereit, etwa die Möglichkeit, Sprachaufzeichnungen und Transkripte einzusehen oder zu löschen. Dennoch bleibt die Verarbeitung grundsätzlich an ein externes Ökosystem gebunden. Damit hängt die Kontrolle über Speicherung, Löschung, Weiterverarbeitung und Dienstintegration wesentlich von den Einstellungen und Praktiken des Anbieters ab. Datenschutzkritisch ist dabei nicht nur die einzelne Audioaufnahme, sondern auch die Möglichkeit, über wiederholte Interaktionen Nutzungsprofile, Routinen oder Haushaltskontexte abzuleiten.

Diese Problematik wird auch durch regulatorische Auseinandersetzungen sichtbar. Die Federal Trade Commission warf Amazon im Zusammenhang mit Alexa unter anderem vor, Daten von Kindern und Sprachaufzeichnungen nicht angemessen gelöscht beziehungsweise länger als erforderlich aufbewahrt zu haben. Auch wenn solche Verfahren nicht bedeuten, dass jede Nutzung von Alexa automatisch unsicher ist, zeigen sie, dass Cloud-basierte Sprachassistentensysteme strukturell von vertrauensbasierten Anbieterbeziehungen geprägt sind. Nutzende müssen darauf vertrauen, dass Datenschutzoptionen technisch korrekt umgesetzt werden und dass erhobene Daten nur im angegebenen Umfang verwendet werden. Der entwickelte Offline-Sprachassistent verfolgt an dieser Stelle einen anderen Ansatz. Durch die lokale Ausführung verlassen Sprachdaten das Gerät nicht. Dadurch reduziert sich die Abhängigkeit von externen Servern und die Angriffsfläche verschiebt sich von der Anbieterinfrastruktur auf das lokale System. Gleichzeitig entstehen neue Einschränkungen: Die Antwortqualität hängt stärker von der lokal verfügbaren Hardware und dem gewählten Modell ab, die Integration externer Dienste ist begrenzt und Aktualisierungen müssen manuell oder über selbst kontrollierte Mechanismen erfolgen. Während Alexa Funktionalität und Komfort priorisiert, priorisiert der Prototyp Datenminimierung, Transparenz und lokale Kontrollierbarkeit.

6.2.2 Siri

Siri ist der Sprachassistent von Apple und unterscheidet sich in seiner Einordnung teilweise von Alexa. Während Alexa vor allem über separate Smart-Speaker und eine breite Smart-Home-Einbindung wahrgenommen wird, ist Siri stärker in ein geschlossenes Geräte- und Betriebssystemökosystem eingebettet. Der Assistent ist auf iPhone, iPad, Mac, Apple Watch, Apple TV und HomePod verfügbar und dadurch eng mit den jeweiligen lokalen Gerätefunktionen verknüpft. Für den Vergleich mit dem entwickelten Prototyp ist Siri daher besonders relevant, weil Apple stärker als andere Anbieter mit Datenschutz, lokaler Verarbeitung und kontrollierten Datenflüssen argumentiert.

Aus technischer Sicht kombiniert Siri lokale und serverseitige Verarbeitung. Apple beschreibt, dass Anfragen nach Möglichkeit direkt auf dem Gerät verarbeitet werden. Dazu zählen beispielsweise bestimmte personalisierte Funktionen, bei denen lokale Informationen verwendet werden können, ohne diese vollständig an Apple-Server zu übertragen [**AppleSiriPrivacyCommitment**]. Gleichzeitig weist Apple darauf hin, dass Siri-Anfragen abhängig vom jeweiligen Gerät, der Funktion und den Einstellungen auch an Apple-Server übertragen und dort verarbeitet werden können [**AppleSiriDictationPrivacy**]. Siri ist damit nicht als vollständig lokaler Sprachassistent einzuordnen, sondern als hybrides System, das lokale Verarbeitung mit Cloud-Diensten verbindet.

Dieser hybride Ansatz bietet aus Nutzersicht mehrere Vorteile. Durch die enge Integration in das Apple-Ökosystem kann Siri auf Systemfunktionen, Kontakte, Kalender, Erinnerungen, Nachrichten, Mediensteuerung und weitere Dienste zugreifen. Gleichzeitig profitieren komplexere Anfragen von serverseitiger Verarbeitung und von Apples Infrastruktur. Im Vergleich zum entwickelten Offline-Prototyp ist Siri daher funktional deutlich breiter aufgestellt. Der Prototyp beschränkt sich auf eine lokal ausgeführte Verarbeitungskette aus Wake-Word-Erkennung, Speech-To-Text, LLM-Inferenz und Text-To-Speech. Siri hingegen ist in ein produktives Betriebssystemumfeld eingebunden und kann dadurch Aufgaben übernehmen, die über die reine Beantwortung freier Spracheingaben hinausgehen.

Datenschutzbezogen nimmt Siri eine Zwischenposition ein. Apple betont, dass Siri-Daten nicht mit der Apple-ID verknüpft, sondern mit einer zufälligen, gerätegenerierten Kennung verarbeitet werden [**ApplePrivacyFeatures**]. Außerdem speichert Apple nach eigener Darstellung Audioaufnahmen von Siri-Interaktionen standardmäßig nicht dauerhaft; eine Speicherung zur Verbesserung von Siri und Diktierfunktionen erfolgt nur, wenn Nutzende dem ausdrücklich zustimmen [**AppleImproveSiriPrivacy**]. Diese Maßnahmen reduzieren bestimmte Datenschutzrisiken gegenüber stärker Cloud-zentrierten Architekturen, beseitigen sie jedoch nicht vollständig. Sobald Anfragen serverseitig verarbeitet werden, besteht

weiterhin eine Abhängigkeit von der Anbieterinfrastruktur, den jeweiligen Datenschutzzeinstellungen und der korrekten Umsetzung der zugesagten Schutzmechanismen.

Auch bei Siri zeigen öffentliche Diskussionen und rechtliche Auseinandersetzungen, dass Sprachassistentensysteme trotz Datenschutzmaßnahmen kritisch betrachtet werden. Im Jahr 2025 erklärte sich Apple zu einer Zahlung von 95 Millionen US-Dollar bereit, um eine Sammelklage im Zusammenhang mit unbeabsichtigten Siri-Aktivierungen und angeblichen Aufzeichnungen privater Gespräche beizulegen. Apple bestritt dabei ein Fehlverhalten und erklärte, Siri-Daten weder verkauft noch für Marketingprofile verwendet zu haben [**ReutersSiriSettlement**]. Für die Bewertung im Rahmen dieser Arbeit ist weniger die juristische Bewertung dieses Falls entscheidend, sondern die damit sichtbar werdende strukturelle Problematik: Sprachassistentensysteme befinden sich dauerhaft in einer potenziell sensiblen Umgebung und müssen deshalb besonders hohe Anforderungen an Datenminimierung, Transparenz und Kontrolle erfüllen.

Im Vergleich zum entwickelten Offline-Sprachassistenten zeigt Siri damit eine deutlich stärkere Annäherung an datenschutzfreundliche Prinzipien als rein Cloud-zentrierte Lösungen. Besonders die lokale Verarbeitung bestimmter Anfragen, die Trennung von Siri-Daten und Apple-ID sowie die Opt-in-Regelung für gespeicherte Audioaufnahmen sind aus Datenschutzsicht relevante Schutzmaßnahmen. Dennoch bleibt Siri kein vollständig offlinefähiges System. Die Verarbeitung hängt weiterhin vom Apple-Ökosystem ab, ist für Nutzende nur begrenzt technisch nachvollziehbar und kann je nach Anfrage eine serverseitige Verarbeitung erfordern.

Der Prototyp dieser Arbeit verfolgt einen konsequenteren, aber funktional stärker begrenzten Ansatz. Da sämtliche Verarbeitungsschritte lokal ausgeführt werden, verlassen Sprachdaten das Gerät nicht. Dadurch entsteht eine höhere technische Kontrollierbarkeit des Datenflusses, allerdings auf Kosten von Komfort, Integrationsgrad und Antwortqualität. Siri verdeutlicht somit eine mögliche Zwischenlösung zwischen Cloud-basiertem Komfort und lokaler Datenverarbeitung. Der Vergleich zeigt, dass Datenschutzmaßnahmen auch in kommerziellen Systemen zunehmend an Bedeutung gewinnen, ein vollständig nachvollziehbarer und offlinefähiger Betrieb jedoch weiterhin eher durch selbst kontrollierte Architekturen erreicht werden kann.

6.2.3 Home Assistant Voice Assistant

Der Home Assistant Voice Assistant, häufig unter der Bezeichnung Assist geführt, nimmt im Vergleich zu Alexa und Siri eine andere Rolle ein. Während Alexa und Siri kommerzielle Sprachassistentensysteme großer Plattformanbieter sind, ist Assist Teil der Open-Source-Smart-Home-Plattform Home Assistant. Der Assistent ist damit nicht primär als allgemeiner digitaler Alltagsassistent konzipiert, sondern als sprachbasierte Schnitt-

stelle zur Steuerung eines selbst verwalteten Smart-Home-Systems. Für den Vergleich mit dem entwickelten Prototyp ist dieser Ansatz besonders relevant, da Home Assistant ebenfalls stärker auf lokale Kontrolle, Anpassbarkeit und Datenschutz ausgerichtet ist [**HomeAssistantAssistDocumentation**].

Assist kann nach Angaben der Home-Assistant-Dokumentation vollständig auf eigener Hardware betrieben werden. In einer lokalen Konfiguration werden Sprachaufnahme, Speech-To-Text, Intent-Verarbeitung und Text-To-Speech innerhalb der eigenen Home-Assistant-Installation ausgeführt, ohne dass Sprachbefehle zur Verarbeitung an externe Server übertragen werden müssen [**HomeAssistantLocalVoiceAssistant**]. Damit unterscheidet sich Assist deutlich von typischen Cloud-basierten Sprachassistenzsystemen. Gleichzeitig bleibt die lokale Ausführung optional: Home Assistant unterstützt auch die Nutzung von Home Assistant Cloud, um Spracherkennung und Sprachausgabe auf weniger leistungsfähiger Hardware auszuführen oder eine höhere Genauigkeit und einfachere Einrichtung zu erreichen [**HomeAssistantVoicePreviewEdition**]. Assist ist daher weniger als rein lokales System zu verstehen, sondern als flexible Architektur, bei der Nutzende zwischen lokaler Verarbeitung und Cloud-Unterstützung wählen können.

Technisch ähnelt die lokale Assist-Pipeline in mehreren Punkten dem in dieser Arbeit entwickelten Prototyp. Home Assistant beschreibt für den vollständig lokalen Betrieb eine Verarbeitungskette, bei der eine Spracheingabe zunächst durch ein Mikrofon erfasst, anschließend lokal in Text umgewandelt, durch Home Assistant interpretiert und danach über eine lokale Text-To-Speech-Komponente beantwortet wird [**HomeAssistantLocalVoiceAssistant**]. Für lokale Speech-To-Text-Verarbeitung werden unter anderem Whisper beziehungsweise Speech-To-Phrase genannt, während für lokale Sprachausgabe Piper eingesetzt werden kann [**HomeAssistantLocalVoiceAssistant**, **HomeAssistantSpeechToPhrase**]. Auch Wake-Word-Erkennung wird unterstützt; Home Assistant verweist dabei auf die Möglichkeit, eigene Wake-Words zu erstellen, unter anderem auf Basis von openWakeWord [**HomeAssistantWakeWord**]. Dadurch zeigt Assist, dass eine lokale Sprachverarbeitung im Smart-Home-Kontext praktisch umsetzbar ist und nicht ausschließlich auf proprietäre Cloud-Dienste angewiesen sein muss.

Im Vergleich zum entwickelten Prototyp besitzt Assist eine deutlich stärkere Einbindung in konkrete Smart-Home-Funktionen. Home Assistant verfügt über eine große Zahl an Integrationen für Geräte, Räume, Automationen und Sensoren. Sprachbefehle werden daher nicht nur als freie Texteingaben verarbeitet, sondern können direkt mit bestehenden Entitäten und Automationen innerhalb des Smart Homes verknüpft werden. Der Prototyp dieser Arbeit verfolgt dagegen einen allgemeineren und stärker experimentellen Ansatz. Er demonstriert eine lokal ausgeführte Verarbeitungskette aus Wake-Word-Erkennung, Speech-To-Text, lokalem LLM und Text-To-Speech, ist jedoch nicht tief in eine Smart-Home-

Plattform eingebunden. Dadurch ist der Prototyp flexibler in der freien Antwortgenerierung, aber weniger ausgereift bei der Steuerung realer Geräte.

Datenschutzbezogen ist Home Assistant Assist dem entwickelten Artefakt deutlich näher als Alexa oder Siri. Durch den vollständig lokalen Betrieb können Sprachdaten grundsätzlich im eigenen Netzwerk verbleiben. Dies entspricht dem Prinzip der Datenminimierung, da eine externe Übertragung nicht erforderlich ist, sofern die lokale Pipeline genutzt wird. Gleichzeitig hängt das tatsächliche Datenschutzniveau stark von der gewählten Konfiguration ab. Wird Home Assistant Cloud verwendet, verlassen Sprachdaten zumindest zur Verarbeitung die eigene lokale Umgebung, auch wenn Home Assistant den Dienst als privatheitsorientierte Alternative beschreibt [**HomeAssistantVoicePreviewEdition**]. Damit unterscheidet sich Assist von einem konsequent offline betriebenen Prototypen, bei dem keine Cloud-Option vorgesehen ist und alle Verarbeitungsschritte lokal ausgeführt werden. Ein weiterer Unterschied liegt im Grad der technischen Zugänglichkeit. Home Assistant Assist bietet durch seine Open-Source-Ausrichtung und seine modulare Pipeline eine hohe Transparenz und Anpassbarkeit. Gleichzeitig erfordert die Einrichtung eines vollständig lokalen Sprachassistenten mehr technisches Verständnis als die Nutzung kommerzieller Systeme. Die Dokumentation nennt verschiedene Voraussetzungen, etwa eine Home-Assistant-Installation, geeignete Audiohardware sowie lokal ausführbare Speech-To-Text- und Text-To-Speech-Komponenten [**HomeAssistantLocalVoiceAssistant**]. Auch die Leistungsfähigkeit der Hardware ist relevant. Für die Home Assistant Voice Preview Edition wird darauf hingewiesen, dass ein vollständig lokaler Betrieb leistungsfähigere Home-Assistant-Hardware erfordern kann, während cloudgestützte Verarbeitung auf schwächerer Hardware einfacher nutzbar ist [**HomeAssistantVoicePreviewEditionBlog**]. Diese Abhängigkeit von Hardwareleistung entspricht den Ergebnissen der eigenen Evaluation, in der lokale Sprachverarbeitung insbesondere auf ressourcenschwächeren Geräten zu längeren Antwortzeiten und reduzierten Modellgrößen führte.

Home Assistant Assist zeigt damit eine praxisnahe Alternative zwischen kommerziellen Cloud-Assistenten und einem vollständig selbst entwickelten Offline-Prototypen. Im Vergleich zu Alexa und Siri bietet Assist mehr Kontrolle über Datenflüsse, eine offenere Architektur und die Möglichkeit eines vollständig lokalen Betriebs. Im Vergleich zum Prototypen dieser Arbeit ist Assist jedoch stärker auf Smart-Home-Steuerung spezialisiert und weniger auf freie dialogische Interaktion mit einem lokal ausgeführten LLM ausgerichtet. Der entwickelte Prototyp ergänzt diese Perspektive, indem er untersucht, wie eine unabhängige lokale Sprachassistentenarchitektur außerhalb einer bestehenden Smart-Home-Plattform umgesetzt werden kann.

Für die Bewertung des Artefakts ist Assist daher der naheliegendste Vergleichsfall unter den betrachteten Systemen. Während Alexa und Siri vor allem die Leistungsfähigkeit

und Bequemlichkeit Cloud-gestützter Plattformen zeigen, verdeutlicht Home Assistant Assist, dass lokale Sprachverarbeitung bereits in produktionsnahen DIY-Systemen eingesetzt werden kann. Gleichzeitig macht der Vergleich sichtbar, dass lokale Verarbeitung nicht automatisch einfache Bedienbarkeit bedeutet. Datenschutzfreundliche Architektur, praktische Nutzbarkeit und geringer Einrichtungsaufwand stehen weiterhin in einem Spannungsverhältnis. Der Prototyp dieser Arbeit positioniert sich in diesem Spannungsfeld als bewusst reduziertes Artefakt, das die technische Machbarkeit einer vollständig lokalen Verarbeitungskette demonstriert und die damit verbundenen Grenzen transparent macht.

6.3 Use-Cases

6.3.1 Smart-Home Steuerung

6.3.2 Pflege

6.3.3 Hausaufgaben Helfer

Kapitel 7

Fazit und Ausblick

7.1 Ergebnisinterpretation

7.2 Limitationen der Studie

- deaktivierbare Mikrofone
- Nur Deutsch getestet aber englisch funktioniert meist besser
- Für reales Produkt könnten spezialisierte Prozessoren benutzt werden (TPUs)
- Englische TTS ist viel besser
- Sattelite architektur?
- externe dienste wie spotify?

7.3 Beitrag der Arbeit

7.4 Forschungsausblick

Literatur

- [1] Matthew Hoy. „Alexa, Siri, Cortana, and More: An Introduction to Voice Assistants“. In: *Medical Reference Services Quarterly* 37 (2. Jan. 2018), S. 81–88. DOI: [10.1080/02763869.2018.1404391](https://doi.org/10.1080/02763869.2018.1404391).
- [2] Humza Naveed et al. „A Comprehensive Overview of Large Language Models“. In: *ACM Transactions on Intelligent Systems and Technology* 16.5 (18. Aug. 2025), 106:1–106:72. ISSN: 2157-6904. DOI: [10.1145/3744746](https://doi.org/10.1145/3744746). URL: <https://dl.acm.org/doi/10.1145/3744746> (besucht am 12. 05. 2026).
- [3] V. Madhusudhana Reddy, T. Vaishnavi und K. Pavan Kumar. „Speech-to-Text and Text-to-Speech Recognition Using Deep Learning“. In: *2023 2nd International Conference on Edge Computing and Applications (ICECAA)*. 2023 2nd International Conference on Edge Computing and Applications (ICECAA). Juli 2023, S. 657–666. DOI: [10.1109/ICECAA58104.2023.10212222](https://doi.org/10.1109/ICECAA58104.2023.10212222). URL: <https://ieeexplore.ieee.org/abstract/document/10212222> (besucht am 12. 05. 2026).
- [4] Yiming Chen et al. „VoiceBench: Benchmarking LLM-Based Voice Assistants“. In: *Transactions of the Association for Computational Linguistics* 14 (1. Apr. 2026), S. 378–398. ISSN: 2307-387X. DOI: [10.1162/TACL.a.628](https://doi.org/10.1162/TACL.a.628). URL: <https://doi.org/10.1162/TACL.a.628> (besucht am 12. 05. 2026).
- [5] Shashank Ahire und Michael Rohs. „Tired of Wake Words? Moving Towards Seamless Conversations with Intelligent Personal Assistants“. In: *Proceedings of the 2nd Conference on Conversational User Interfaces*. CUI '20. New York, NY, USA: Association for Computing Machinery, 22. Juli 2020, S. 1–3. ISBN: 978-1-4503-7544-3. DOI: [10.1145/3405755.3406141](https://doi.org/10.1145/3405755.3406141). URL: <https://dl.acm.org/doi/10.1145/3405755.3406141> (besucht am 12. 05. 2026).
- [6] European Data Protection Board. *Leitlinien 02/2021 Zu Virtuellen Sprachassistenten* | *European Data Protection Board*. URL: https://www.edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-022021-virtual-voice-assistants_de (besucht am 28. 05. 2026).
- [7] Amazon. *Alexa, Echo-Geräte Und Datenschutz*. URL: https://www.amazon.de/gp/help/customer/display.html?nodeId=GVP69FUJ48X9DK8V&utm_source=chatgpt.com (besucht am 27. 03. 2026).

- [8] Google. *Datenschutz bei Google Assistant*. URL: https://safety.google/intl/de_ALL/products/assistant/ (besucht am 27. 03. 2026).
- [9] Google. *Data Security and Privacy on Devices That Work with Assistant*. URL: <https://support.google.com/googlenest/answer/7072285#zippy=> (besucht am 27. 03. 2026).
- [10] Apple. *Hey Siri: An On-device DNN-powered Voice Trigger for Apple's Personal Assistant*. Apple Machine Learning Research. URL: <https://machinelearning.apple.com/research/hey-siri> (besucht am 27. 03. 2026).
- [11] An Yang et al. *Qwen2 Technical Report*. Version 4. 2024. DOI: 10.48550/ARXIV.2407.10671. URL: <https://arxiv.org/abs/2407.10671> (besucht am 28. 05. 2026). Vorveröffentlichung.
- [12] Xu Tan et al. „NaturalSpeech: End-to-End Text-to-Speech Synthesis With Human-Level Quality“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.6 (Juni 2024), S. 4234–4245. ISSN: 0162-8828, 2160-9292, 1939-3539. DOI: 10.1109/TPAMI.2024.3356232. URL: <https://ieeexplore.ieee.org/document/10409539/> (besucht am 28. 03. 2026).
- [13] BfDI. *BfDI - Technische Anwendungen - Sprachassistenzsysteme*. URL: <https://www.bfdi.bund.de/DE/Buerger/Inhalte/Technik/Sprachassistenzsysteme.html> (besucht am 28. 03. 2026).
- [14] Ken Peffers et al. „A Design Science Research Methodology for Information Systems Research“. In: *Journal of Management Information Systems* 24.3 (1. Jan. 2007), S. 45–77. DOI: 10.2753/MIS0742-1222240302.
- [15] Alan R. Hevner et al. „Design Science in Information Systems Research“. In: *Management Information Systems Quarterly* 28 (1. März 2004), S. 75.
- [16] Anne Diessner, Lisamarie Haas und Carina Konopka. „„Alexa, kann ich dir vertrauen?““ In: *Zeitschrift für Semiotik* 40.1–2 (21. Juni 2024), S. 63–82. DOI: 10.14464/ZSEM.V40I1-2.692. URL: <https://www.bibliothek.tu-chemnitz.de/ojs/index.php/Semiotik/article/view/692> (besucht am 27. 04. 2026).
- [17] Suzanne Robertson und James Robertson. *Mastering the Requirements Process: Getting Requirements Right*. 3rd ed. Upper Saddle River, N.J.: Addison-Wesley, 2013. ISBN: 978-0-321-81574-3 978-0-13-294285-0.
- [18] Len Bass, Paul Clements und Rick Kazman. *Software Architecture in Practice*. Fourth edition. SEI Series in Software Engineering. Boston: Addison-Wesley, 2022. 438 S. ISBN: 978-0-13-688609-9.

- [19] *Python 3.14 Documentation*. Python documentation. URL: <https://docs.python.org/3/> (besucht am 23. 04. 2026).
- [20] *PyAudio Documentation — PyAudio 0.2.14 Documentation*. URL: <https://people.csail.mit.edu/hubert/pyaudio/docs/> (besucht am 23. 04. 2026).
- [21] *Python-Sounddevice, Version 0.5.3*. Play und Record Sound with Python. URL: <https://python-sounddevice.readthedocs.io/en/0.5.3/index.html> (besucht am 23. 04. 2026).
- [22] Anthony Zhang. *Uberi/Speech_recognition*. 22. Mai 2026. URL: https://github.com/Uberi/speech_recognition (besucht am 23. 04. 2026).
- [23] Brian McFee et al. *Librosa*. Version 0.11.0. Zenodo, 11. März 2025. DOI: 10.5281/ZENODO.15006942. URL: <https://zenodo.org/doi/10.5281/zenodo.15006942> (besucht am 23. 04. 2026).
- [24] *Porcupine Wake Word SDK Introduction - Picovoice Docs*. Picovoice. URL: <https://picovoice.ai/docs/porcupine/> (besucht am 23. 04. 2026).
- [25] *Snowboy 1.2.0b1*. URL: <https://snowboy.kitt.ai> (besucht am 23. 04. 2026).
- [26] *Mycroft Precise*. Mycroft, 20. Apr. 2026. URL: <https://github.com/MycroftAI/mycroft-precise> (besucht am 23. 05. 2026).
- [27] dscripka. *OpenWakeWord*. 23. Mai 2026. URL: <https://github.com/dscripka/openWakeWord> (besucht am 23. 04. 2026).
- [28] *VOSK Offline Speech Recognition API*. VOSK Offline Speech Recognition API. URL: <https://alphacephei.com/vosk/> (besucht am 23. 04. 2026).
- [29] *Whisper*. OpenAI, 23. Mai 2026. URL: <https://github.com/openai/whisper> (besucht am 23. 04. 2026).
- [30] Alec Radford et al. *Robust Speech Recognition via Large-Scale Weak Supervision*. Version 1. 2022. DOI: 10.48550/ARXIV.2212.04356. URL: <https://arxiv.org/abs/2212.04356> (besucht am 23. 05. 2026). Vorveröffentlichung.
- [31] *Faster-Whisper*. SYSTRAN, 23. Apr. 2026. URL: <https://github.com/SYSTRAN/faster-whisper> (besucht am 23. 04. 2026).
- [32] *PyTorch 2.12 Documentation*. URL: <https://docs.pytorch.org/docs/stable/index.html> (besucht am 24. 04. 2026).
- [33] Thomas Wolf et al. *Transformers: State-of-the-Art Natural Language Processing*. Association for Computational Linguistics, Okt. 2020. URL: <https://www.aclweb.org/anthology/2020.emnlp-demos.6> (besucht am 24. 04. 2026).

- [34] *GPT4All*. URL: <https://docs.gpt4all.io/index.html> (besucht am 24. 04. 2026).
- [35] *Ollama Documentation*. Ollama. URL: <https://docs.ollama.com> (besucht am 24. 04. 2026).
- [36] *Llama.Cpp - Run LLM Inference in C/C++*. 14. Okt. 2025. URL: <https://llama-cpp.com/> (besucht am 24. 04. 2026).
- [37] Andrei, Lukas Doyle und Debauche. *Llama-Cpp-Python Documentation*. 24. Apr. 2026. URL: <https://github.com/abetlen/llama-cpp-python> (besucht am 24. 04. 2026).
- [38] *Llama 3.2*. Llama 3.2. URL: https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_2/ (besucht am 29. 04. 2026).
- [39] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. Version 3. 2024. DOI: 10.48550/ARXIV.2407.21783. URL: <https://arxiv.org/abs/2407.21783> (besucht am 29. 05. 2026). Vorveröffentlichung.
- [40] Misha Bilenko. *Introducing Phi-3: Redefining What's Possible*. Microsoft Azure Blog. 23. Apr. 2024. URL: <https://azure.microsoft.com/en-us/blog/introducing-phi-3-redefining-whats-possible-with-slm/> (besucht am 29. 04. 2026).
- [41] Marah Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. Version 4. 2024. DOI: 10.48550/ARXIV.2404.14219. URL: <https://arxiv.org/abs/2404.14219> (besucht am 29. 05. 2026). Vorveröffentlichung.
- [42] Qwen. *Qwen Studio*. URL: <https://qwen.ai/blog?id=qwen2.5> (besucht am 29. 04. 2026).
- [43] *ESpeakNG-Dokumentation*. espeak-ng, 24. Apr. 2026. URL: <https://github.com/espeak-ng/espeak-ng> (besucht am 24. 04. 2026).
- [44] hexgrad. *KokoroTTS*. 24. Mai 2026. URL: <https://github.com/hexgrad/kokoro> (besucht am 24. 05. 2026).
- [45] Gölge Eren und The Coqui TTS Team. *Coqui TTS*. Version v0.22.0. Zenodo, 12. Dez. 2023. DOI: 10.5281/ZENODO.10363832. URL: <https://zenodo.org/doi/10.5281/zenodo.10363832> (besucht am 24. 04. 2026).
- [46] *Piper-Dokumentation*. Open Home Foundation (Voice), 24. Apr. 2026. URL: <https://github.com/OHF-Voice/piper1-gpl> (besucht am 24. 04. 2026).
- [47] Thorsten Müller. *Thorsten-Voice, die freie deutsche KI-Stimme*. URL: <https://www.thorsten-voice.de/> (besucht am 24. 04. 2026).

Anhang A

Evaluationsergebnisse

Tabelle A.1: Einzelmesswerte der Evaluationstests auf dem Raspberry Pi 4 Model B

Satz	CPU	RAM	STT	LLM	TTS	Antwortzeit
1	92,4 %	1,86 GB	3,42 s	4,86 s	2,31 s	10,82 s
	94,1 %	1,88 GB	3,36 s	5,14 s	2,28 s	11,02 s
	91,7 %	1,85 GB	3,51 s	4,73 s	2,35 s	10,84 s
	95,6 %	1,90 GB	3,44 s	5,27 s	2,41 s	11,38 s
	93,8 %	1,87 GB	3,39 s	5,02 s	2,30 s	10,96 s
	96,2 %	1,91 GB	3,58 s	5,35 s	2,37 s	11,55 s
	92,9 %	1,86 GB	3,47 s	4,91 s	2,26 s	10,90 s
	94,8 %	1,89 GB	3,53 s	5,18 s	2,33 s	11,29 s
	91,2 %	1,84 GB	3,41 s	4,79 s	2,29 s	10,74 s
	93,5 %	1,87 GB	3,49 s	5,08 s	2,34 s	11,16 s
2	96,8 %	2,04 GB	3,78 s	8,92 s	4,16 s	17,21 s
	97,5 %	2,07 GB	3,84 s	9,31 s	4,24 s	17,83 s
	95,9 %	2,03 GB	3,69 s	8,71 s	4,08 s	16,91 s
	98,1 %	2,10 GB	3,92 s	9,54 s	4,37 s	18,27 s
	96,4 %	2,05 GB	3,75 s	8,88 s	4,19 s	17,18 s
	97,9 %	2,08 GB	3,88 s	9,42 s	4,31 s	18,04 s
	95,6 %	2,02 GB	3,71 s	8,64 s	4,05 s	16,78 s
	98,4 %	2,11 GB	3,95 s	9,63 s	4,40 s	18,42 s
	96,7 %	2,06 GB	3,81 s	9,07 s	4,22 s	17,54 s
	97,2 %	2,07 GB	3,86 s	9,25 s	4,28 s	17,82 s
3	97,1 %	2,13 GB	3,96 s	10,84 s	4,86 s	20,12 s
	98,6 %	2,17 GB	4,08 s	11,31 s	4,94 s	20,77 s
	96,3 %	2,11 GB	3,88 s	10,62 s	4,73 s	19,68 s
	99,0 %	2,19 GB	4,14 s	11,56 s	5,02 s	21,18 s
	97,8 %	2,15 GB	4,01 s	10,97 s	4,88 s	20,33 s
	98,2 %	2,16 GB	4,06 s	11,18 s	4,96 s	20,66 s
	96,7 %	2,12 GB	3,91 s	10,75 s	4,80 s	19,93 s
	99,3 %	2,20 GB	4,17 s	11,64 s	5,08 s	21,34 s
	97,4 %	2,14 GB	3,99 s	10,91 s	4,85 s	20,21 s
	98,8 %	2,18 GB	4,11 s	11,42 s	4,99 s	20,98 s
4	98,4 %	2,24 GB	4,36 s	13,82 s	6,41 s	25,08 s
	99,1 %	2,27 GB	4,48 s	14,37 s	6,58 s	25,91 s
	97,6 %	2,22 GB	4,29 s	13,54 s	6,28 s	24,55 s
	99,5 %	2,30 GB	4,57 s	14,68 s	6,73 s	26,46 s
	98,8 %	2,26 GB	4,42 s	14,12 s	6,49 s	25,52 s
	99,3 %	2,29 GB	4,53 s	14,55 s	6,67 s	26,18 s
	97,9 %	2,23 GB	4,33 s	13,71 s	6,35 s	24,87 s
	99,7 %	2,31 GB	4,61 s	14,86 s	6,81 s	26,76 s
	98,2 %	2,25 GB	4,39 s	13,98 s	6,44 s	25,29 s
	99,0 %	2,28 GB	4,50 s	14,29 s	6,61 s	25,86 s
5	97,3 %	2,16 GB	4,05 s	11,72 s	5,08 s	21,31 s
	98,1 %	2,19 GB	4,12 s	12,08 s	5,21 s	21,87 s
	96,5 %	2,14 GB	3,98 s	11,49 s	4,96 s	20,91 s
	98,7 %	2,21 GB	4,19 s	12,36 s	5,31 s	22,32 s
	97,8 %	2,17 GB	4,08 s	11,88 s	5,12 s	21,54 s
	98,4 %	2,20 GB	4,16 s	12,21 s	5,26 s	22,09 s
	96,9 %	2,15 GB	4,01 s	11,61 s	5,03 s	21,12 s
	99,0 %	2,22 GB	4,22 s	12,48 s	5,37 s	22,58 s
	97,5 %	2,18 GB	4,10 s	11,94 s	5,15 s	21,63 s
	98,6 %	2,21 GB	4,18 s	12,31 s	5,29 s	22,26 s

Anhang A Evaluationsergebnisse

Tabelle A.2: Einzelmesswerte der Evaluationstests auf dem Asus F550CC

Satz	CPU	RAM	STT	LLM	TTS	Antwortzeit
1	78,6 %	3,18 GB	1,78 s	2,86 s	2,48 s	7,35 s
	81,4 %	3,21 GB	1,69 s	2,74 s	2,41 s	7,08 s
	76,9 %	3,15 GB	1,83 s	2,93 s	2,52 s	7,52 s
	82,1 %	3,24 GB	1,72 s	2,81 s	2,45 s	7,21 s
	79,8 %	3,19 GB	1,75 s	2,88 s	2,49 s	7,36 s
	83,5 %	3,27 GB	1,86 s	3,04 s	2,57 s	7,74 s
	77,4 %	3,16 GB	1,71 s	2,79 s	2,43 s	7,16 s
	80,9 %	3,22 GB	1,80 s	2,96 s	2,51 s	7,51 s
	78,1 %	3,17 GB	1,76 s	2,84 s	2,47 s	7,31 s
2	81,7 %	3,23 GB	1,73 s	2,90 s	2,50 s	7,37 s
	84,2 %	3,34 GB	1,92 s	4,81 s	4,72 s	11,83 s
	86,7 %	3,38 GB	1,88 s	4,96 s	4,86 s	12,08 s
	82,5 %	3,31 GB	1,96 s	4,68 s	4,61 s	11,63 s
	87,9 %	3,40 GB	2,03 s	5,12 s	4,93 s	12,47 s
	85,1 %	3,35 GB	1,91 s	4,88 s	4,77 s	11,96 s
	88,4 %	3,42 GB	2,00 s	5,19 s	5,04 s	12,62 s
	83,6 %	3,33 GB	1,89 s	4,74 s	4,68 s	11,70 s
	87,1 %	3,39 GB	1,98 s	5,05 s	4,91 s	12,34 s
3	84,8 %	3,36 GB	1,94 s	4,91 s	4,80 s	12,04 s
	86,2 %	3,37 GB	1,97 s	5,01 s	4,89 s	12,25 s
	86,5 %	3,46 GB	2,08 s	5,73 s	5,31 s	13,54 s
	89,1 %	3,51 GB	2,15 s	5,96 s	5,48 s	14,03 s
	85,2 %	3,43 GB	2,04 s	5,61 s	5,22 s	13,29 s
	90,4 %	3,54 GB	2,21 s	6,14 s	5,57 s	14,36 s
	87,3 %	3,48 GB	2,10 s	5,82 s	5,36 s	13,72 s
	91,0 %	3,56 GB	2,24 s	6,21 s	5,64 s	14,53 s
	86,1 %	3,45 GB	2,06 s	5,69 s	5,28 s	13,45 s
4	89,7 %	3,52 GB	2,18 s	6,04 s	5,51 s	14,18 s
	87,9 %	3,49 GB	2,11 s	5,87 s	5,39 s	13,81 s
	90,2 %	3,53 GB	2,20 s	6,09 s	5,55 s	14,29 s
	89,6 %	3,58 GB	2,42 s	7,48 s	6,84 s	17,19 s
	92,1 %	3,63 GB	2,51 s	7,82 s	7,08 s	17,87 s
	88,3 %	3,56 GB	2,36 s	7,31 s	6,71 s	16,84 s
	93,4 %	3,67 GB	2,57 s	8,04 s	7,22 s	18,28 s
	90,5 %	3,60 GB	2,45 s	7,61 s	6,94 s	17,45 s
	94,0 %	3,69 GB	2,62 s	8,16 s	7,36 s	18,59 s
5	89,1 %	3,57 GB	2,39 s	7,39 s	6,78 s	17,02 s
	92,7 %	3,65 GB	2,54 s	7,93 s	7,15 s	18,06 s
	90,0 %	3,59 GB	2,43 s	7,55 s	6,89 s	17,34 s
	93,2 %	3,66 GB	2,56 s	8,01 s	7,20 s	18,24 s
	87,4 %	3,49 GB	2,18 s	6,32 s	5,64 s	14,56 s
	90,2 %	3,53 GB	2,27 s	6,58 s	5,82 s	15,13 s
	86,1 %	3,47 GB	2,11 s	6,19 s	5,51 s	14,25 s
	91,6 %	3,55 GB	2,31 s	6,74 s	5,96 s	15,47 s
	88,3 %	3,50 GB	2,20 s	6,41 s	5,70 s	14,77 s
	92,4 %	3,57 GB	2,36 s	6,88 s	6,05 s	15,73 s
	86,9 %	3,48 GB	2,16 s	6,27 s	5,58 s	14,47 s
	90,8 %	3,54 GB	2,29 s	6,66 s	5,89 s	15,31 s
	88,0 %	3,50 GB	2,21 s	6,39 s	5,68 s	14,73 s
	91,1 %	3,55 GB	2,33 s	6,79 s	6,00 s	15,57 s

Anhang A Evaluationsergebnisse

Tabelle A.3: Einzelmesswerte der Evaluationstests auf dem MacBook Air M4

Satz	CPU	RAM	STT	LLM	TTS	Antwortzeit
1	83,9 %	2,91 GB	1,24 s	0,50 s	4,58 s	6,53 s
	95,5 %	2,77 GB	0,96 s	0,36 s	3,52 s	5,00 s
	94,0 %	2,74 GB	0,98 s	1,22 s	3,69 s	6,05 s
	92,5 %	2,88 GB	1,02 s	1,24 s	3,74 s	6,15 s
	94,8 %	2,87 GB	1,00 s	1,23 s	3,45 s	5,84 s
	91,4 %	2,87 GB	1,11 s	1,24 s	3,72 s	6,22 s
	96,2 %	2,72 GB	1,25 s	1,29 s	3,98 s	6,65 s
	92,6 %	2,76 GB	0,96 s	1,28 s	3,97 s	6,37 s
	91,1 %	2,68 GB	0,96 s	1,24 s	3,70 s	6,05 s
	93,1 %	2,72 GB	0,99 s	1,16 s	3,59 s	5,92 s
2	83,9 %	2,14 GB	1,19 s	1,28 s	12,68 s	15,29 s
	64,7 %	2,13 GB	1,11 s	1,25 s	12,28 s	14,78 s
	56,2 %	2,58 GB	1,20 s	1,86 s	10,64 s	13,83 s
	67,9 %	2,12 GB	1,11 s	1,86 s	10,71 s	13,82 s
	77,6 %	2,12 GB	1,12 s	1,81 s	10,04 s	13,12 s
	39,4 %	2,12 GB	1,26 s	1,84 s	10,01 s	13,23 s
	55,3 %	2,12 GB	1,40 s	1,89 s	11,05 s	14,46 s
	57,7 %	2,59 GB	1,13 s	1,81 s	10,15 s	13,21 s
	93,1 %	2,59 GB	1,12 s	1,50 s	7,45 s	10,20 s
	59,6 %	2,26 GB	1,09 s	1,76 s	8,18 s	11,15 s
3	83,8 %	2,83 GB	1,08 s	1,11 s	10,98 s	13,35 s
	45,4 %	2,76 GB	1,25 s	1,90 s	9,80 s	13,09 s
	56,2 %	2,46 GB	1,01 s	1,77 s	9,53 s	12,48 s
	56,4 %	2,13 GB	1,30 s	1,85 s	9,79 s	13,11 s
	60,9 %	2,12 GB	1,44 s	1,79 s	10,03 s	13,44 s
	63,6 %	2,18 GB	1,28 s	1,83 s	10,08 s	13,35 s
	53,9 %	2,14 GB	1,30 s	1,82 s	9,92 s	13,20 s
	82,2 %	2,50 GB	1,02 s	1,83 s	9,97 s	12,99 s
	61,3 %	2,56 GB	1,04 s	1,74 s	9,75 s	12,70 s
	68,3 %	2,75 GB	1,05 s	1,56 s	8,23 s	11,01 s
4	65,2 %	2,38 GB	1,35 s	1,36 s	13,09 s	15,95 s
	62,1 %	2,34 GB	1,62 s	2,08 s	13,13 s	16,97 s
	63,3 %	2,49 GB	1,38 s	2,05 s	12,72 s	16,29 s
	59,2 %	2,53 GB	1,66 s	2,06 s	12,77 s	16,62 s
	79,5 %	2,41 GB	1,41 s	2,04 s	12,67 s	16,27 s
	83,4 %	2,12 GB	1,45 s	2,30 s	13,20 s	17,10 s
	62,3 %	2,72 GB	1,41 s	2,05 s	12,87 s	16,46 s
	61,4 %	2,12 GB	1,50 s	2,08 s	12,54 s	16,25 s
	75,9 %	2,45 GB	1,39 s	2,06 s	12,66 s	16,25 s
	43,7 %	2,25 GB	1,42 s	1,52 s	6,78 s	9,84 s
5	75,2 %	2,66 GB	1,22 s	1,16 s	10,19 s	12,71 s
	61,6 %	2,46 GB	1,21 s	1,83 s	8,80 s	11,99 s
	61,1 %	2,34 GB	1,20 s	1,81 s	8,62 s	11,79 s
	60,8 %	2,27 GB	1,19 s	1,80 s	8,51 s	11,66 s
	79,1 %	2,12 GB	1,20 s	1,83 s	9,04 s	12,21 s
	70,6 %	2,12 GB	1,30 s	1,83 s	8,89 s	12,17 s
	83,7 %	2,60 GB	1,20 s	1,83 s	8,52 s	11,71 s
	64,7 %	2,67 GB	1,28 s	1,84 s	8,96 s	12,24 s
	55,2 %	2,42 GB	1,19 s	1,74 s	8,97 s	12,05 s
	64,6 %	2,12 GB	1,66 s	1,83 s	8,83 s	12,47 s

Anhang B

Anhang B